

第6章 图 (P185)

6.1 图的定义和基本术语

6.3 图的抽象数据类型定义

6.4 图的存储结构

6.5 图的遍历

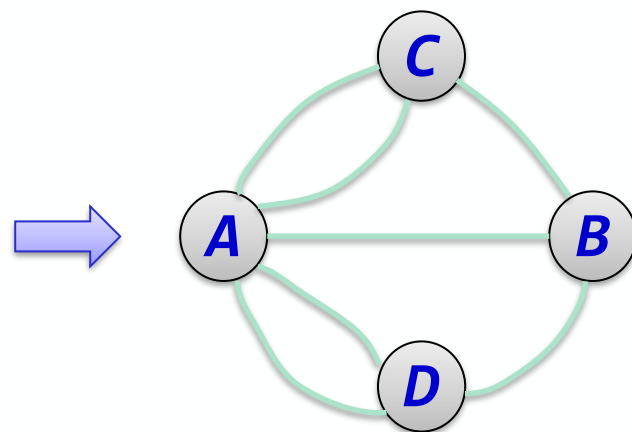
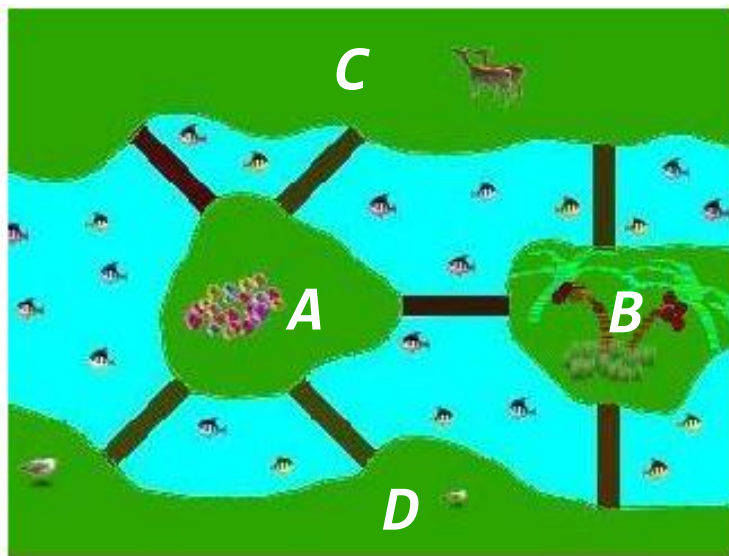
6.6 图的应用



教学目标

- 1.掌握：图的基本概念及相关术语和性质
- 2.熟练掌握：图的邻接矩阵和邻接表两种存储表示方法
- 3.熟练掌握：图的两种遍历方法DFS和BFS
- 4.熟练掌握：最短路径算法（Dijkstra算法）
- 5.掌握：最小生成树的两种算法（Prim和Kruskal）
- 6.掌握：拓扑排序算法的思想

6.1 图的定义和术语(P185)



Vertex Edge

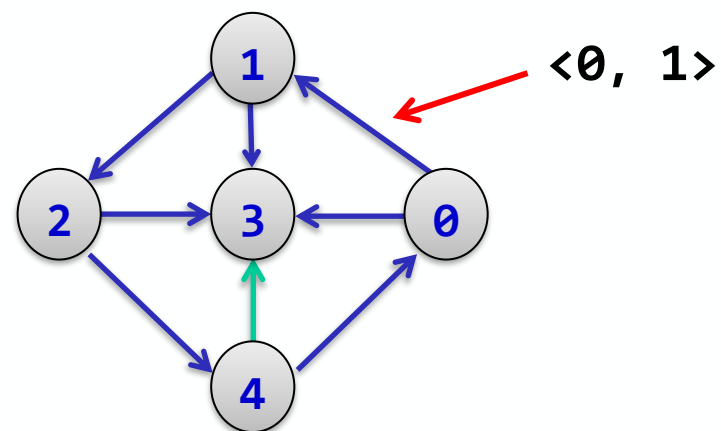
图 (Graph) : 一个图(G)定义为一个偶对(V,E) ,
记为 $G=(V,E)$

V(Vertex): 顶点(数据元素)的有穷非空集合;

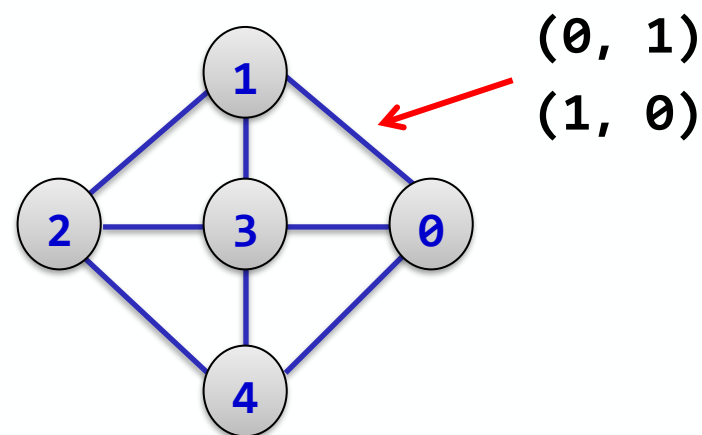
E(Edge): 边的有穷集合。

6.1 图的定义和术语(P185)

在图G中，若E是**有向边**（也称**弧**）的有限集合时，称G是**有向图**。用**尖括号**序偶表示有向边，弧 $\langle v, w \rangle$ 表示从顶点v到顶点w有一条**弧**。其中v是**弧尾**，w是**弧头**，弧是顶点的**有序对**。



若E是**无向边**（简称**边**）的有限集合时，称G是**无向图**。用**圆括号**序偶表示无向边，记为 (v, w) 或 (w, v) ， $(v, w) = (w, v)$ ，表示同一条边。边是顶点的**无序对**。



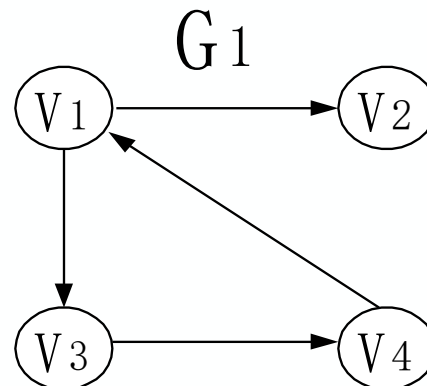
6.1 图的定义和术语(P185)

有向图： 弧的英文Arc

$$G_1=(V_1, A)$$

$$V_1=\{v_1, v_2, v_3, v_4\}$$

$$A=\{<v_1, v_2>, <v_1, v_3>, <v_3, v_4>, <v_4, v_1>\}$$

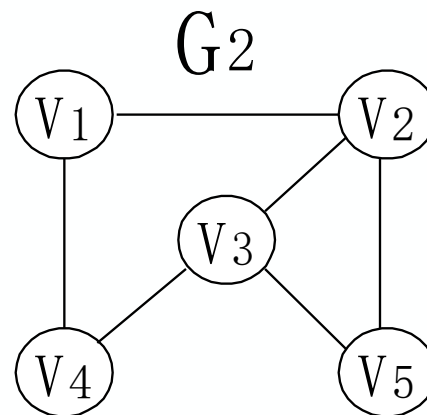


无向图： 边的英文Edge

$$G_2=(V_2, E)$$

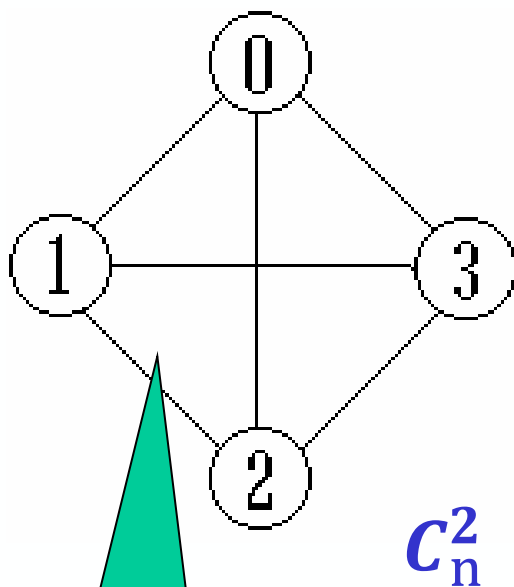
$$V_2=\{v_1, v_2, v_3, v_4, v_5\}$$

$$E=\{(v_1, v_2), (v_1, v_4), (v_2, v_3), (v_2, v_5), (v_3, v_4), (v_3, v_5)\}$$

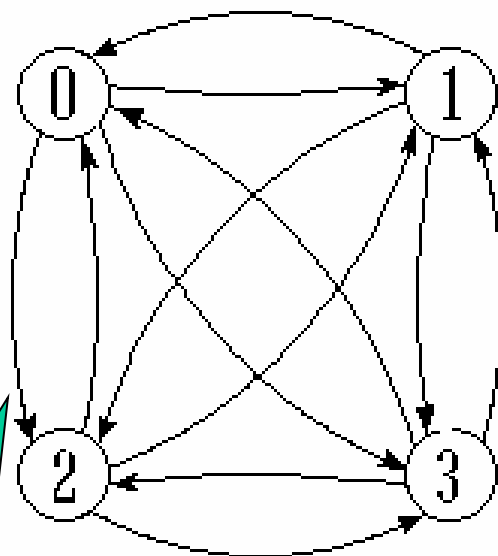


6.1 图的定义和术语(P185)

完全图：任意两个顶点都有边相连



无向完全图 $n(n-1)/2$ 条边



有向完全图 $n(n-1)$ 条弧

稀疏图：有很少的边或弧的图

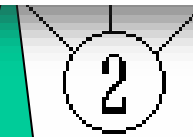
稠密图：有较多边或弧的图，几乎接近完全图

组合 (Combination) 的计算公式:

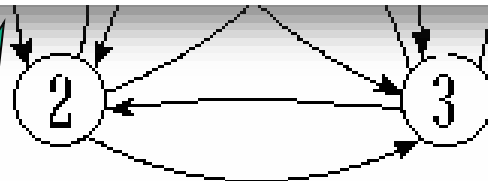
组合是指从 n 个不同元素中任取 m ($m \leq n$) 个元素, 无序地组成集合的操作, 这样的组合数可以使用下面的公式来计算:

$$\text{公式: } C(n, m) = n! / [m!(n - m)!]$$

其中, $n!$ 表示 n 的阶乘, 即 $n \times (n-1) \times (n-2) \times \dots \times 2 \times 1$, $m!$ 表示 m 的阶乘。


$$C_n^2$$

无向完全图 $n(n-1)/2$ 条边



有向完全图 $n(n-1)$ 条弧

稀疏图: 有很少的边或弧的图

稠密图: 有较多边或弧的图, 几乎接近完全图

【练习1】 设G是一个含有6个顶点的无向图，该图至多有（）条边。

A. 6

B. 10

C. 15

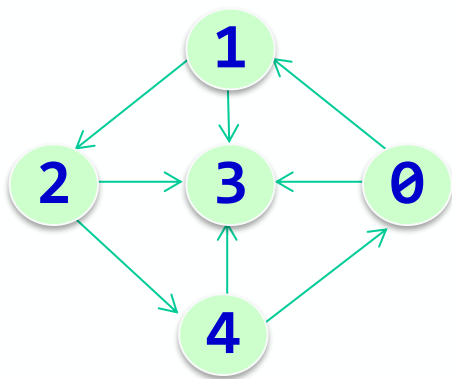
D. 20

答案：C

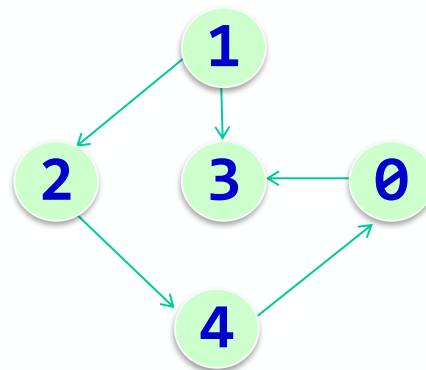
**解释：边数最多时为完全无向图，
有 $n(n-1)/2=15$ 条边。**

6.1 图的定义和术语(P185)

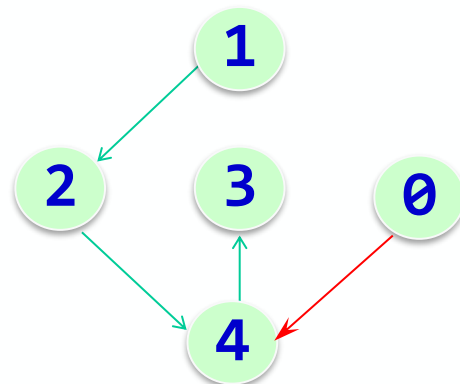
子图: 设有两个图 $G=(V, E)$ 和 $G'=(V', E')$,
若 $V' \subseteq V$, $E' \subseteq E$, 则称 G' 是 G 的**子图**。



是子图



不是子图



6.1 图的定义和术语(P185)

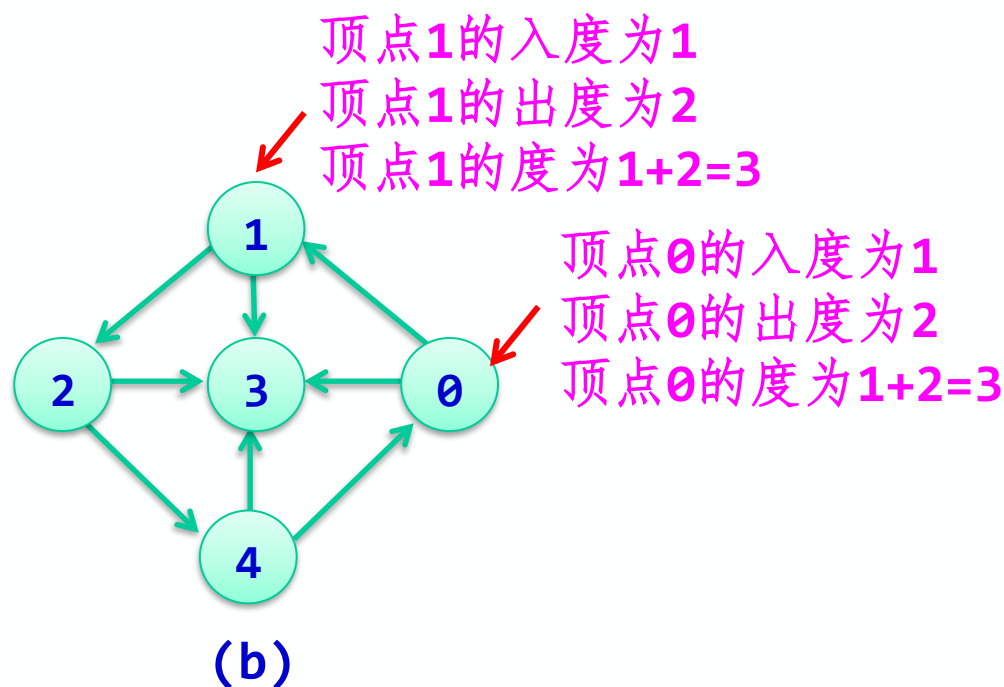
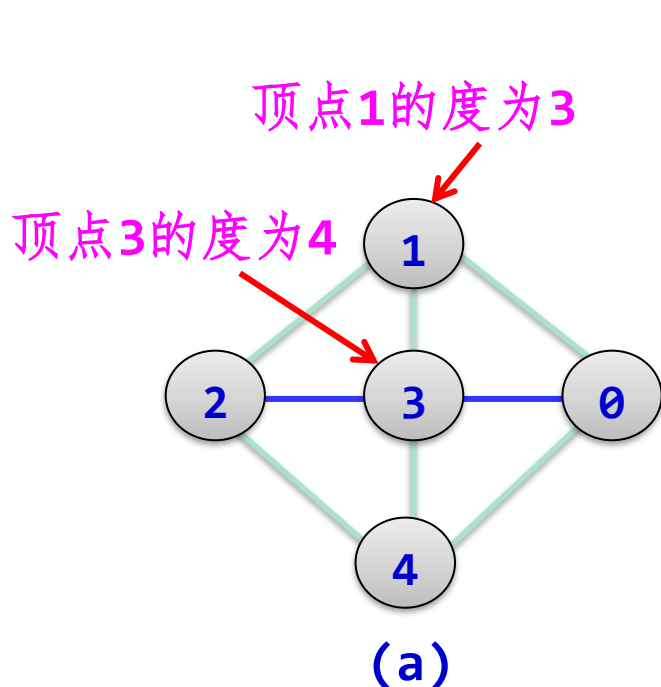
端点和邻接点

- 无向图中，若存在一条边 (v, v') $\Rightarrow$ 顶点 v 和顶点 v' 为**端点**，它们互为**邻接点**，即 v 和 v' 相邻接，边 (v, v') 依附于顶点 v 和 v' 。
- 有向图中，若存在一条弧 $\langle i, j \rangle$ $\Rightarrow$ 顶点 i 为**起始端点**（简称为**起点**），顶点 j 为**终止端点**（简称**终点**），它们互为**邻接点**。

6.1 图的定义和术语(P185)

顶点的度、入度和出度

- 无向图中，以顶点 v 为端点的边数称为该**顶点的度**。
- 有向图中，以顶点 v 为终点的入边的数目，称为该**顶点的入度**。
以顶点 v 为起点的出边的数目，称为该**顶点的出度**。有向图中一个顶点的度是该顶点的入度与出度的和。



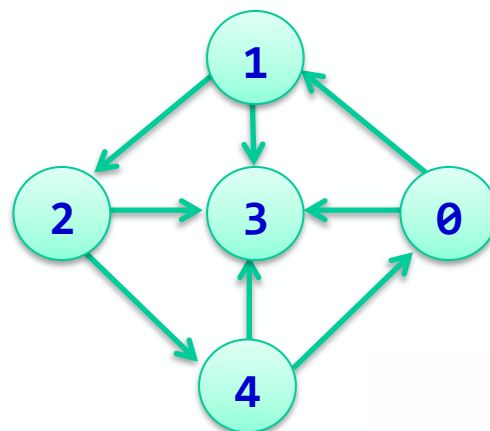
【练习2】 在一个图中，所有顶点的度数之和等于该图总边数的（ ）倍。

A. $1/2$

B. 1

C. 2

D. 4



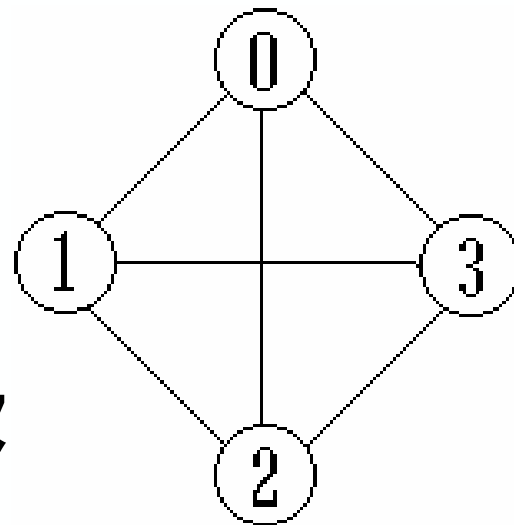
**8条弧
度之和=16**

答案： C

解释： 6条边， 度之和= $3*4=12$ ，

一条边会连接两个顶点， 两个顶点

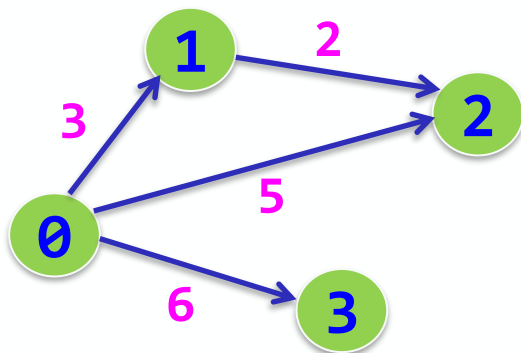
计算度都会涉及到该条边， 所以会计算两次



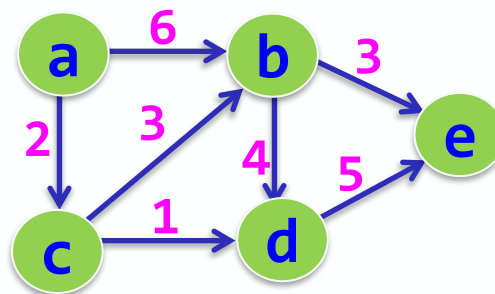
6.1 图的定义和术语(P185)

权、网

- **权**：图中边或弧都可以附带一个对应的数值，这种与边相关的数值称为权。权可以表示从一个顶点到另一个顶点的距离或花费的代价。
- **网**：带权的图称为带权图，也称作网。



有向网1

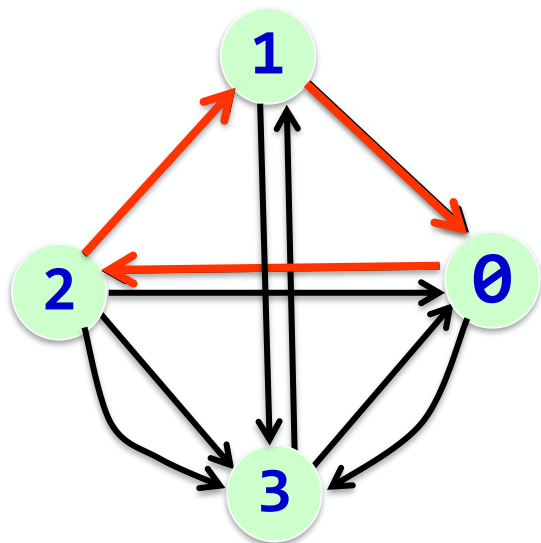


有向网2

6.1 图的定义和术语(P185)

路径、路径长度、简单路径、回路（环）、简单回路

- **路径**：接续的边构成的顶点序列。
- **路径长度**：一条路径上边或弧的数目/权值之和。
- **简单路径**：序列中顶点不重复出现的路径。
- **回路(环)**：第一个顶点和最后一个顶点相同的路径。
- **简单回路（简单环）**：除路径起点和终点可以相同外，其余顶点均不相同



从1到2的路径 $\langle 1, 0, 2 \rangle$ 路径长度为2

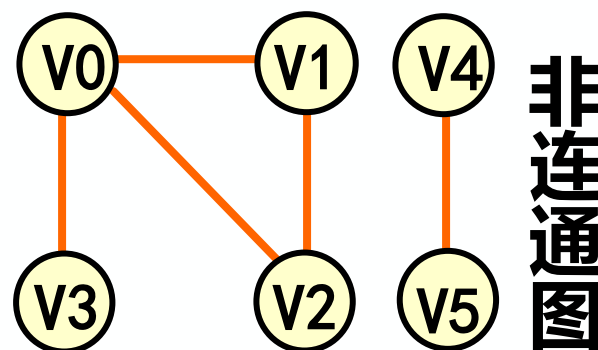
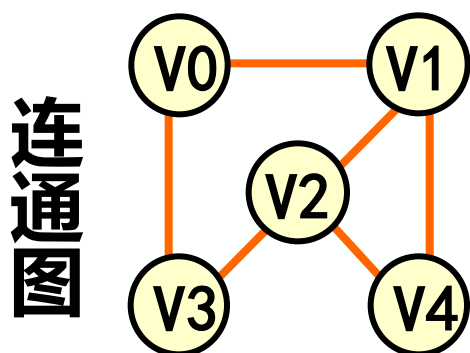
0到1的一条简单路径为 $0 \rightarrow 2 \rightarrow 1$ ，长度为2

$\langle 0, 2, 1, 0 \rangle$ 就是一条简单回路，其长度为3。

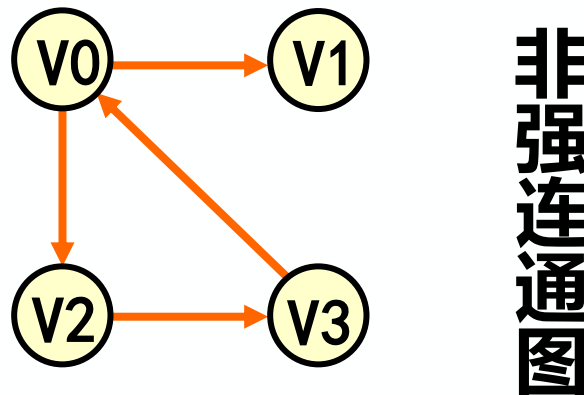
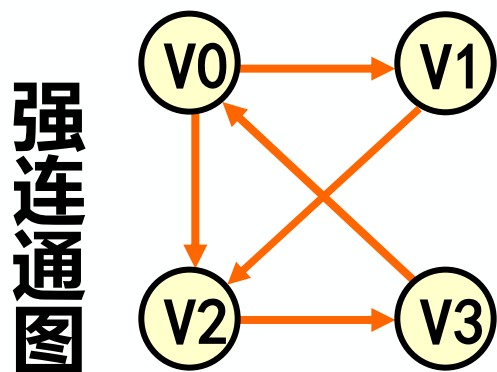
6.1 图的定义和术语(P185)

连通图 (强连通图)

在**无** (**有**) 向图 $G=(V, \{E\})$ 中, 若对**任何**两个顶点 v 、 u 都存在从 v 到 u 的**路径**, 则称 G 是**连通图** (**强连通图**)。



n 个顶点的连通图至少有 $n-1$ 条边, 至多有 $n(n-1)/2$ 条边。

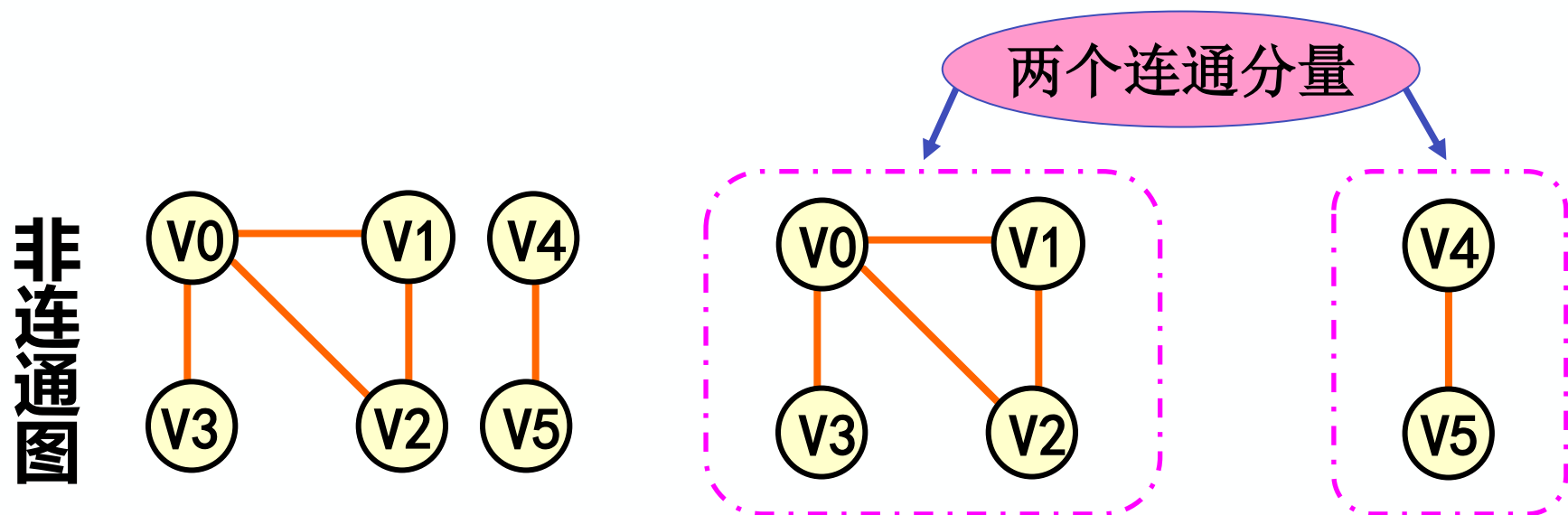


6.1 图的定义和术语(P185)

连通分量 (针对无向图)

无向图G的**极大连通子图**称为G的**连通分量**。

极大连通子图意思是：该子图是G的连通子图，并且是连通的。若将G中任何不在该子图中的顶点加入，子图就不再连通。



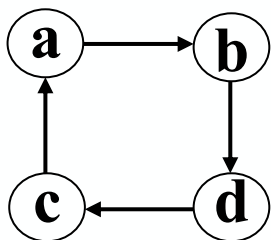
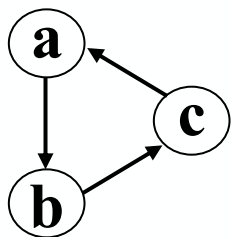
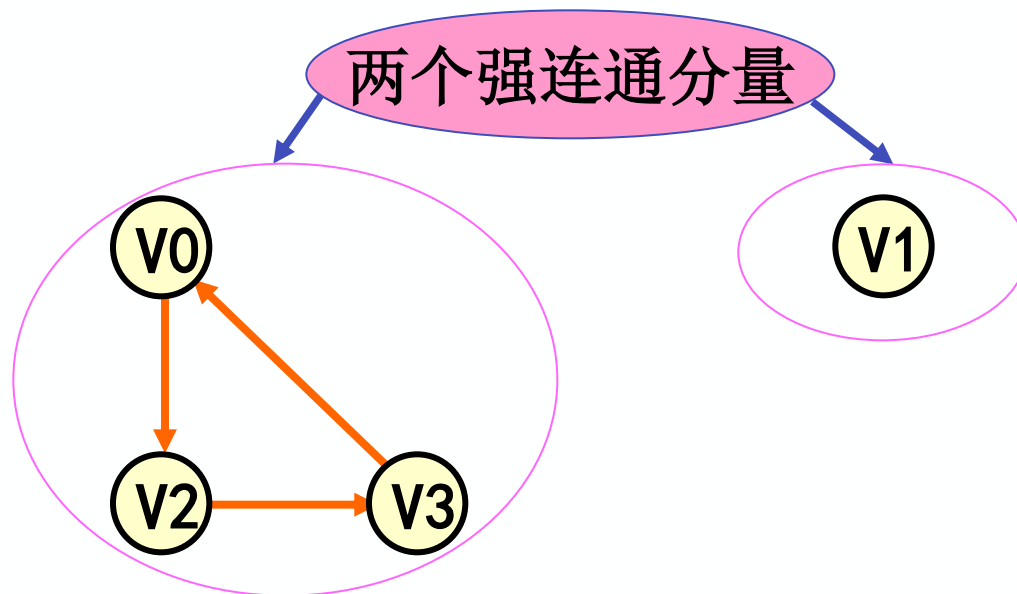
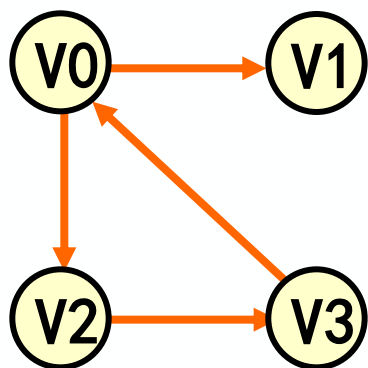
“极大”：①子图中顶点数目达到最多，子图再增加图G中的其它不属于该子图的顶点，就不再连通。②极大连通子图包含依附于这些顶点的所有边。

6.1 图的定义和术语(P186)

强连通分量 (针对有向图)

有向图G的极大强连通子图称为G的强连通分量。

极大强连通子图意思是：该子图是G的强连通子图，将G中任何不在该子图中的顶点加入，子图不再是强连通的。



n 个顶点的强连通图，至少有 n 条弧，至多有 $n(n-1)$ 条弧。

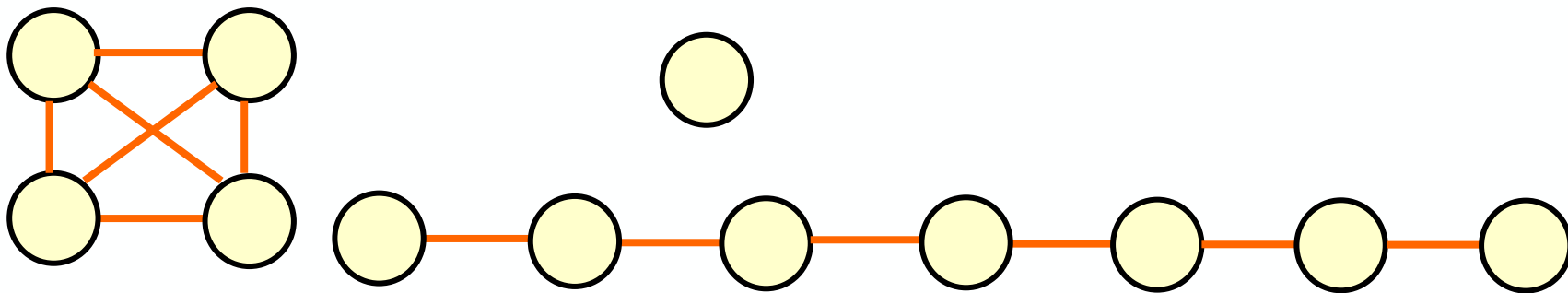
【练习3】 设G是一个非连通无向图，有6条边，则该图至少有（ ）个顶点。

A. 2

B. 3

C. 4

D. 5



答案：D

解释：要使顶点个数最少且为非连通图，**至少**要由**两个连通分量构成：单个顶点+完全无向图**。完全无向图： $n(n-1)/2=6$ $n=4$ 。单个顶点：1个顶点。对应的图有 $4+1=5$ 个顶点。

**【练习4】 一个有28条边的非连通无向图至少有
() 个顶点。**

A. 7 B. 8 C. 9 D. 10 E. 11

答案： C

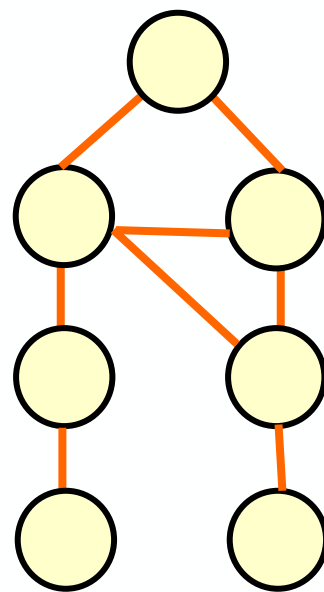
解释： 考虑非连通图的最极端情况， 即由一个完全无向图加一个独立的顶点构成， 若再加上一条边则必然由非连通图变成连通图。 $28 = n * (n - 1) / 2$, $n = 8$ 的完全无向图， 再加上1个不连通的顶点， 共9个顶点。

【练习5】若无向图 $G=(V, E)$ 中含有7个顶点，要保证图G在任何情况下都是连通的，则需要的边数最少是（ ）。

A. 6 B. 7 C. 15 D. 16 E. 21

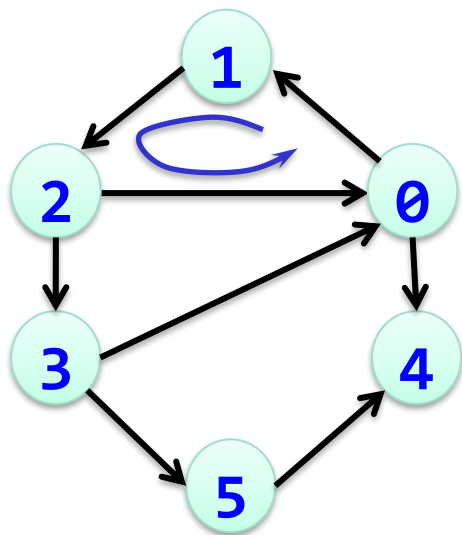
答案：D

解释：考虑最极端情形，即图G的6个顶点构成一个完全无向图，再加上一条边后，第7个顶点必然与该完全无向图构成一个连通图，所以最少是 $6*5/2+1=16$ 。

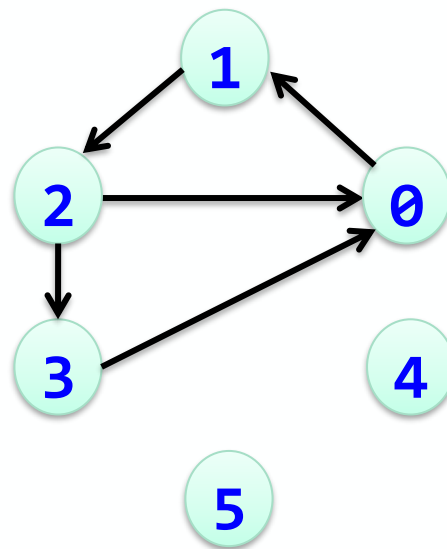


在一个非强连通图中找强连通分量的方法：

- ① 在图中找有向环。
- ② 扩展该有向环：如果某个顶点到该环中任一顶点有路径，并且该环中任一顶点到这个顶点也有路径，则加入这个顶点。



一个非强连通图



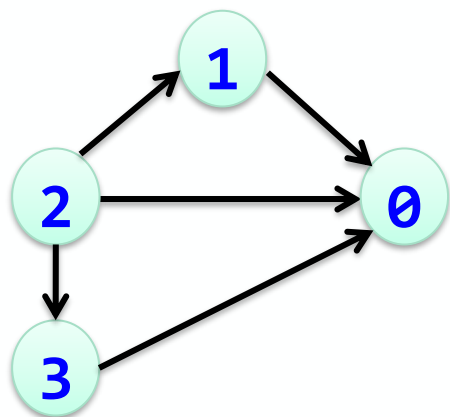
3个强连通分量

【练习6】 以下关于图的叙述中，正确的是（ ）。

- A. 强连通有向图的任何顶点到其他所有顶点都有弧
- B. 图的任意顶点的入度等于出度
- C. 有向完全图一定是强连通有向图
- D. 有向图的边集的子集和顶点集的子集可构成原有图的子集

答案：C

解释： 强连通有向图的任何顶点到其他所有顶点都有路径；



边 $\langle 1, 0 \rangle$

顶点 $\{2, 3\}$



若边集中的某条边对应的顶点不在**给出**的顶点子集中，则有向图边集的子集和顶点集的子集无法构成子图。

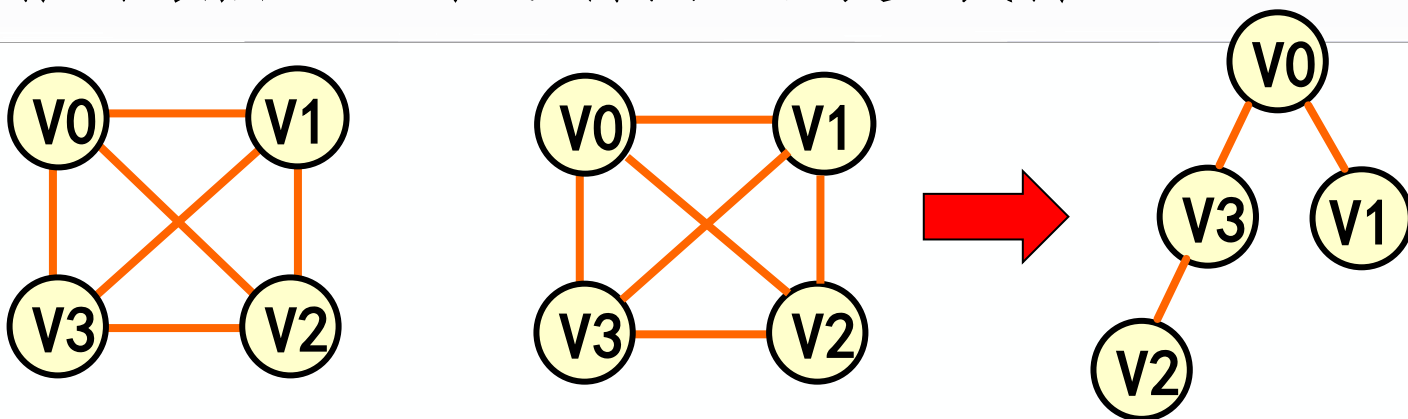
6.1 图的定义和术语(P186)

连通图的生成树

连通图的生成树是包含图中**全部 n 个顶点**的一个极小连通子图，若删去生成树中任意一条边都会变成非连通图，若再加上一条边则会形成一个回路。

关于无向图生成树的几个结论：

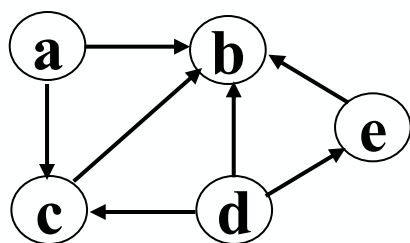
- 一棵有 n 个顶点的生成树有且仅有 $n-1$ 条边；
- 如果一个图有 n 个顶点和小于 $n-1$ 条边，则是**非连通图**；
- 如果多于 $n-1$ 条边，则**一定有环**；
- 有 n 个顶点、 $n-1$ 条边的图不一定是生成树。



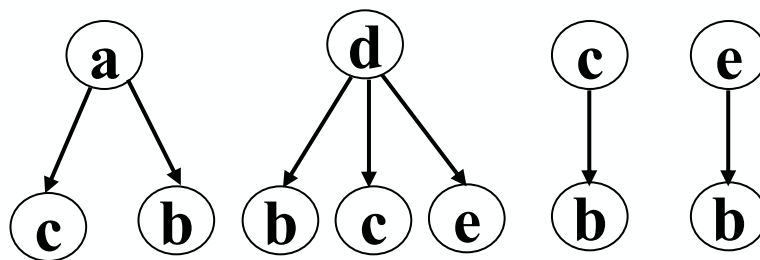
6.1 图的定义和术语(P186)

有向树、生成森林

有向图的生成森林是由若干棵有向树组成的，包含图中全部顶点，若有向图恰有一个顶点的入度为0，其余顶点入度均为1，则该有向图就是一棵有向树。如下图所示。



(a) 有向图



(b) 生成森林

有向图及其生成森林

6.3 图的抽象数据类型(P187)

ADT Graph

{ 数据对象V: V是具有相同特性的数据元素的集合, 顶点集。

数据关系R: $R=\{VR\}$

$VR=\{ \langle v, w \rangle | v, w \text{ 且 } P(v, w), \langle v, w \rangle \text{ 表示从 } v \text{ 到 } w \text{ 的弧, 谓词 } P(v, w) \text{ 定义了弧 } \langle v, w \rangle \text{ 的意义或信息 } \}$

基本操作 P: (1) CreatGraph(G) 输入图G的顶点和边, 建立图G

(2) DestroyGraph(G) 释放图G占用的存储空间

(3) GetVex(G, v) 在图G中找到顶点v, 并返回顶点v
的相关信息

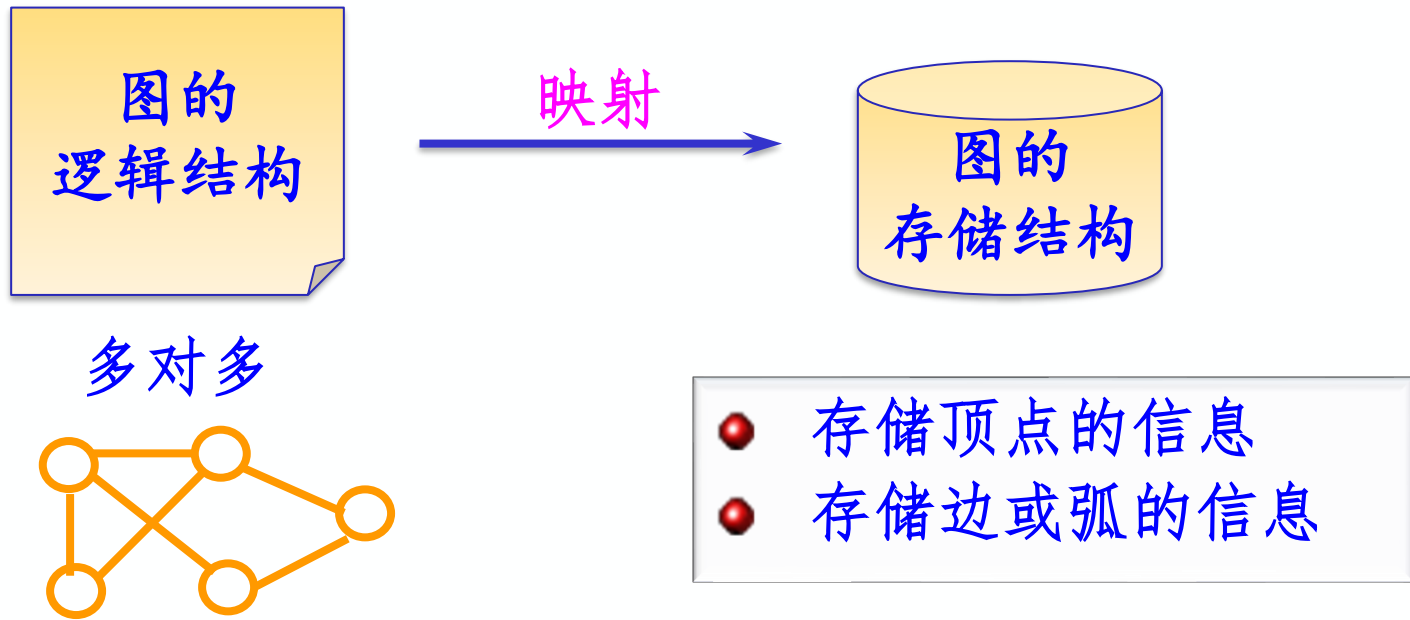
(4) PutVex(G, v, value) 在图G中找到顶点v, 并将
value值赋给顶点v

6.3 图的抽象数据类型(P188)

- (5) **InsertVex(G, v)** 在图 G 中增添新顶点 v
- (6) **DeleteVex(G, v)** 删除图 G 中顶点 v 及和顶点 v 相关联的边或弧
- (7) **InsertArc(G, v, w)** 在图 G 中增添一条从顶点 v 到顶点 w 的边或弧
- (8) **DeleteArc(G, v, w)** 在图 G 中删除一条从顶点 v 到顶点 w 的边或弧
- (9) **DFSTraverse(G, v)** 在图 G 中, 从顶点 v 出发深度优先遍历图 G
- (10) **BFSTraverse(G, v)** 在图 G 中, 从顶点 v 出发广度优先遍历图 G 。
- (11) **FirstAdjVex(G, v)** 在图 G 中, 返回 v 的第一个邻接点。若顶点在 G 中没有邻接顶点, 则返回空。
- (12) **NextAdjVex(G, v, w)** 在图 G 中, 返回 v 的(相对于 w 的) 下一个邻接顶点。若 w 是 v 的最后一个邻接点, 则返回空。
- (13) **LocateVex(G, u)** 在图 G 中定位顶点 u , 返回该顶点在图中位置。

}ADT Graph

6.4 图的存储结构(P189)

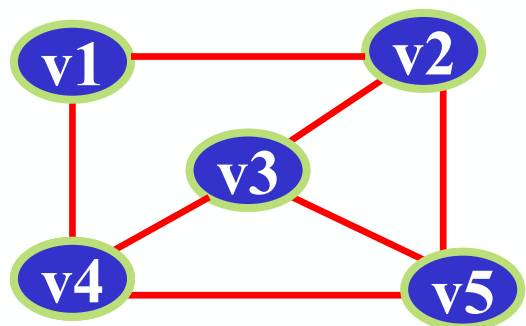


图的两种主要存储结构:

- 邻接矩阵（二维数组表示法）
- 邻接表（链式存储）

6.4.1 邻接矩阵表示法——无向图

$$A[i][j] = \begin{cases} 1 & \text{若}(v_i, v_j) \in E \text{ 或 } (v_j, v_i) \in E, \text{ 即 } v_i, v_j \text{ 邻接} \\ 0 & \text{其它, 即 } v_i, v_j \text{ 不邻接} \end{cases}$$



邻接矩阵: $(v1 \ v2 \ v3 \ v4 \ v5)$

$$A[i][j] = \begin{matrix} & \begin{matrix} v1 & v2 & v3 & v4 & v5 \end{matrix} \\ \begin{matrix} v1 \\ v2 \\ v3 \\ v4 \\ v5 \end{matrix} & \begin{pmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{pmatrix} \end{matrix}$$

对称

结论：邻接矩阵是表示图中有n个顶点之间相邻关系的矩阵。

无向图的邻接矩阵是对称矩阵。

由于每个顶点没有到自身的边，故主对角线上的值都为0。

顶点 v_i 的度 = 第*i*行 (列) 中非零元素的个数

特例：无向完全图的邻接矩阵中，主对角元素为0，其余均为1

【练习7】 在含有 n 个顶点和 e 条边的无向图的邻接矩阵中，零元素的个数为（ ）。

A. e

B. $2e$

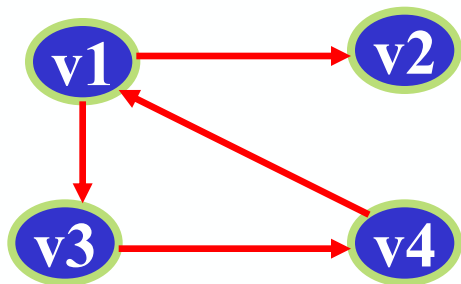
C. $n^2 - e$

D. $n^2 - 2e$

答案：D

解释：无向图的邻接矩阵中，矩阵大小为 n^2 ，无向图中边会被统计两次，因此矩阵中非零元素个数为 $2e$ ，故零元素的个数为 $n^2 - 2e$

6.4.1 邻接矩阵表示法——有向图



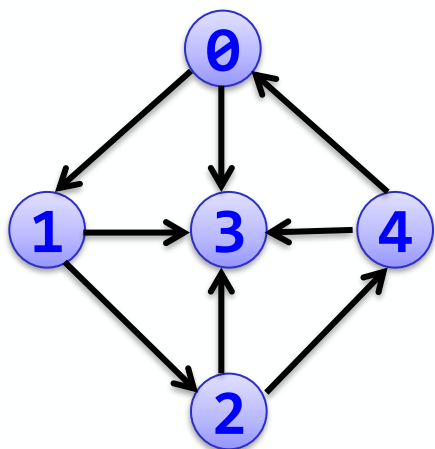
邻接矩阵:

$$A[i][j]=$$

	(v1 v2 v3 v4)
v1	0 1 1 0
v2	0 0 0 0
v3	0 0 0 1
v4	1 0 0 0

不对称

说明: 第*i*行非零元素的个数就是结点 v_i 的**出度**, 如v1的出度为2
第*i*列非零元素的个数就是结点 v_i 的**入度**, 如v1的入度为1
第*i*个顶点的度=第*i*行+第*i*列非零元素的总个数, v1的度为3
邻接矩阵中非零元素的个数就表示图中弧数。



$A_2=$

	0	1	2	3	4
0	0	1	0	1	0
1	0	0	1	1	0
2	0	0	0	1	1
3	0	0	0	0	0
4	1	0	0	1	0

【练习8】 设图的邻接矩阵A如下所示，各顶点的度依次是（ ）。

$$A[i][j]=\begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

A. 1, 2, 1, 2

B. 2, 2, 1, 1

C. 3, 4, 2, 3

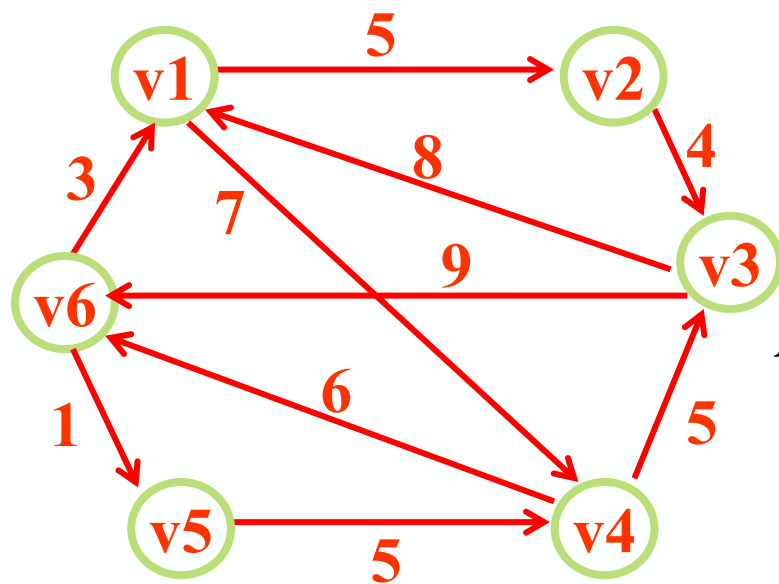
D. 4, 4, 2, 2

答案：C

解释：非对称矩阵说明是有向图，度等于出度（行）+入度（列）的非零元素个数

6.4.1 邻接矩阵表示法——网(即带权图)

定义为: $A[i][j] = \begin{cases} W_{ij} & \langle v_i, v_j \rangle \text{ 或 } (v_i, v_j) \in VR \\ \infty & \text{无边 (弧)} \end{cases}$



$A[i][j] =$

	v1	v2	v3	v4	v5	v6
v1	∞	5	∞	7	∞	∞
v2	∞	∞	4	∞	∞	∞
v3	8	∞	∞	∞	∞	9
v4	∞	∞	5	∞	∞	6
v5	∞	∞	∞	5	∞	∞
v6	3	∞	∞	∞	1	∞

邻接矩阵



对于顶点 v_i :

第 i 个顶点的出度: 第 i 行的非无穷大元素的个数

第 i 个顶点的入度: 第 i 列的非无穷大元素的个数

第 i 个顶点的度: 第 i 行+第 i 列非无穷大元素总个数

6.4.1 邻接矩阵表示法

邻接矩阵的主要特点：

- 一个图的邻接矩阵表示是唯一的。
- 特别适合于稠密图（特别是完全图）的存储。

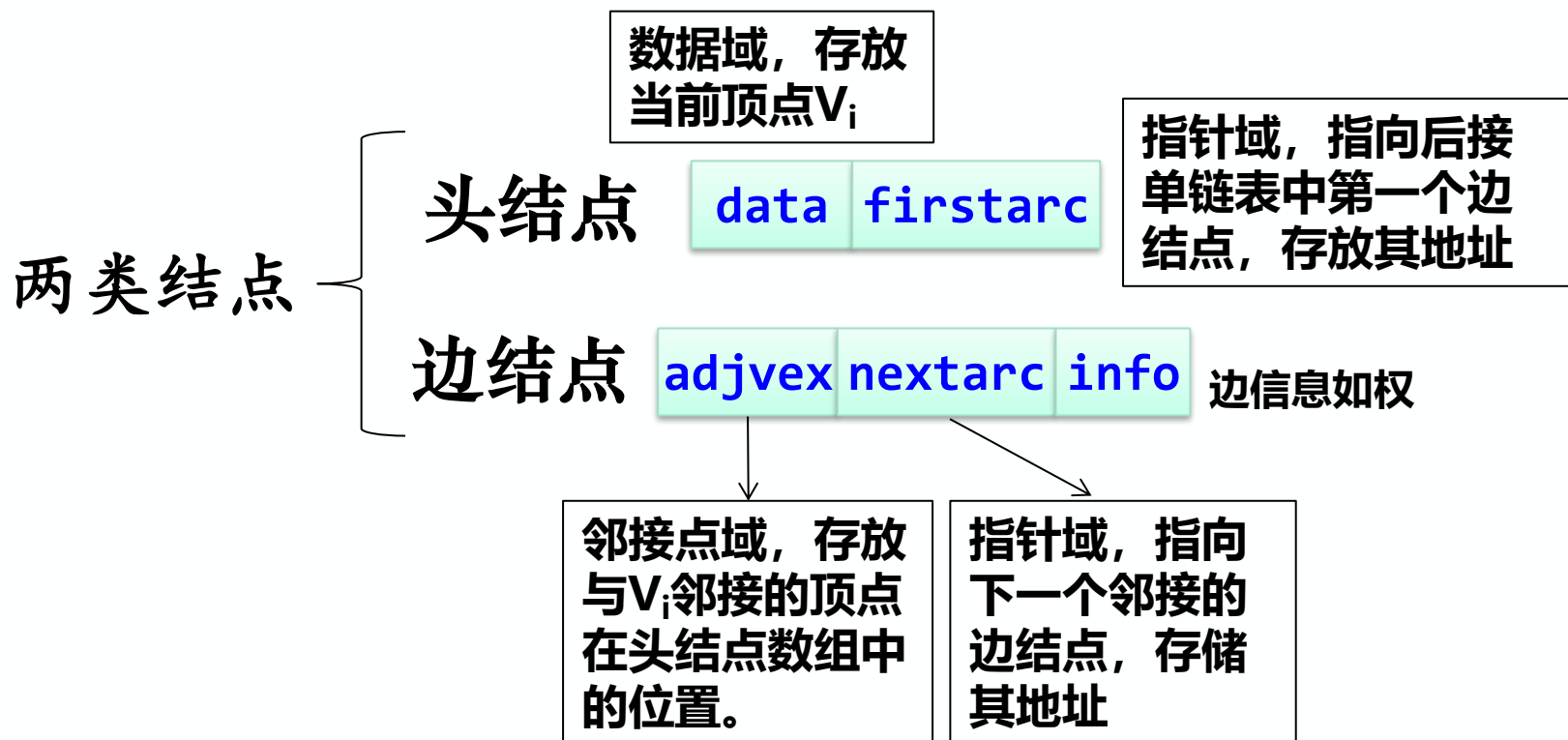
邻接矩阵的空间复杂度为 $O(n^2)$



6.4.2 邻接表(链式)表示法 (P191)

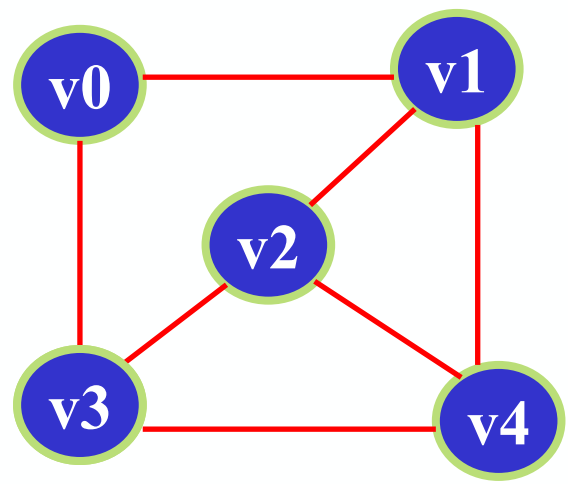
基本思想：对于图的每个顶点建立一个单链表，存储该顶点所有邻接顶点及其相关信息。每一个单链表设置一个表头结点，表示顶点信息。第*i*个单链表就表示依附于顶点 V_i 的边(弧)。

图的邻接表表示法是一种顺序分配与链式分配相结合的存储方法。
头结点合并为顶点数组 边结点组成边链表



6.4.2 邻接表(链式)表示法—无向图(P191)

每个单链表之前添加一个头结点（表示顶点信息）



两类结点



头结点



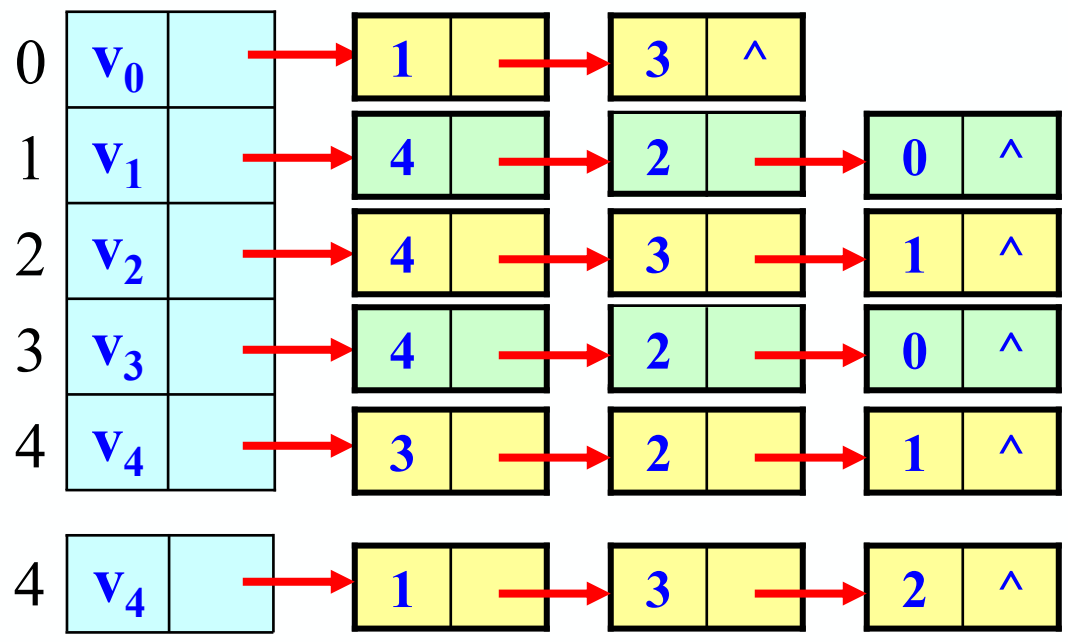
边结点

边信息如权

说明：邻接表对每个顶点 v_i 建立一个**单链表**，将 v_i 的所有邻接点链起来。

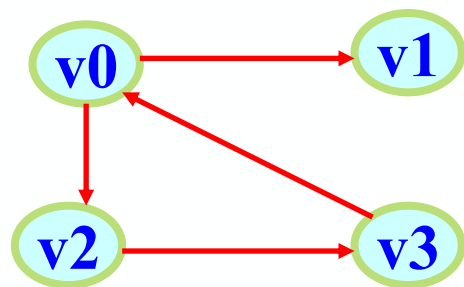
邻接表不唯一，因各个边结点的链入顺序是任意的。

所需空间复杂度： $O(n+e)$

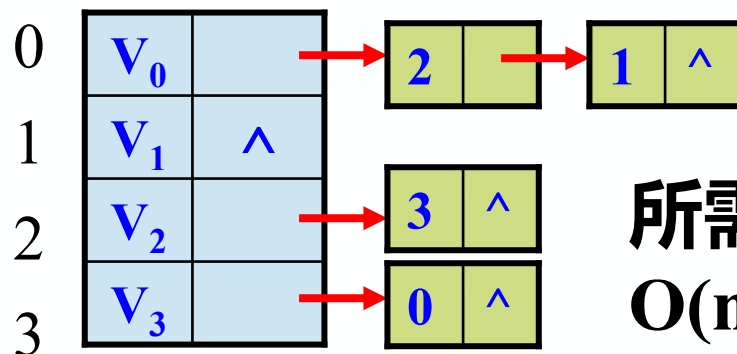


若无向图中有 n 个顶点， e 条边，则邻接表需 n 个头结点， $2e$ 个边结点

6.4.2 邻接表(链式)表示法—有向图(P191)



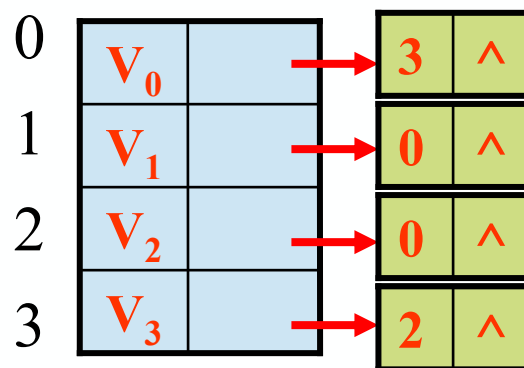
邻接表(出度弧), 出度直观



所需空间复杂度:
 $O(n+e)$

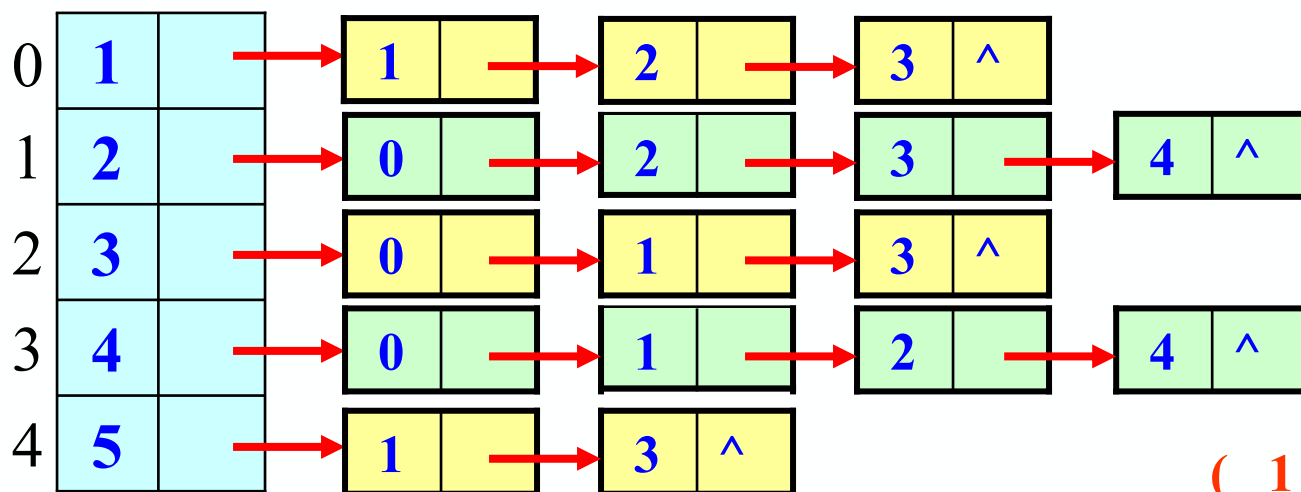
- 顶点 v_i 的出度为对应单链表中的结点个数
- 求入度, 需遍历整个邻接表。

逆邻接表(入度弧), 入度直观

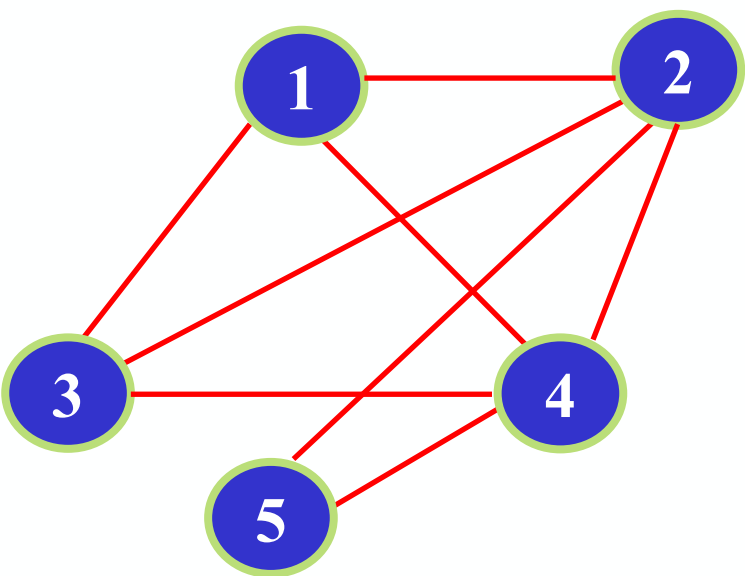


在有向图的逆邻接表中, 对于每个顶点, 链接的是指向该顶点的入度弧。

【练习9】 设图 $G=(V, E)$ 以邻接表存储，画出图 G 及其邻接矩阵。

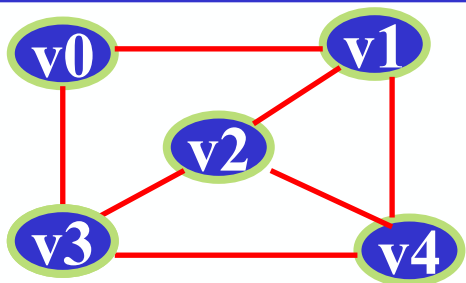


在邻接表中，
每条边都存储了
两次，无向图



$$A[i][j] = \begin{pmatrix} & \textcolor{red}{1} & \textcolor{red}{2} & \textcolor{red}{3} & \textcolor{red}{4} & \textcolor{red}{5} \\ \textcolor{red}{1} & 0 & 1 & 1 & 1 & 0 \\ \textcolor{red}{2} & 1 & 0 & 1 & 1 & 1 \\ \textcolor{red}{3} & 1 & 1 & 0 & 1 & 0 \\ \textcolor{red}{4} & 1 & 1 & 1 & 0 & 1 \\ \textcolor{red}{5} & 0 & 1 & 0 & 1 & 0 \end{pmatrix}$$

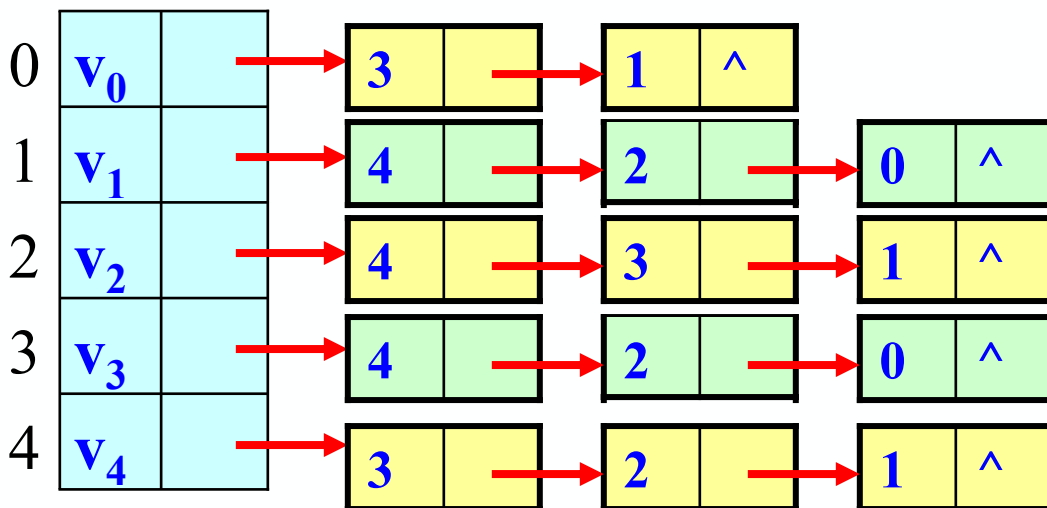
邻接矩阵与邻接表表示法的比较



联系：邻接表中每个链表对应邻接矩阵中的行，边结点的个数等于矩阵一行中非零元素的个数。

(v0 v1 v2 v3 v4)

v0	0	1	0	1	0
v1	1	0	1	0	1
v2	0	1	0	1	1
v3	1	0	1	0	1
v4	0	1	1	1	0



区别：

- ① 对于任一确定的无向图，邻接矩阵是**唯一**的，邻接表**不唯一**。
- ② 邻接矩阵的空间复杂度为 $O(n^2)$ ，而邻接表的空间复杂度为 $O(n+e)$ 。
- ③ 邻接矩阵容易实现图的操作，如求某顶点的度、判断顶点间是否有边
- ④ 邻接表便于查找任一顶点的所有邻接点，便于增加和删除顶点。
- ⑤ 邻接矩阵多用于**稠密图**；而邻接表多用于**稀疏图**。

6.4.3 十字链表 (P193)

有向图的另一种链式存储结构，是有向图的邻接表和逆邻接表的结合。有向图中的每个顶点在十字链表中对应有一个顶点结点，有向图中的每一条弧对应十字链表中的一个弧结点。

顶点结点

data firstin firstout

指针域，指向第一个出度弧结点

顶点有
关信息
V1,V2..

指针域，
指向第一个
入度弧结点

指针域，
指向弧头相同的
下一个弧结点

弧结点
 $\langle i, j \rangle$

tailvex headvex hlink tlink info

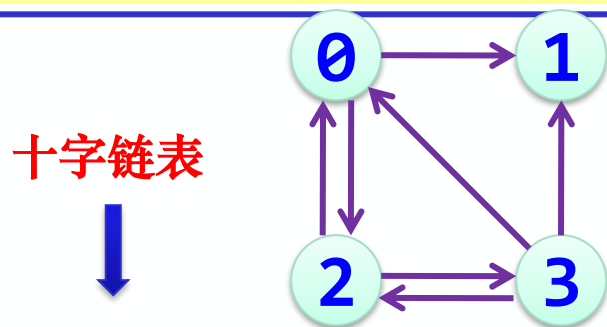
弧的相关信息(权)

弧尾的
下标i

弧头的
下标j

指针域，指向弧尾相同
的下一个弧结点

6.4.3 十字链表 (P193)



4个顶点结点，7个弧结点

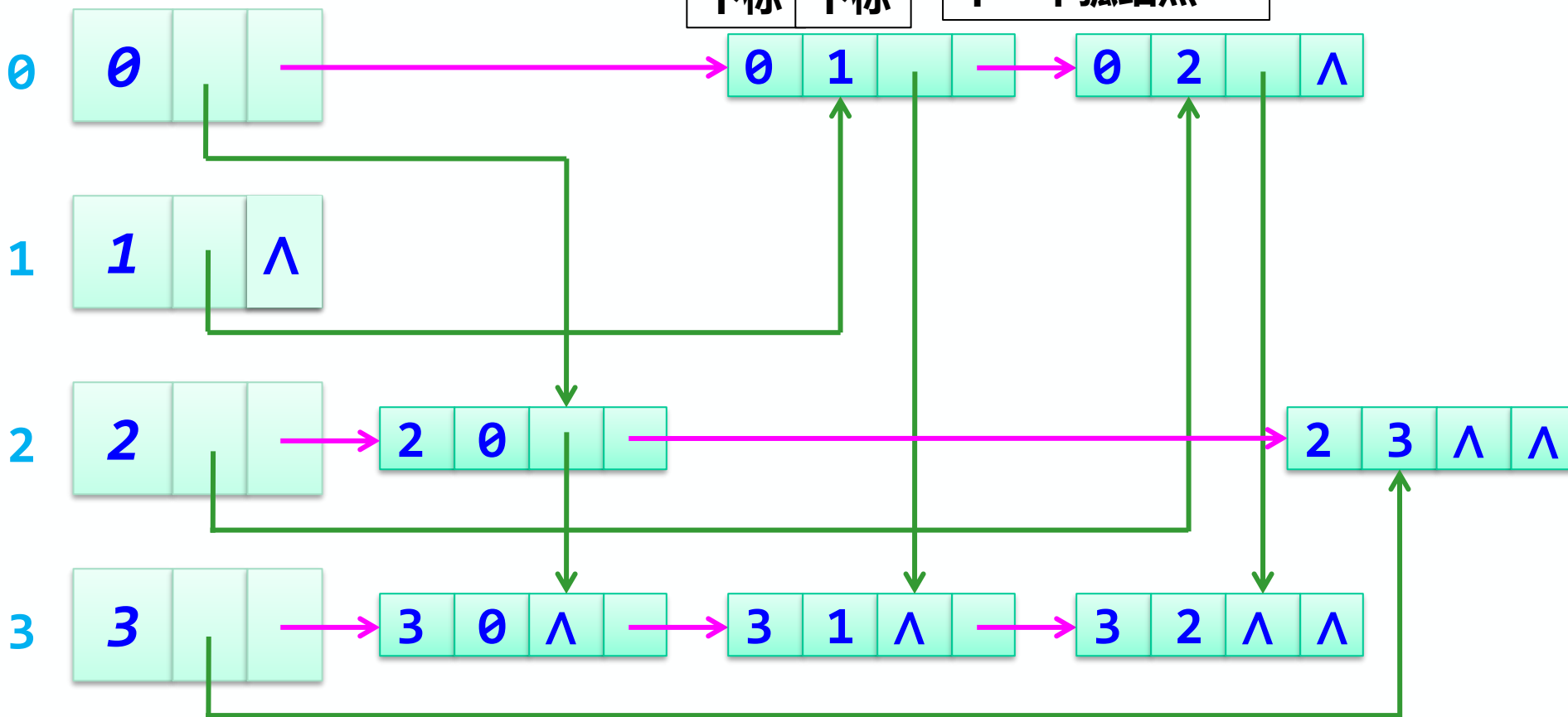
指向弧头相同的 下一个弧结点

弧尾下标

弧头下标

指向弧尾相同的 下一个弧结点

入边 出边



6.4.4 邻接多重表(P194)

- 邻接多重表是**无向图**的另外一种链式存储结构；
- 无向图的邻接表中每条边都要存储两遍边结点，与之相比，邻接多重表对边的操作（标记或删除）更便利。

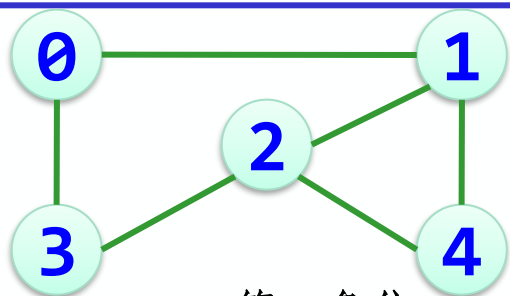
头结点类型

边结点类型： (i, j)

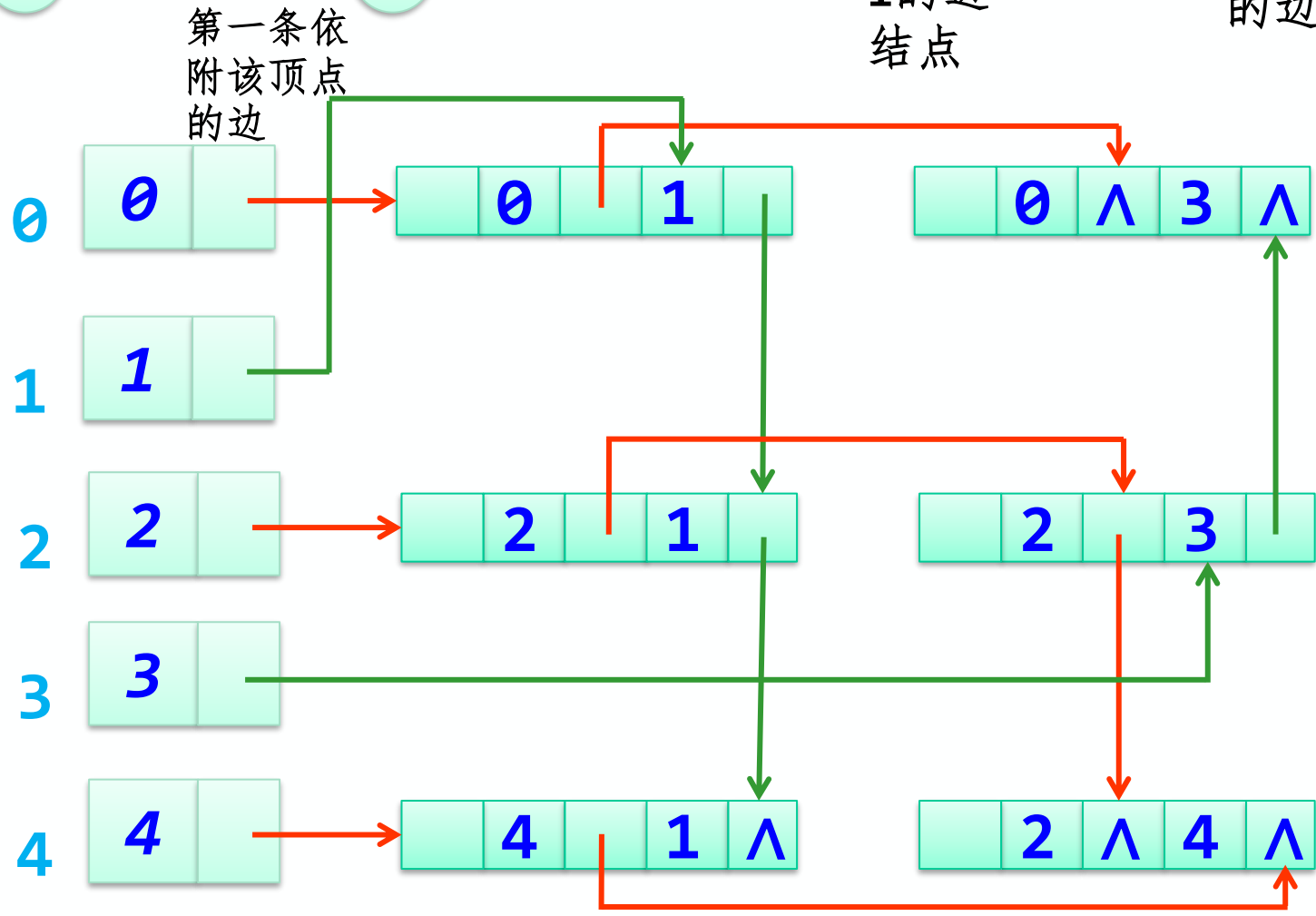
<i>data</i>	<i>firstedge</i>	<i>mark</i>	<i>ivex</i>	<i>ilink</i>	<i>ivex</i>	<i>jlink</i>	<i>info</i>
顶点信息	指针，第一条依附该顶点的边	标志域，标记是否被搜索过	边的顶点 <i>i</i>	指针，下一条依附于顶点 <i>i</i> 的边结点	边的顶点 <i>j</i>	指针，下一条依附于顶点 <i>j</i> 的边结点	边的权

6.4.4 邻接多重表(P194)

边结点类型: (i, j)



ivex	ilink	jvex	jlink
边的顶点i	下一条依附于i的边结点	边的顶点j	下一条依附于顶点j的边结点



6.5 图的遍历(P196)

图的遍历(Traversing Graph): 从图的某一顶点出发, 沿着一些边按照某种方法对图中所有顶点访问且仅访问一次。图的遍历算法是各种图的操作的基础。

- ◆ **复杂性:** 图的任意顶点都可能和其余的顶点相邻接, 又可能在访问了某个顶点后, 沿某条路径搜索后又回到原顶点。
- ◆ **解决办法:** 在遍历过程中记下已被访问过的顶点。设置一个辅助数组visited[1...n](n为顶点数), 其初值为0, 一旦访问了顶点i, 使visited[i]为1, 表示顶点i已被访问过。



- ◆ **常用遍历算法:** 有**深度优先搜索算法**和**广度优先搜索算法**
DFS **BFS**

Depth First Search

Breadth First Search

搜索引擎的两种基本抓取策略



爬行和抓取

- 搜索引擎**蜘蛛**通过跟踪链接访问网页，获得页面HTML代码存入数据库

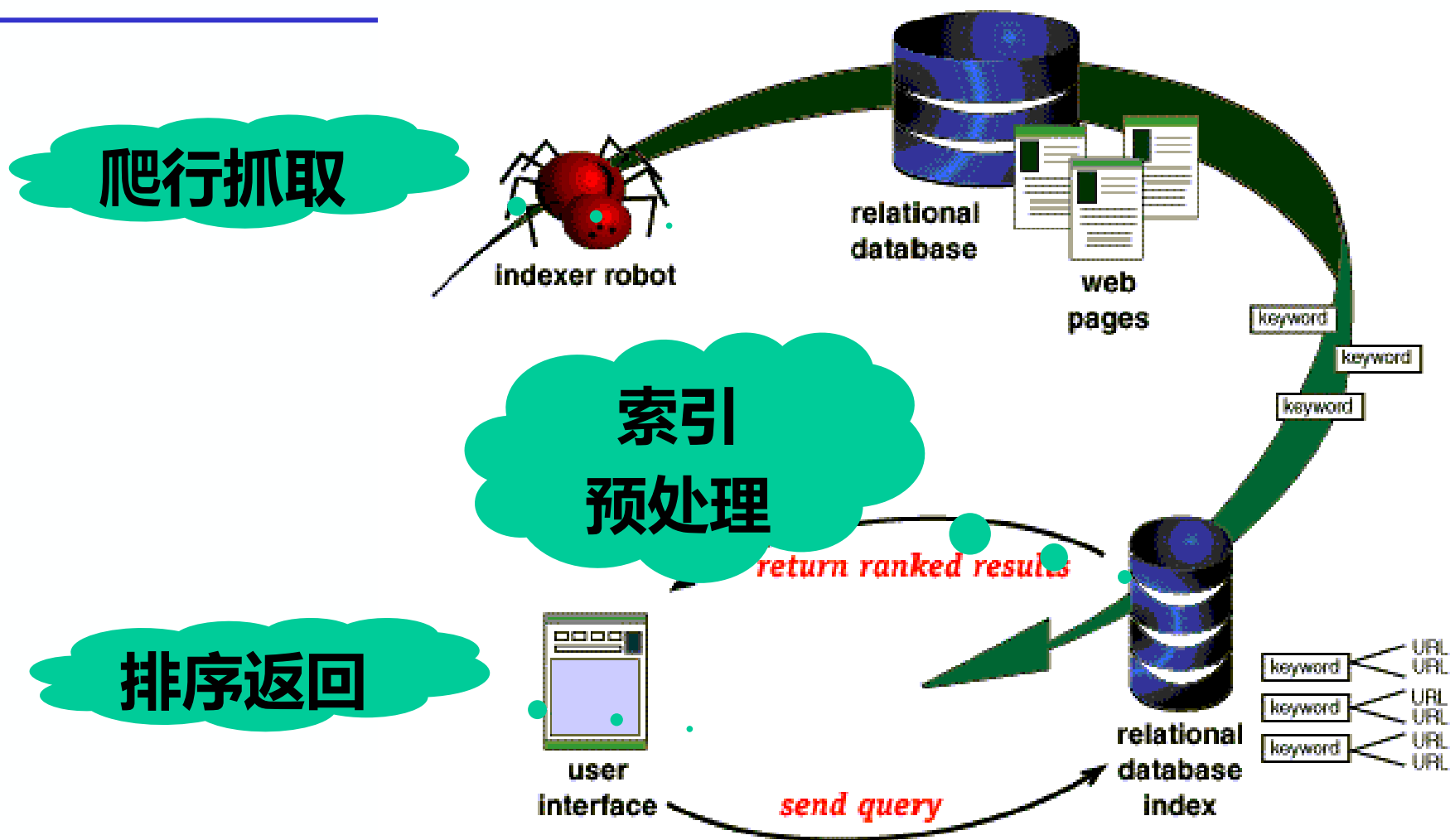
预处理

- 索引程序对抓取来的页面数据进行文字提取、中文分词、**索引**等处理，以备排名程序调用

排名

- 用户输入关键词后，**排名程序**调用索引库数据，计算相关性，然后按一定格式生成搜索结果页面

爬行抓取网页的HTML代码存入到数据库中



用户搜索：输入某关键词后，通过排序程序进行排名，返回用户想要的结果，呈现在用户面前

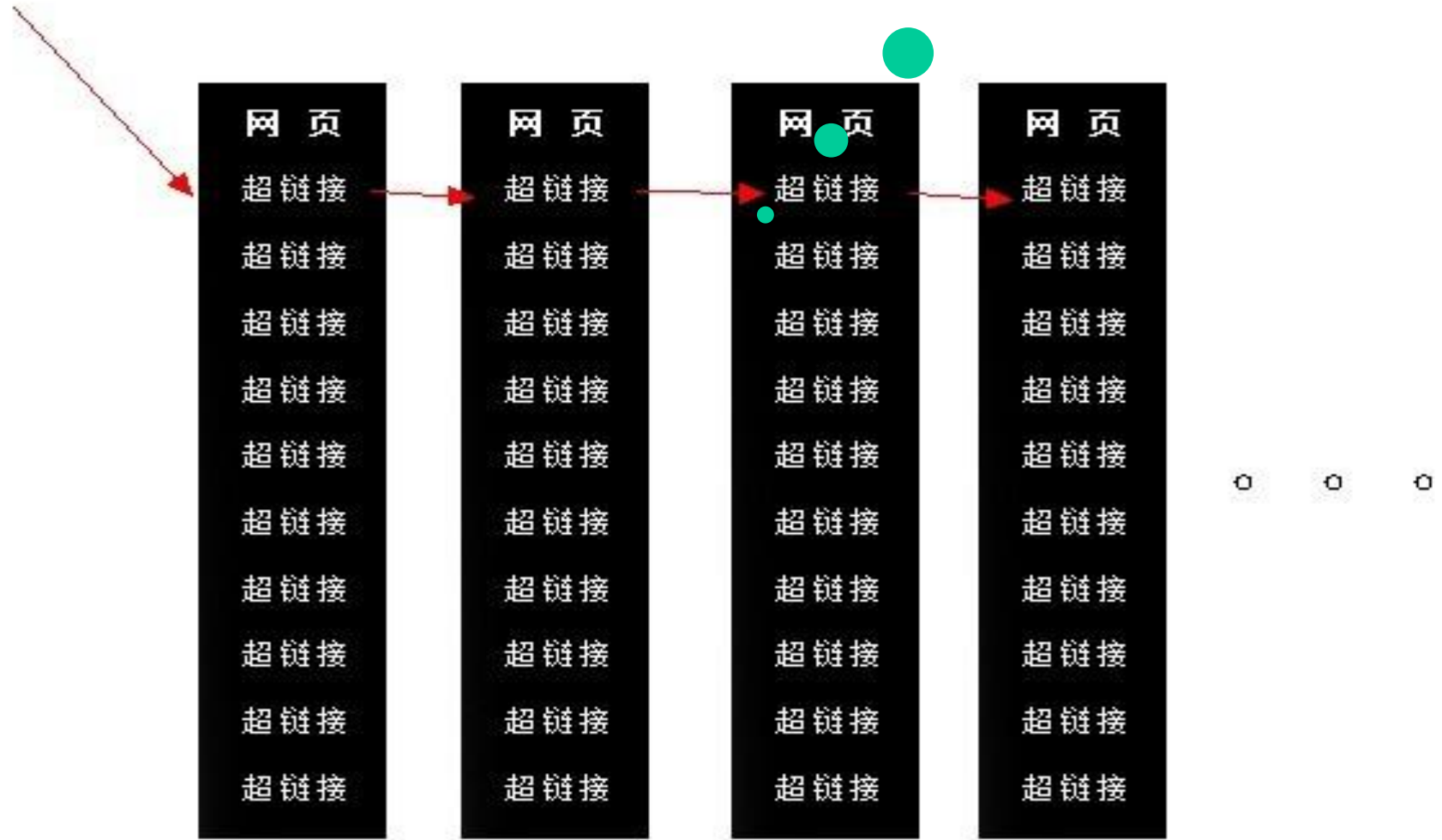
索引程序对于抓取的页面进行文字提取、索引编目等处理，已备调用

搜索引擎的两种基本抓取策略

---深度优先

如**封建帝位的继承**。
不能深入的情况下才
考虑其他分支的策略

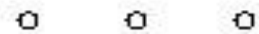
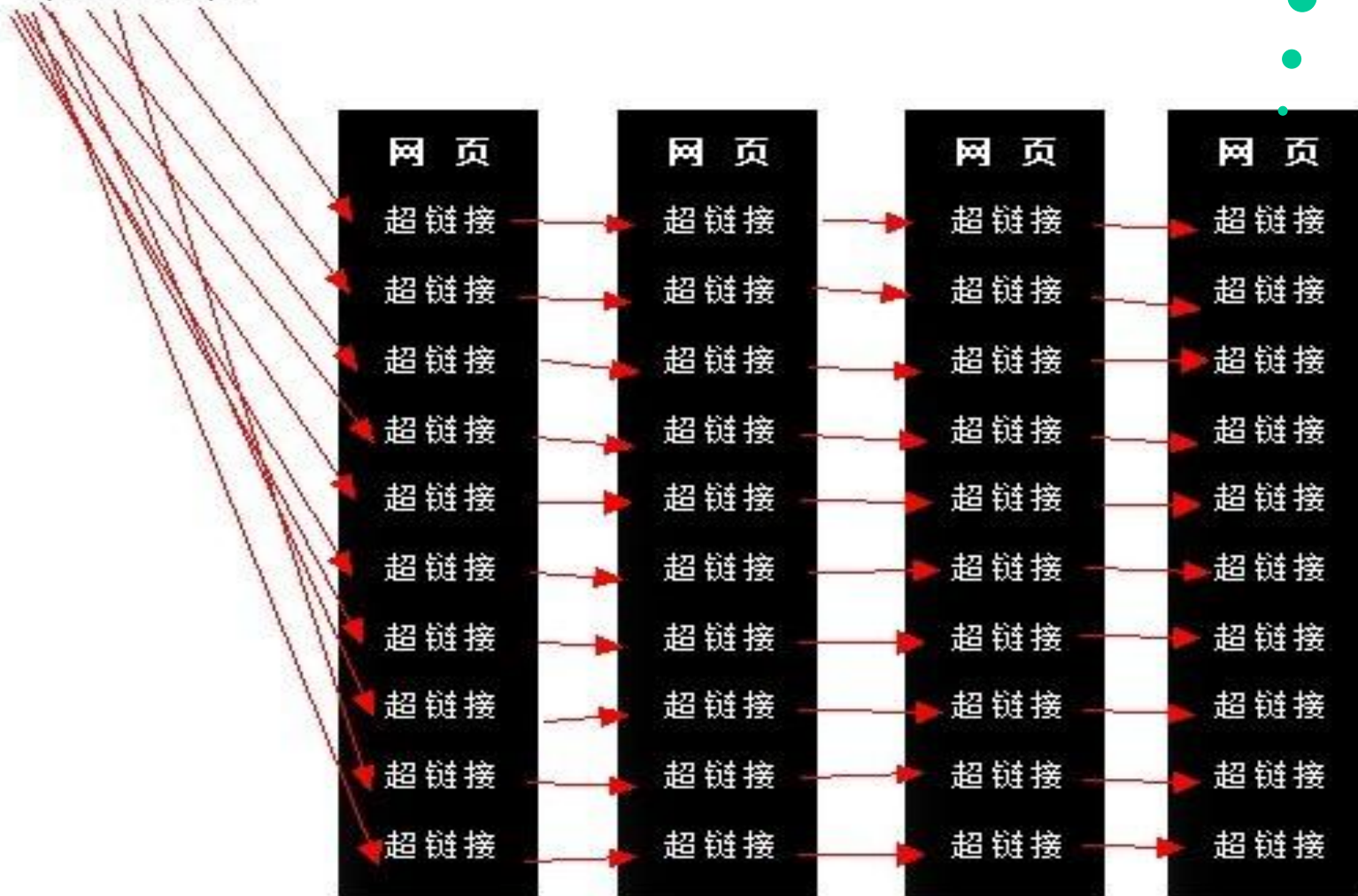
蜘蛛



搜索引擎的两种基本抓取策略 ---广度优先

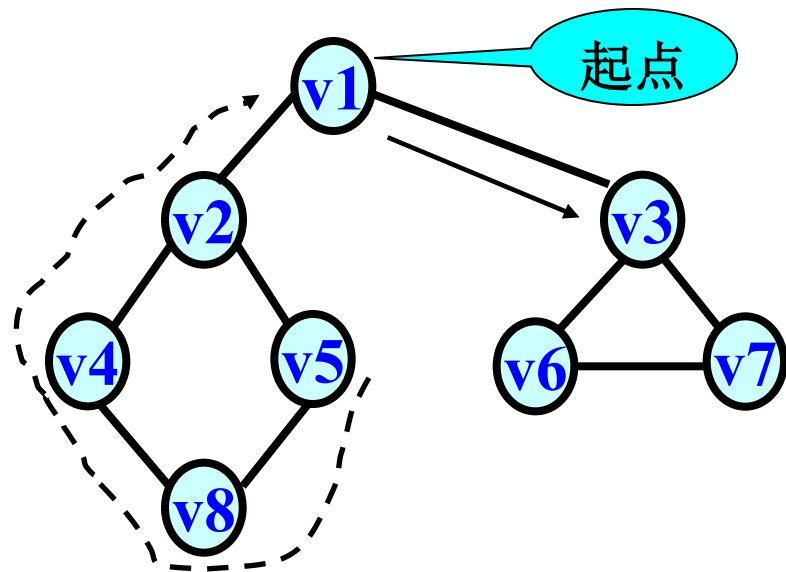
类似**长幼有序、先来后到**的规则

蜘蛛



6.5.1 深度优先搜索(DFS —Depth First Search)

基本思想：——仿树的先根/二叉树的先序遍历过程。



DFS 结果

$v1 \rightarrow v2 \rightarrow v4 \rightarrow v8 \rightarrow$
 $v5 \rightarrow v3 \rightarrow v6 \rightarrow v7$

$v1 \rightarrow v2 \rightarrow v5 \rightarrow v8 \rightarrow v4 \rightarrow v3 \rightarrow v6 \rightarrow v7$

$v1 \rightarrow v3 \rightarrow v6 \rightarrow v7 \rightarrow v2 \rightarrow v4 \rightarrow v8 \rightarrow v5$

$v1 \rightarrow v3 \rightarrow v7 \rightarrow v6 \rightarrow v2 \rightarrow v5 \rightarrow v8 \rightarrow v4$

.....

归纳：

先访问起始点v1；再访问起始点第一个未被访问的邻接点，接着在邻接点的邻接点里找未被访问过的，进行访问。

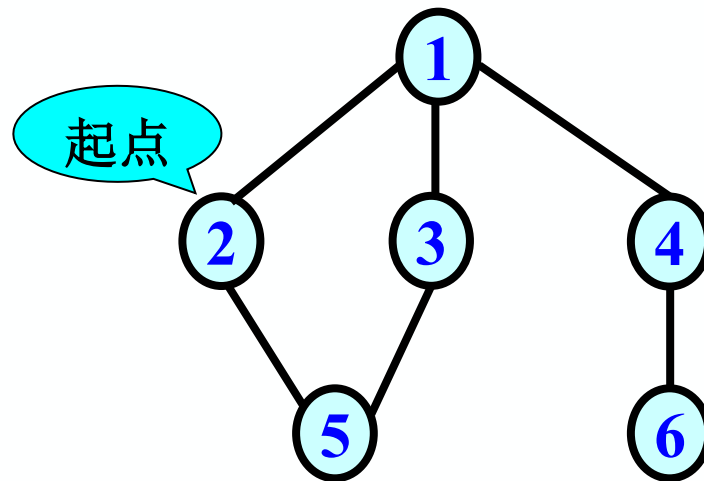
若某个顶点所有邻接点都已被访问，则回退到上一个被访问的顶点。

深度优先遍历通常借助栈来实现，先进后出

【例】 已知图的邻接矩阵，则从顶点2出发DFS的结果是？

邻接矩阵 A

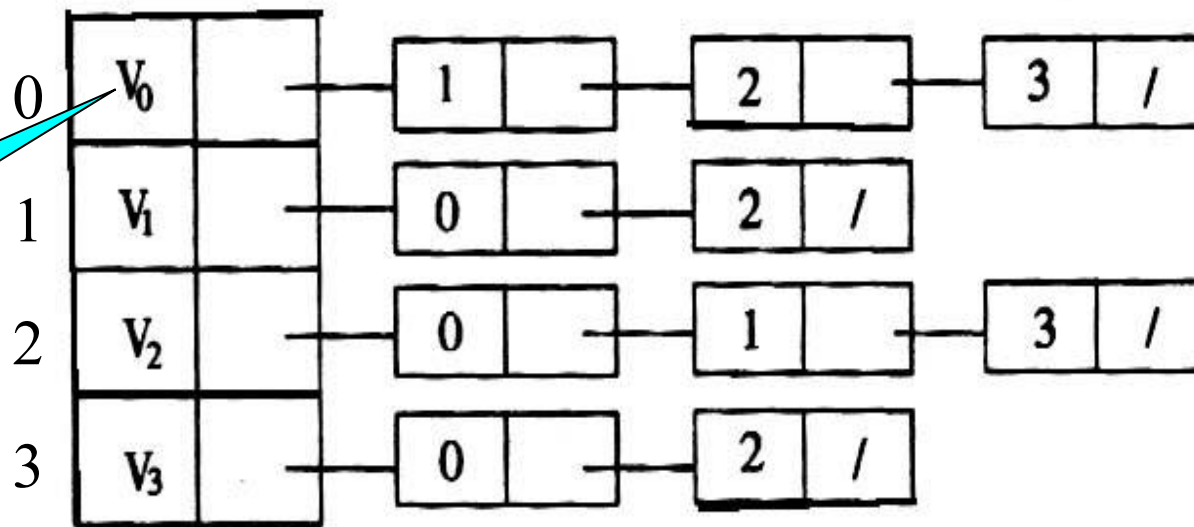
	1	2	3	4	5	6
1	0	1	1	1	0	0
2	1	0	0	0	1	0
3	1	0	0	0	1	0
4	1	0	0	0	0	1
5	0	1	1	0	0	0
6	0	0	0	1	0	0



DFS 结果

2→1→3→5→4→6

【例】 已知图的邻接表，则从顶点V0出发的DFS结果是？



遍历结束

DFS 结果

$v_0 \rightarrow v_1 \rightarrow v_2 \rightarrow v_3$

辅助数组 $visited[i]$

1 true

0 false

实现深度优先搜索的递归函数如下：

```
bool visited[MAXVEX]; //访问标志数组，初值为false

void DFS(Graph G, int v) //从第v个顶点出发递归地深度优先遍历图G
{   cout<<v;   visited[v]=true; //访问顶点v，并置访问标志数组为true
    for (w=FirstAdjVex(G,v);w>=0;w=NextAdjVex(G,v,w)) /*依次检查
v 的所有邻接点，FirstAdjVex (G, v)表示 v 的第一个邻接点，赋给w
NextAdjVex(G,v,w)表示v相对于w的下一个邻接点， w>=0表示存在
邻接点 */
        if(!visited[w])        DFS(G,w);
    //w是v的邻接顶点，若w尚未被访问，则递归调用 DFS
}
```

【练习10】 无向图 $G=(V, E)$, 其中 $V=\{a,b,c,d,e,f\}$
 $E=\{(a,b), (a,e), (a,c), (b,e), (c,f), (f,d), (e,d)\}$,
对该图进行深度优先排序, 得到的顶点序列正确的
是 ()。

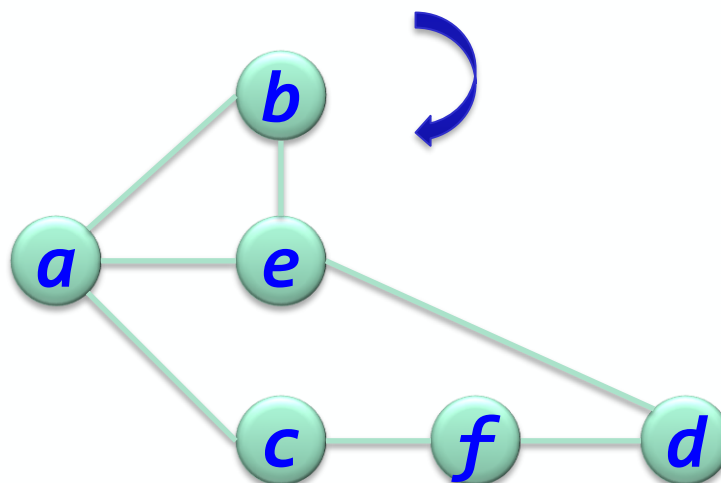
A. a, b, e, c, d, f

B. a, c, f, e, b, d

C. a, e, b, c, f, d

D. a, e, d, f, c, b

$E=\{(a,b), (a,e), (a,c), (b,e),$
 $(c,f), (f,d), (e,d)\}$

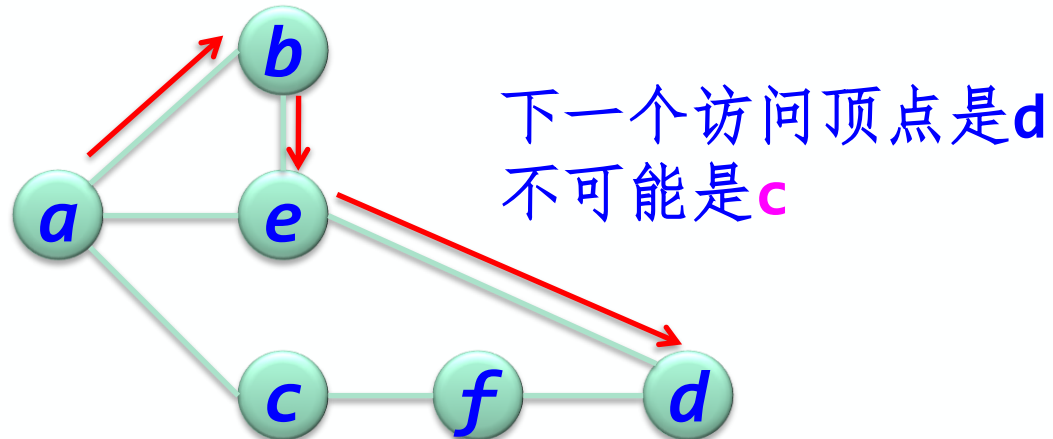


A. a, b, e, c, d, f

B. a, c, f, e, b, d

C. a, e, b, c, f, d

D. a, e, d, f, c, b

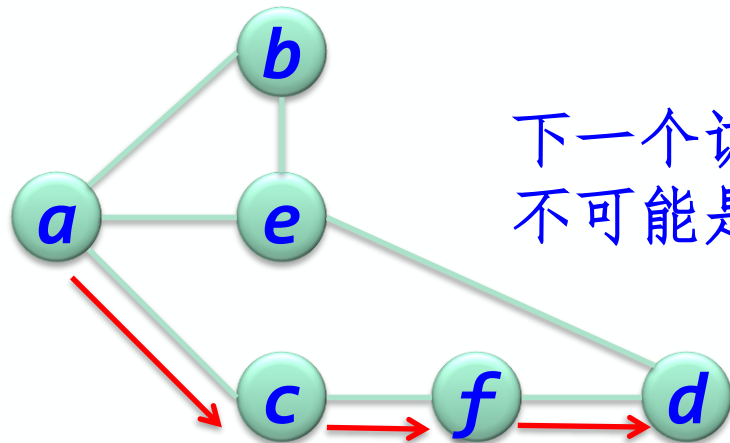


A. a, b, e, c, d, f

B. a, c, f, e, b, d

C. a, e, b, c, f, d

D. a, e, d, f, c, b



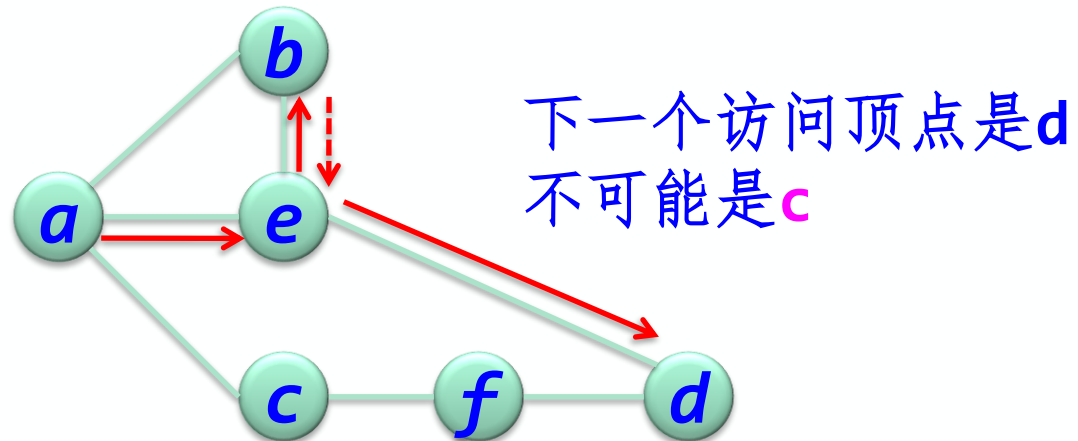
下一个访问顶点是d
不可能是e

A. a, b, e, c, d, f

B. a, c, f, e, b, d

C. a, e, b, c, f, d

D. a, e, d, f, c, b

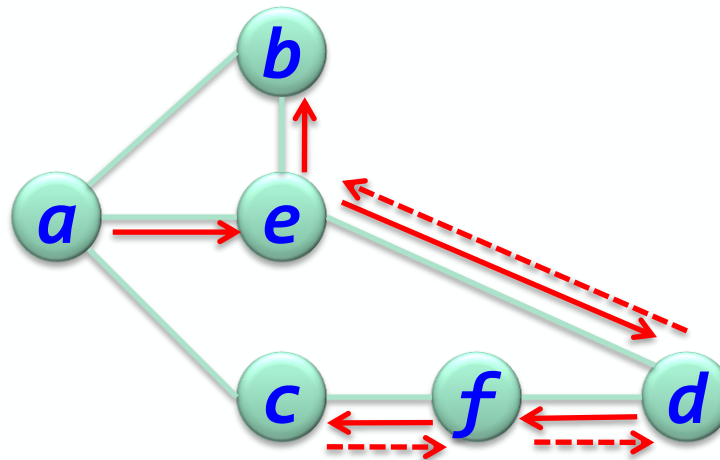


A. a, b, e, c, d, f

B. a, c, f, e, b, d

C. a, e, b, c, f, d

D. a, e, d, f, c, b



【练习11】 下面五个序列中，符合深度优先遍历的序列有（ ）个。**ac, 下一个be择一 aedfbc**

1、 aebfdc

2、 acfdeb

3、 aedfcb

4、 aefdbc

5、 aecfdb **ae, 下一个bdf择一**

A. 5

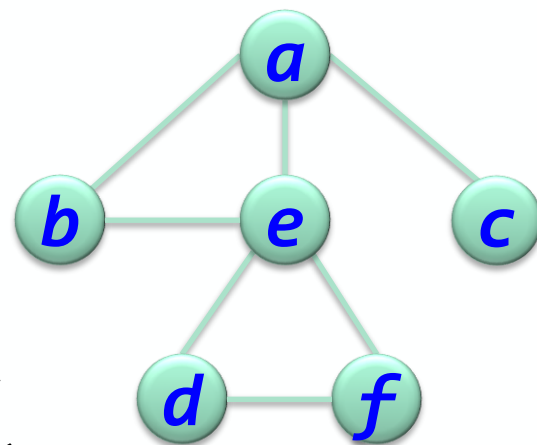
B. 4

C. 3

D. 2

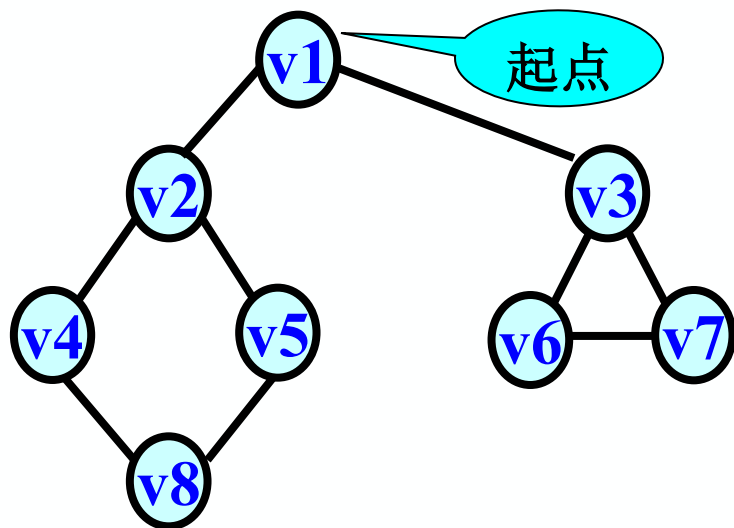
答案： D

解释： 仅1和4正确， 以2为例，
与c邻接且未被访问的结点为空集，
所以下一个应该访问a的邻接点b或e



6.5.2 广度优先搜索(BFS — Breadth First Search)

基本思想：——仿二叉树的层次遍历过程



BFS 结果

$v1 \rightarrow v2 \rightarrow v3 \rightarrow$

$v4 \rightarrow v5 \rightarrow v6 \rightarrow v7 \rightarrow v8$

$v1 \rightarrow v3 \rightarrow v2 \rightarrow v6 \rightarrow v7 \rightarrow v4 \rightarrow v5 \rightarrow v8$

.....

简单归纳：

在访问了起始点v1之后，首先一次性访问 v1的所有邻接点；然后再按这些邻接点被访问的先后次序 依次访问它们所有未被访问的邻接点；重复此过程，直到所有顶点均被访问为止。

分析：

广度优先搜索是一种**分层**的搜索过程，每向前走一步可能访问一批顶点，不像深度优先搜索那样有回退的情况。

广度优先遍历通常借助**队列**来实现，先进先出

广度优先搜索(BFS)

```
void BFS(Graph G,int v)
{ //从图G的顶点v出发, 按广度优先非递归方法 遍历连通图G
    cout<<v;visited[v]=true; //访问第v个顶点, 并置访问标志数组相应值为true
    InitQueue(Q); //初始化辅助队列Q, 置空
    EnQueue(Q,v); //顶点v进队
    while(! QueueEmpty (Q)) { // 队列非空
        DeQueue (Q, u); //队头元素出队并置为u
        for(w=FirstAdjVex(G,u);w>=0;w=NextAdjVex(G,u,w)){
            //依次检查出队顶点的所有邻接点w, FirstAdjVex(G,u)表示u的第一个邻接点
            //NextAdjVex(G,u,w)表示u相对于w的下一个邻接点, w>=0表示存在邻接点
            if (!visited[w]) { ///w为u的尚未访问的邻接顶点
                cout<<w; visited[w]=true; //访问未被访问的邻接点w, 并置访问标志数组相应分量值为true
                EnQueue(Q,w); //让该邻接点w进队
            }
        }
    } //什么时候队列为空, 就表明遍历结束
}
```

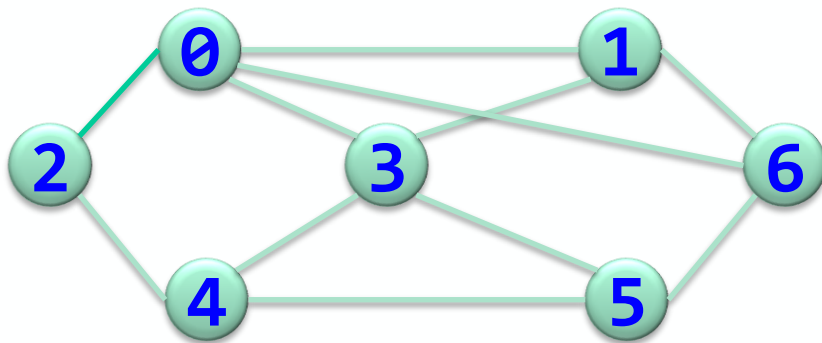
【练习12】 对如下所示的无向图，从顶点0开始进行BFS，不可能得到的顶点访问序列是（ ）。

A. 0231645

B. 0163254

C. 0623154

D. 0243165

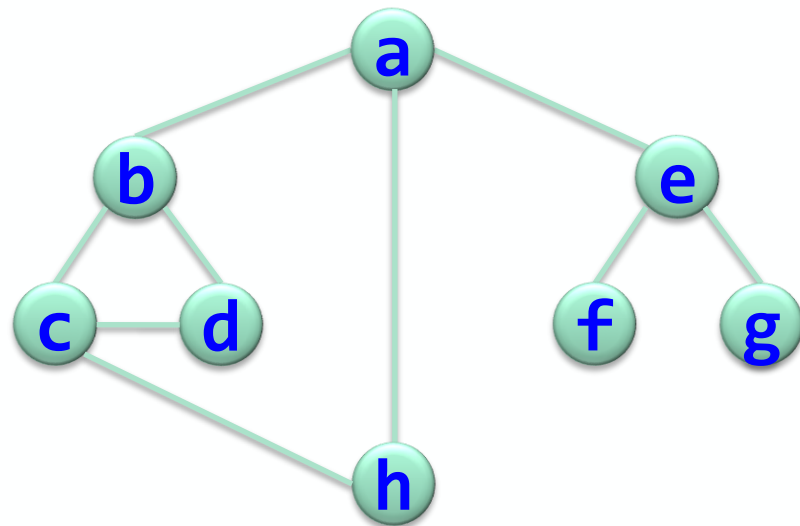


答案： D

解释： D. 0243165，顶点0到2、3、1、6的距离均为1，0的后面一定为2/3/1/6的排列

【练习13】 若对如下无向图进行遍历，不可能得到的广度优先遍历序列是（ ）。

- A. h,c,a,b,d,e,g,f
- B. e,a,f,g,b,h,c,d
- C. d,b,c,a,h,e,f,g
- D. a,b,c,d,h,e,f,g **DFS**



答案： D

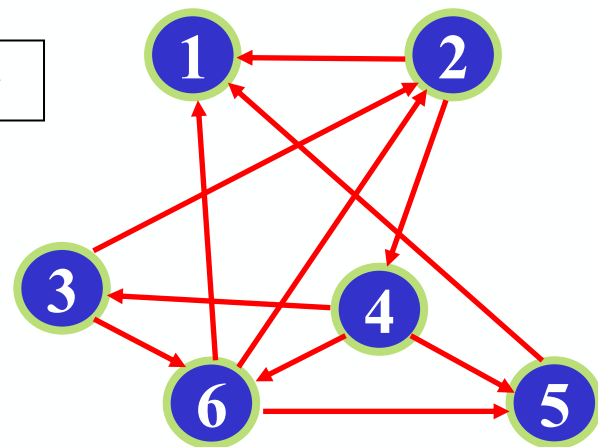
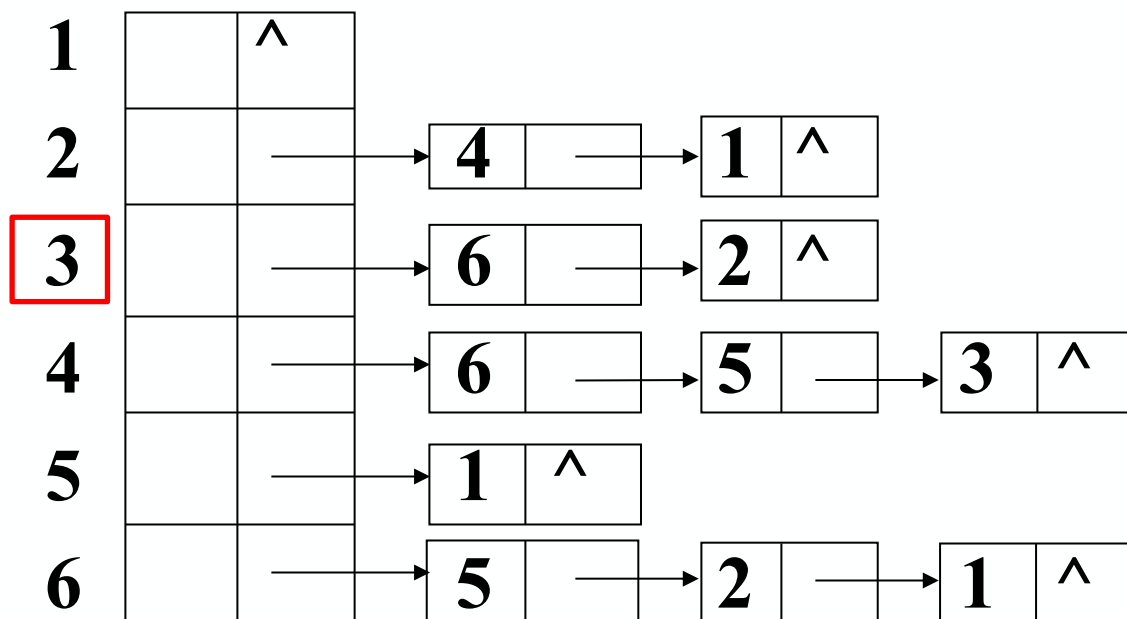
解释： 顶点a与b、 h、 e为邻接点，距离均为1，一定是a，后面为b/h/e的排列，不可能是c和d

练习

深度: 3 6 5 1 2 4

广度: 3 6 2 5 1 4

- ✓ 已知图的邻接表，分别给出用深度优先搜索和广度优先搜索从**顶点3出发**的遍历序列。



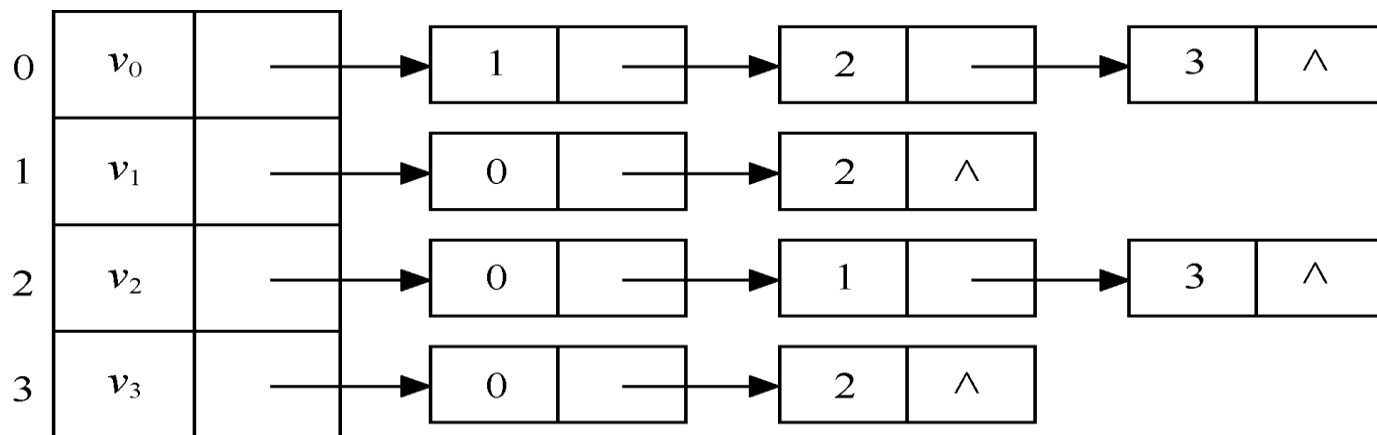
【练习14】 已知图的邻接表如图所示，若同等情况下顶点被访问的优先次序是 $v_0 v_1 v_2 v_3$ ，则从顶点 v_0 出发按广度优先遍历的结果是（ ），按深度优先遍历的结果是（ ）。

A. 0 1 3 2

B. 0 2 3 1

C. 0 3 2 1

D. 0 1 2 3



答案： D D

DFS和BFS的差别

找起点 $\Rightarrow$ 终点的一条路径



■ 看到一条没有走过、可以走的路径就直接走下去，没有道路时回退 $\Rightarrow$ **DFS**

■ 考虑每一条没有走过的路径，尝试走每条道路 $\Rightarrow$ **BFS**

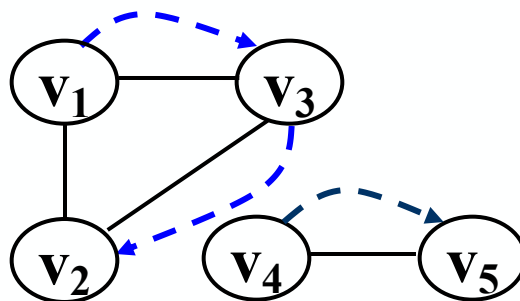
记下路径的所有路口位置：确定一条路径 **DFS和BFS试探的路口数是差不多的！**

深度优先遍历通常借助**栈**来实现算法，广度优先遍历通常借助**队列**来实现算法。

无向图的连通分量和生成树

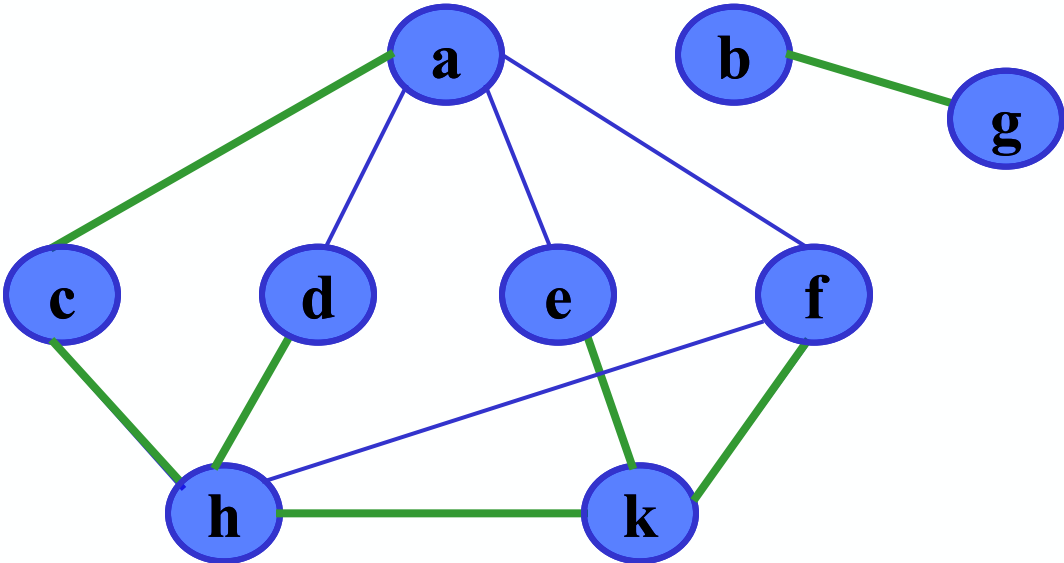
对于无向图，对其进行遍历时：

- ◆ 若是**连通图**：仅需从图中**任一顶点出发**，就能访问图中的所有顶点；
- ◆ 若是**非连通图**：需从图中**多个顶点出发**。每次从一个新顶点出发所访问的顶点集序列恰好是各个连通分量的顶点集；



(a) 无向图G

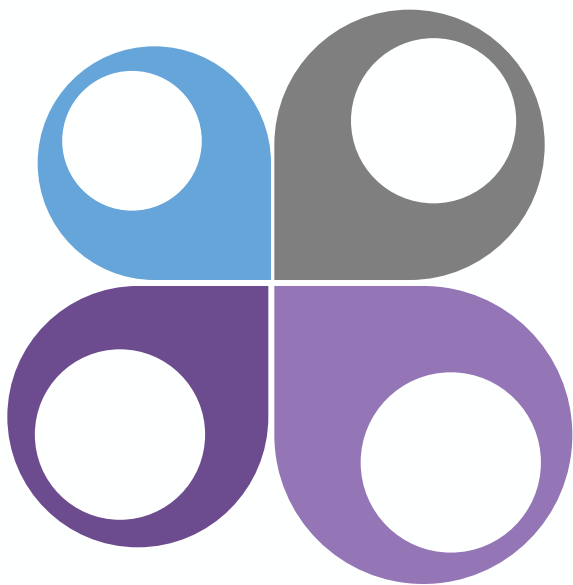
非连通图的深度优先搜索遍历



	0a	1b	2c	3d	4e	5f	6g	7h	8k
访问标志visited[i]:	1	1	1	1	1	1	1	1	1

DFS次序:	a	c	h	d	k	f	e	b	g
--------	---	---	---	---	---	---	---	---	---

▶▶▶ 6.6 图的应用



最小生成树



最短路径



拓扑排序



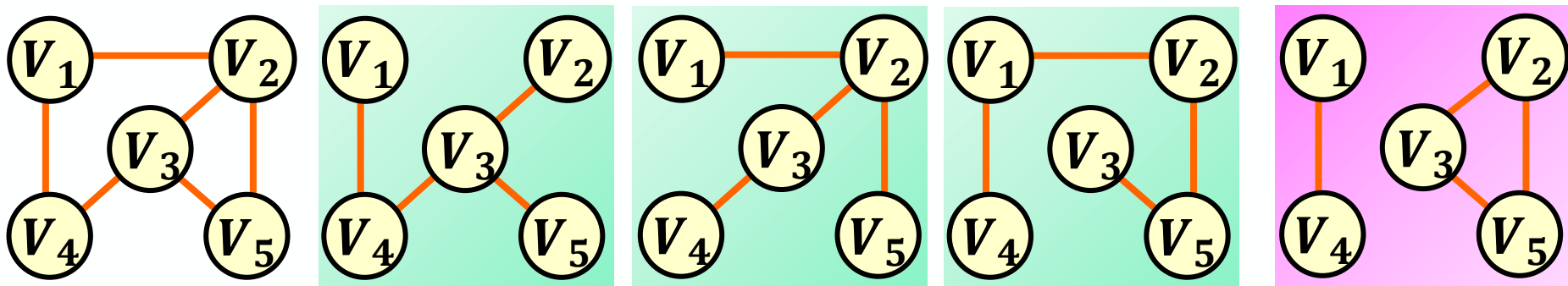
关键路径

6.1 图的定义和术语(P186)

生成树: 连通图的生成树是包含图中全部 n 个顶点的一个极小连通子图，它含有图中全部 n 个顶点和构成该生成树的 $n-1$ 条边。

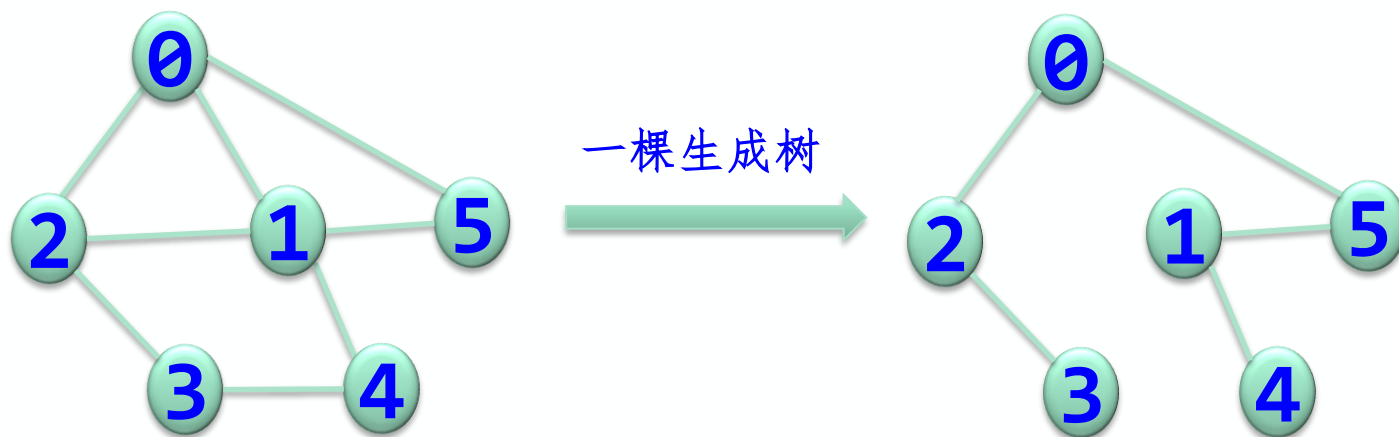
一个图可以有許多棵不同的生成树，关于生成树的几个结论：

- 生成树的顶点个数与图的顶点个数相同；
- 有 n 个顶点的连通图的生成树有且仅有 $n-1$ 条边；
- 去掉生成树的一条边则非连通，生成树中再加一条边必然形成回路
- 有 n 个顶点、 $n-1$ 条边的图不一定是生成树。



6.6.1 生成树

一个连通图的**生成树**是一个极小连通子图，它含有图中所有的 n 个顶点和构成树的 $n-1$ 条边。

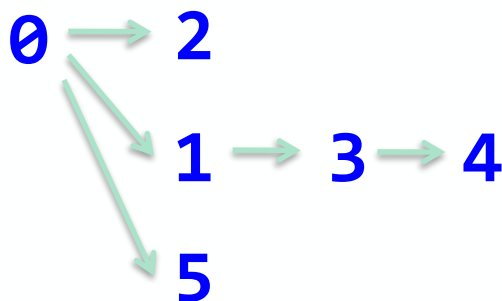
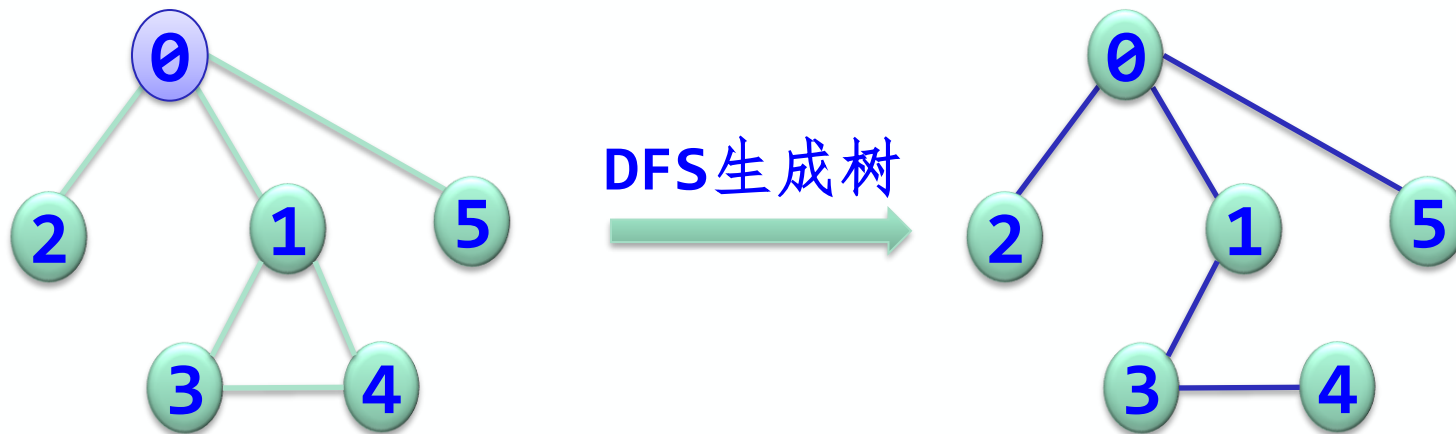


说明：即在该图中删除任何一条边，图就不再连通，添加一条边，必定构成一个环。

图的遍历是许多图的操作的基础，遍历能访问图中全部顶点。通过遍历构造生成树，遍历过程中走过的边形成生成树的边。

可以通过遍历方法产生生成树：

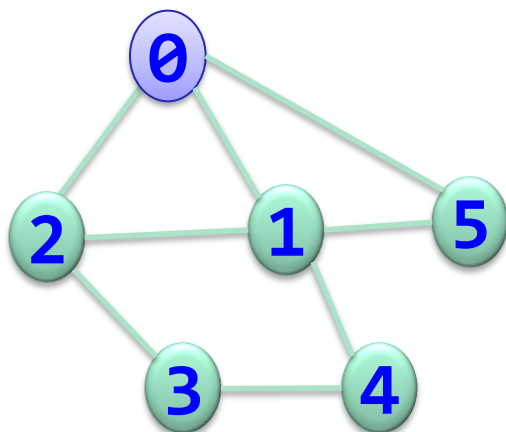
- 从任意顶点出发由深度优先遍历得到的生成树称为**深度优先生成树**。



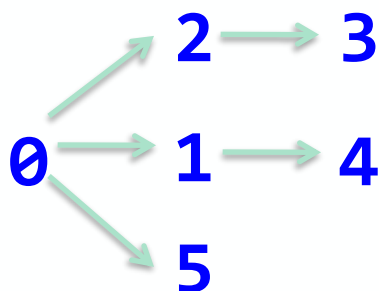
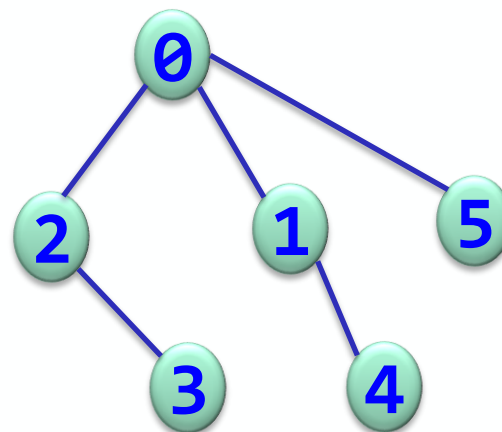
**包含全部的6个顶点，5条边，
且任意两个顶点间都存在路径**

可以通过遍历方法产生生成树：

- 从任意顶点出发由广度优先遍历得到的生成树称为**广度优先生成树**。

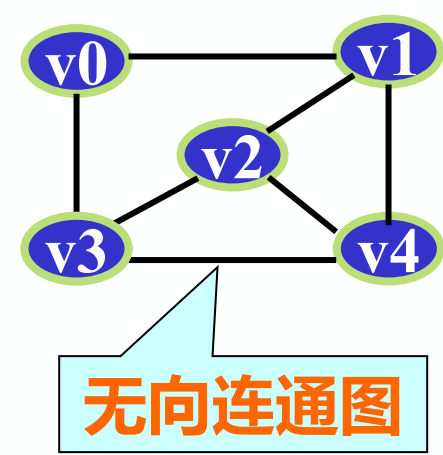


BFS生成树



包含全部的6个顶点，5条边，
且任意两个顶点间都存在路径

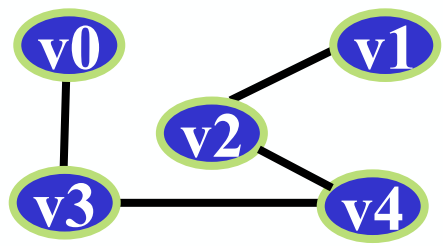
【例】已知图的邻接表，试画出自顶点v0出发进行遍历的深度优先生成树和广度优先生成树。



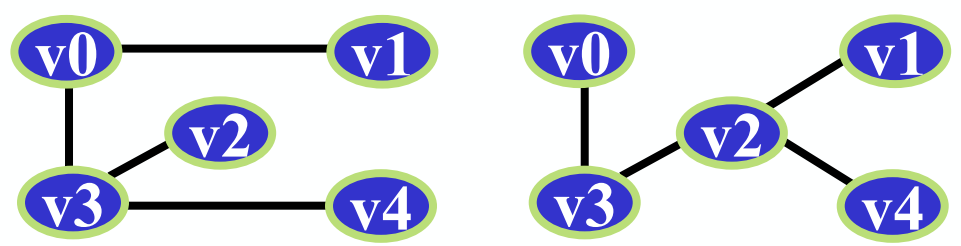
邻接表

0	$\mathbf{v}_0$		→	3		→	1	^			
1	$\mathbf{v}_1$		→	4		→	2		→	0	^
2	$\mathbf{v}_2$		→	4		→	3		→	1	^
3	$\mathbf{v}_3$		→	4		→	2		→	0	^
4	$\mathbf{v}_4$		→	3		→	2		→	1	^

DFS
生成树



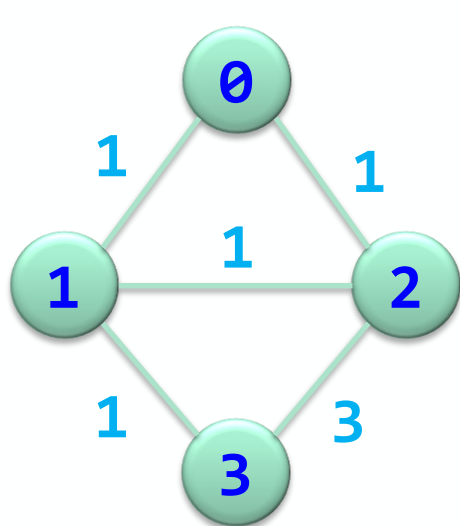
BFS
生成树



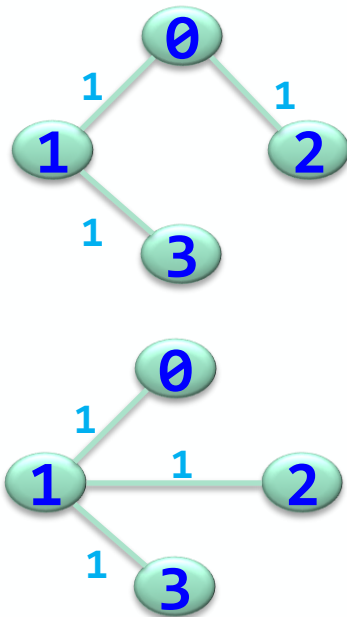
一个连通图的生成树不一定是唯一的！

6.6.1 最小生成树 (P200)

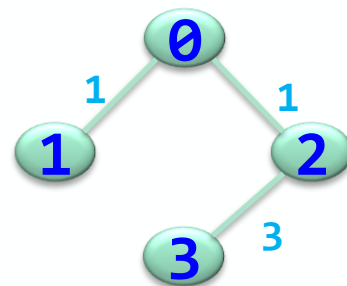
- 对于带权连通图G，使用不同的遍历方法能得到不同的生成树；从不同的顶点出发，也能得到不同的生成树；
- 带权连通图中每棵生成树的所有边的权值之和可能不同。



带权连通图G



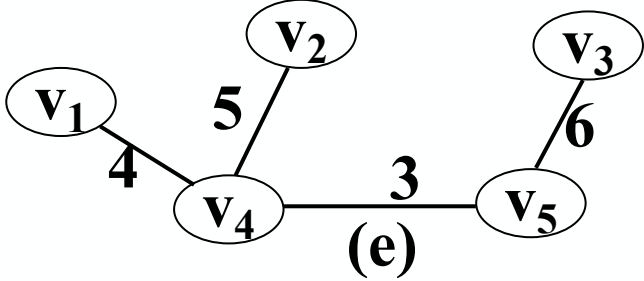
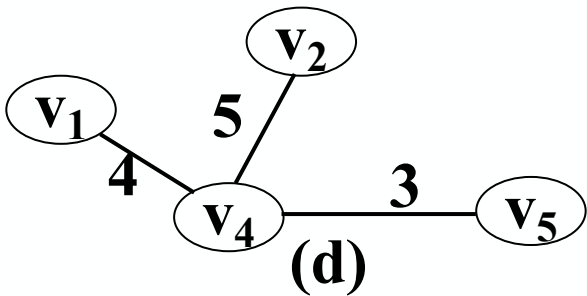
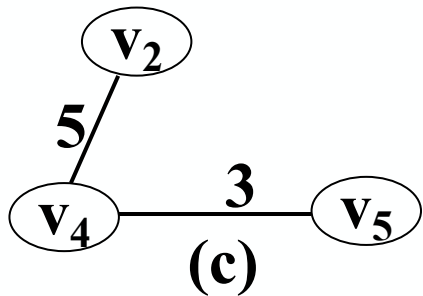
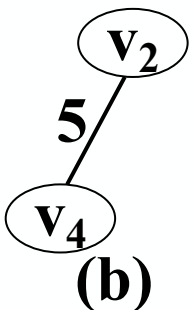
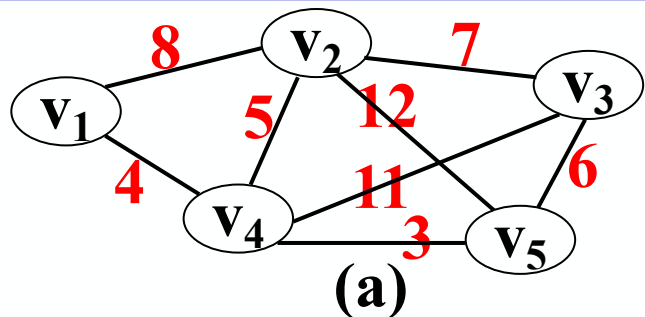
都是图G的**最小生成树**



不是图G的最小生成树

最小生成树：在带权连通图多个生成树中，寻找一个**权值之和的最小生成树**。

【例】利用普里姆算法，写出对带权连通网G1从v2开始构造最小生成树的过程



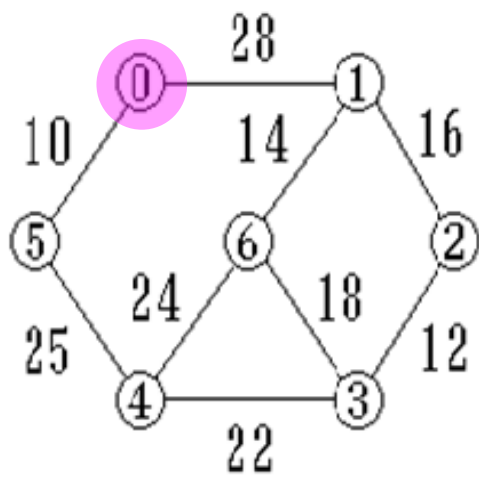
最小生成树

U: 当前顶点	V-U : 剩余顶点	TE: 权值最小的边
{v2}	{v1,v3,v4,v5}	{v2,v4}
{v2,v4}	{v1,v3,v5}	{v4,v5}
{v2,v4,v5}	{v1,v3}	{v4,v1}
{v2,v4,v5,v1}	{v3}	{v5,v3}
{v2,v4,v5,v1,v3}	{}	

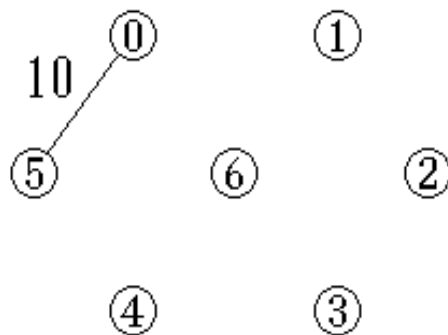
加点法

对连通网G1构造最小生成树过程中辅助参量的变化

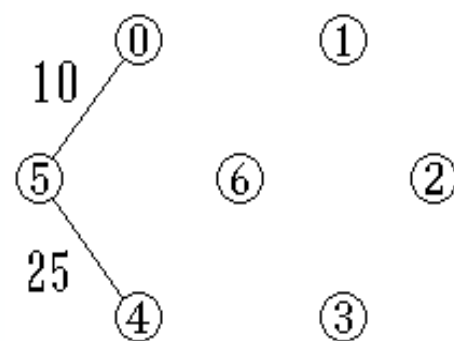
6.6.1 最小生成树——普里姆算法示例(P200)



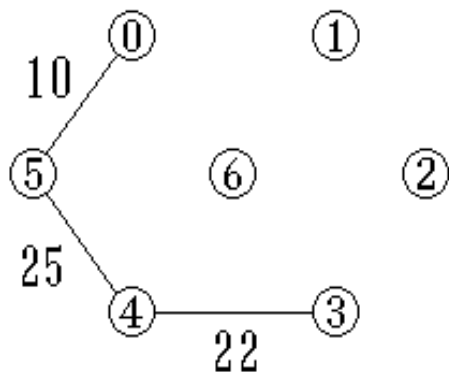
(a)



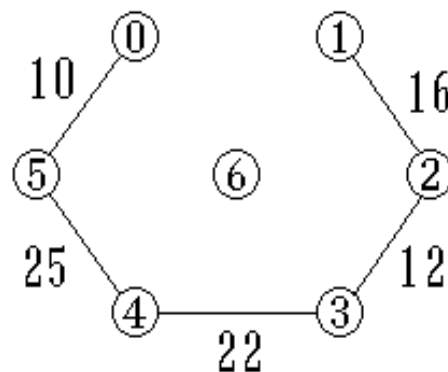
(b)



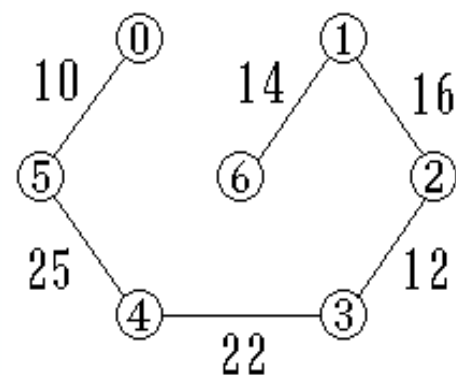
(c)



(d)



(e)

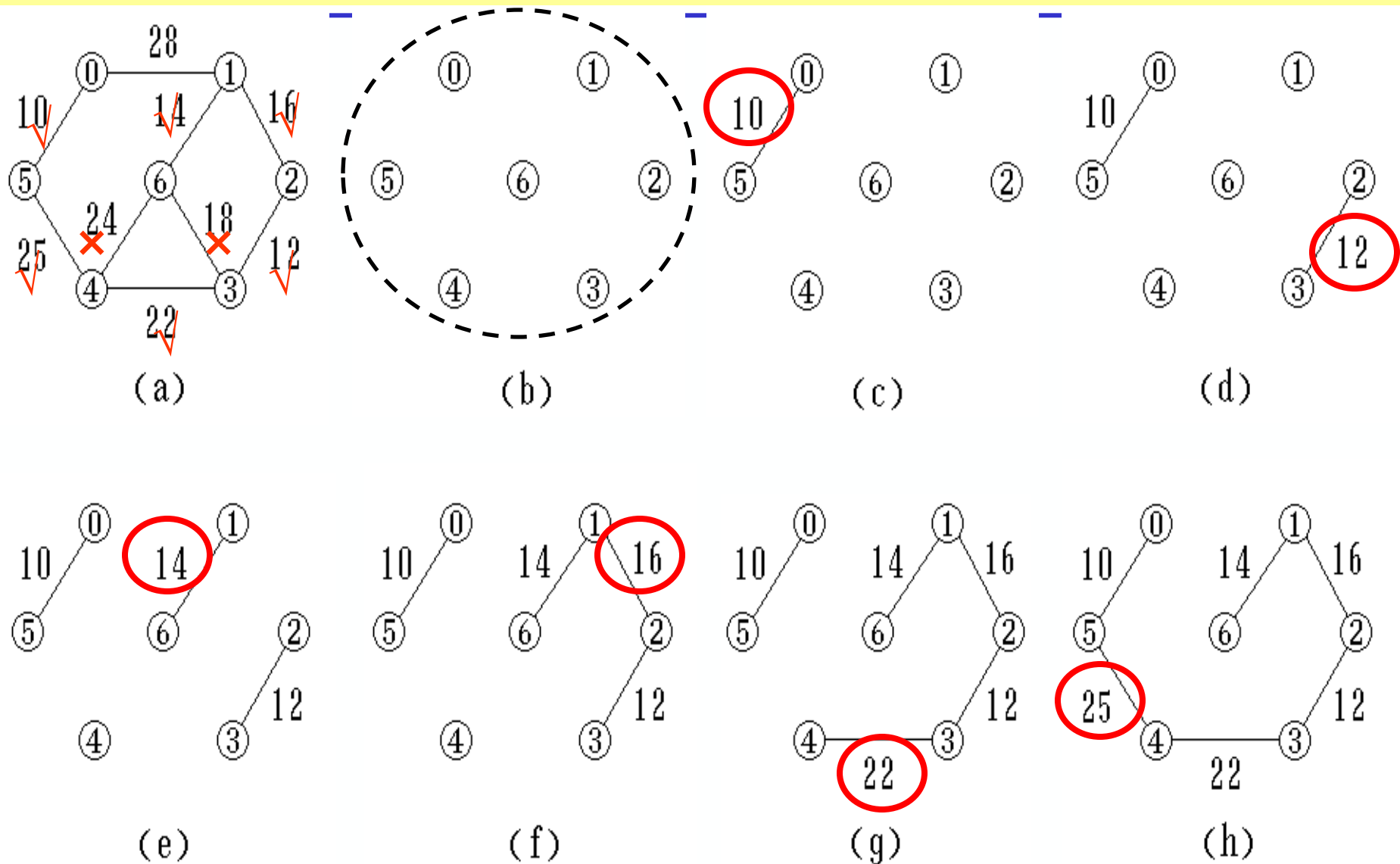


(f)

按Prim算法从顶点0出发构造最小生成树的过程

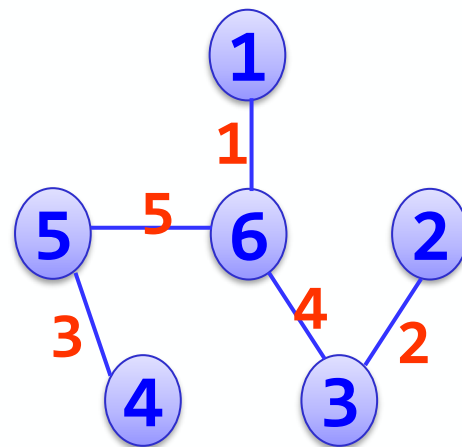
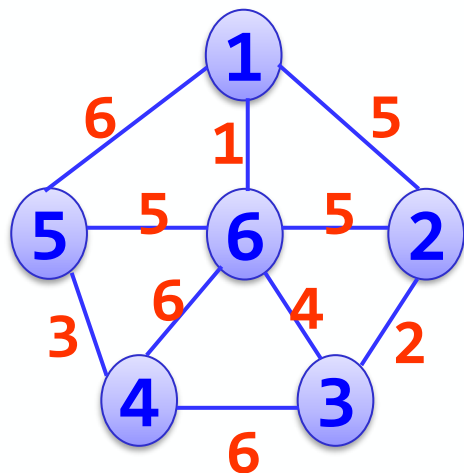
Prim算法: 适合于边稠密的网

6.6.1 最小生成树——克鲁斯卡尔算法示例(P203)



先把全部顶点加入，再按权值的递增次序选择不构成环的边。

6.6.1 最小生成树——克鲁斯卡尔算法示例



最小生成树

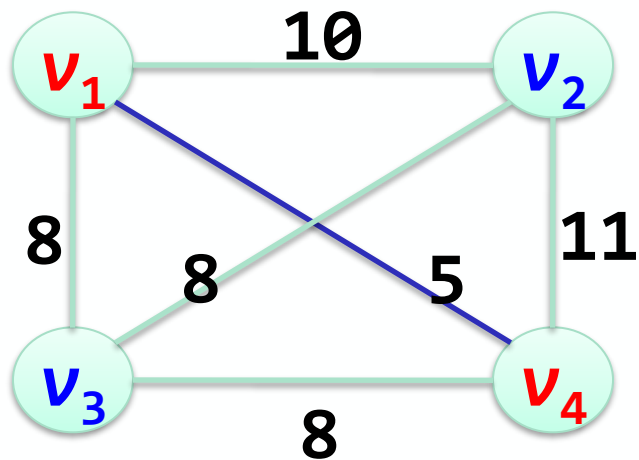
共选取了 $n-1$ 条边

克鲁斯卡尔算法求解最小生成树的过程

Kruskal算法：适合于边稀疏的网

【例16】求下面带权图的最小生成树时，可能是克鲁斯卡算法第二次选中，但不是普里姆算法（从 v_4 开始）第二次选中的边是（ ）

- A. (v_1, v_3) B. (v_1, v_4) C. (v_2, v_3) D. (v_3, v_4)



Kruskal:

- 1: (v_1, v_4)
- 2: (v_1, v_3) 或 (v_3, v_4) 或 (v_2, v_3)

Prim (从 v_4 开始):

- 1: (v_1, v_4)
- 2: (v_1, v_3) 或 (v_3, v_4) ,
不可能是 (v_2, v_3)

答案为C

6.6.1 最小生成树

- ❖ Prim (普里姆) 算法
- ❖ Kruskal (克鲁斯卡尔) 算法

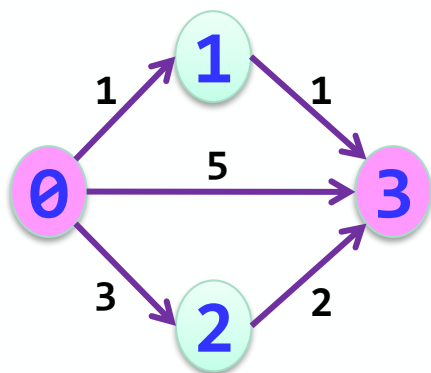
算法名	普里姆算法	克鲁斯卡尔算法
算法思想	归并顶点	归并边
时间复杂度	$O(n^2)$	$O(e \log e)$
适用范围	稠密图	稀疏图

6.6.2 最短路径 (P205)

典型用途：交通问题。如：城市A到城市B有多条线路，但每条线路的交通费（或所需时间）不同，那么如何选择一条线路，使总费用（或总时间）最少？

问题抽象：在带权有向图中源点到达终点的多条路径中，寻找一条各边权值之和最小的路径，即最短路径。

带权有向图



顶点0到3的路径：0→1→3 0→3 0→2→3

● 最短路径：0→1→3 最短路径长度：2

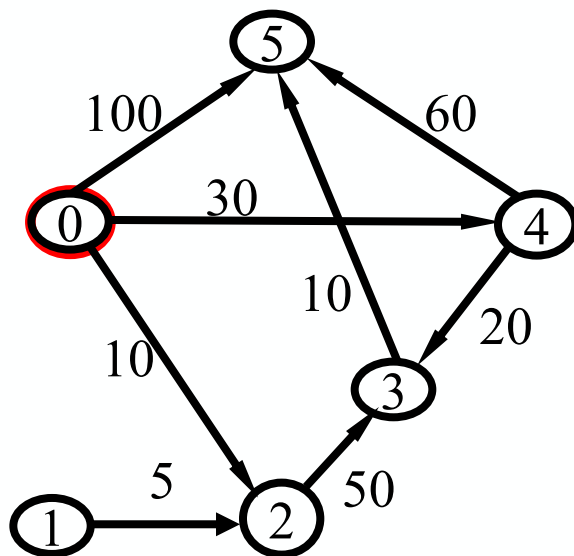
(注：最短路径与最小生成树不同，最短路径上不一定包含全部的n个顶点)

①单源最短路径

②每一对顶点之间的最短路径

【例】在带权有向图G1中求从 v_0 到其余各顶点之间的最短路径

源 点	终 点	最短路径	路径长度
v_0	v_2	(v_0, v_2)	10
	v_4	(v_0, v_4)	30
	v_3	(v_0, v_4, v_3)	50
	v_5	(v_0, v_4, v_3, v_5)	60
	v_1	无	∞



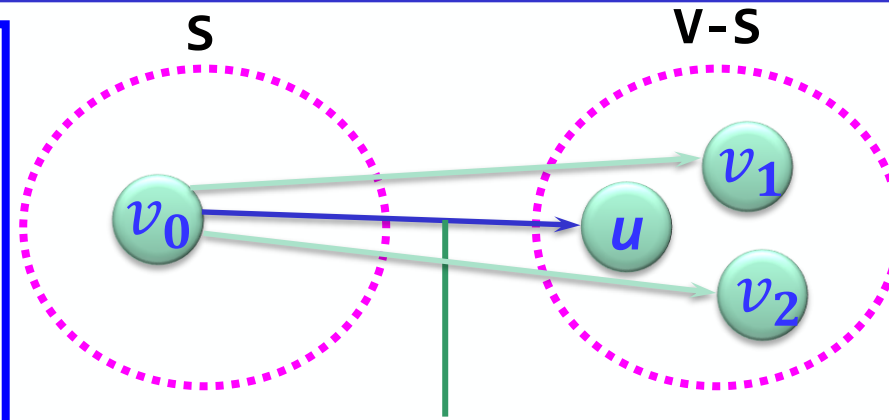
带权有向图G1

6.6.2 单源最短路径 —— Dijkstra算法 (P206)

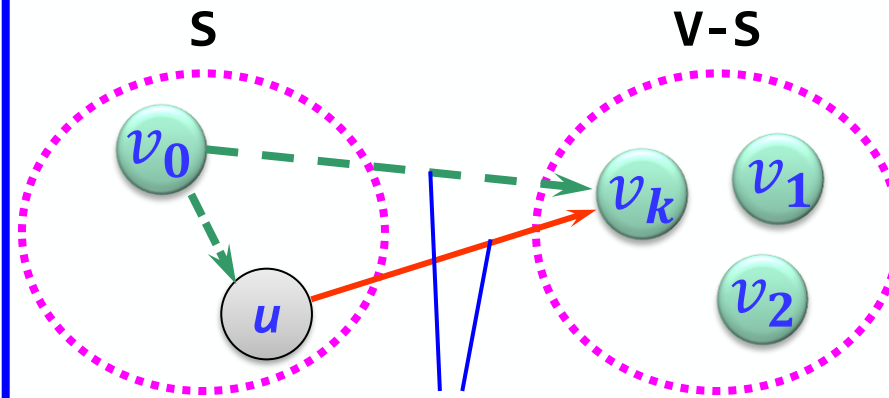
1. **初始化**: 先找出从**源点** v_0 到各**终点** v_k 的**直达**路径 (v_0, v_k) , 即通过一条弧到达的路径。

2. **选择**: 从这些直达路径中找出一条**长度最短**的路径 (v_0, u) 。

3. **更新**: 然后对其余各条路径进行适当**调整**: 若图中存在弧 (u, v_k) 且 $(v_0, u) + (u, v_k) < (v_0, v_k)$, 则以路径 (v_0, u, v_k) 代替 (v_0, v_k) 在调整后的各条路径中, 再找**长度最短的路径**, 依此类推, 直到所有顶点都找到最短路径。



v_0 与V-S中顶点 u 的弧长最小



两条路径进行比较:
若经过 u 的**最短路径长度**更短, 则修正

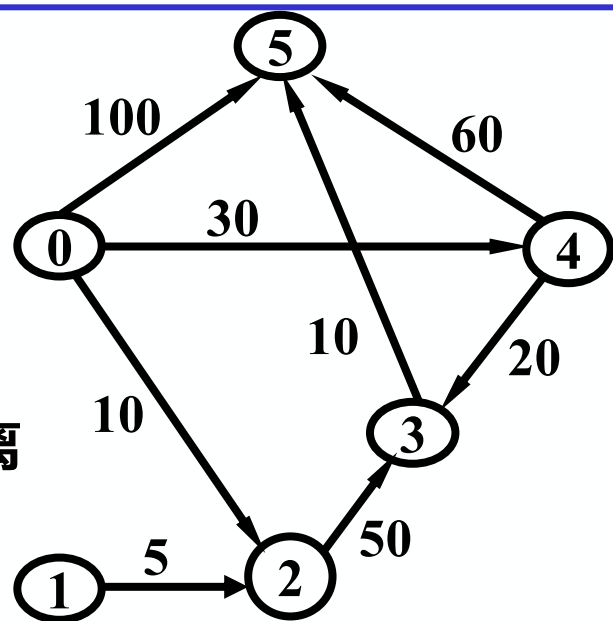
将图中的顶点集 V 分成两组: ① S : 已求出最短路径的顶点集合。

② $V-S$: 尚未确定最短路径的顶点。

如果到某顶点的路径最短, 那就将该顶点加入 S 集合中

存储结构 (顶点个数为n)

- 主：邻接矩阵 $G[n][n]$ (或者邻接表)
- 辅：
 - 数组 $D[n]$ ：记录源点到相应顶点路径长度
 - 数组 $Path[n]$ ：记录源点0到i的当前最短路径
 - 数组 $S[n]$ ：记录相应顶点**是否**已被确定最短距离
true：已被确定 false：尚未被确定



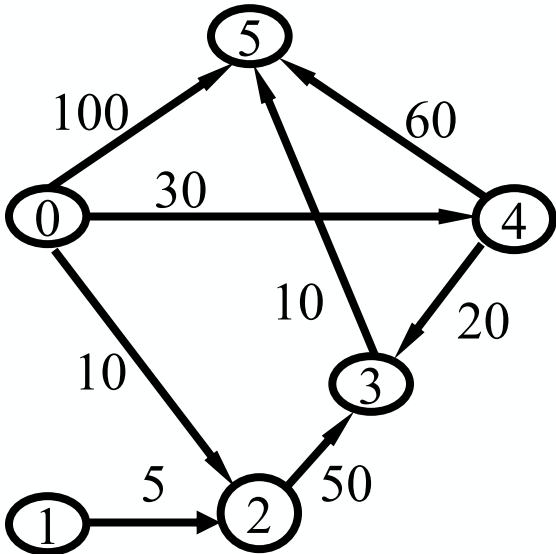
初始化：先找出从源点到各终点的直达路径 (v_0, v_k)，通过一条弧到达的路径

当源点是0	$v = 1$	$v = 2$	$v = 3$	$v = 4$	$v = 5$
D	∞	10	∞	30	100
$Path$		$\{v_0, v_2\}$		$\{v_0, v_4\}$	$\{v_0, v_5\}$
S	false	true	false	false	false

Dijkstra (迪杰斯特拉) 算法初始化状态

例 (P206)

∞	∞	10	∞	30	100
∞	∞	5	∞	∞	∞
∞	∞	∞	50	∞	∞
∞	∞	∞	∞	∞	10
∞	∞	∞	20	∞	60
∞	∞	∞	∞	∞	∞



终点	D[w]	从v ₀ 到各终点的长度和最短路径			
v ₁	∞	∞	∞	∞	∞
v ₂	10 {v₀,v₂}				
v ₃	∞	60 {v ₀ ,v ₂ ,v ₃ }	50 {v₀,v₄,v₃}		
v ₄	30 {v ₀ ,v ₄ }	30 {v₀,v₄}			
v ₅	100 {v ₀ , v ₅ }	100 {v ₀ , v ₅ }	90 {v ₀ ,v ₄ , v ₅ }	60 {v₀,v₄,v₃,v₅}	
新加入 顶点 U	v ₂	v ₄	v ₃	v ₅	
S	S[2]=true {v ₀ ,v ₂ }	S[4]=true {v ₀ ,v ₂ ,v ₄ }	S[3]=true {v ₀ ,v ₂ ,v ₄ ,v ₃ }	S[5]=true {v ₀ ,v ₂ ,v ₄ ,v ₃ ,v ₅ }	

最短路径并不一定包含全部顶点

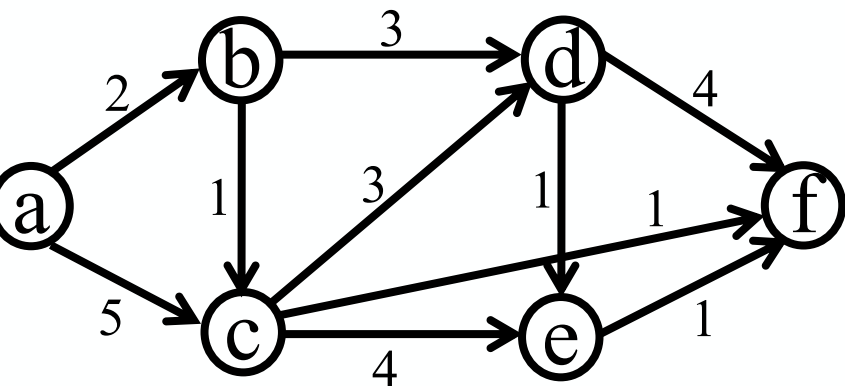
【练习17】 对于如图的有向带权图，用Dijkstra算法求源点a到各顶点的最短路径，则得到的第一条最短路径的目标顶点是b，第二条最短路径的目标顶点是c，后续得到的其余各最短路径的目标顶点依次是（ ）。

A.d,e,f

B.e,d,f

C.f,d,e

D.f,e,d

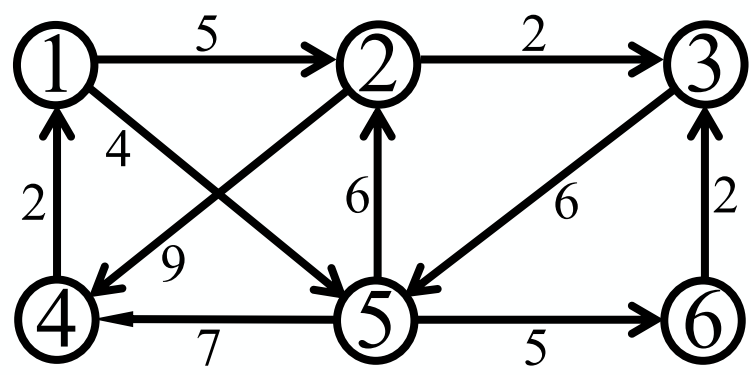


答案： C

终点	D[w] 从a到各终点的长度和最短路径				
b	2 {a,b}				
c	5 {a,c}	3 {a,b,c}			
d	∞	5 {a,b,d}	5 {a,b,d}	5 {a,b,d}	
e	∞	∞	7 {a,b,c,e}	7 {a,b,c,e}	6 {a,b,d,e}
f	∞	∞	4 {a,b,c,f}		
u	b	c	f	d	e
S	{a,b}	{a,b,c}	{a,b,c,f}	{a,b,c,f,d}	{a,b,c,f,d,e}

【练习18】 使用Dijkstra算法求下图源点1到其他各顶点的最短路径，依次得到的各最短路径的目标顶点是（ ）。

- A.5,2,3,4,6
- B.5,2,3,6,4
- C.5,2,4,3,6
- D.5,2,6,3,4



答案： B

最短路径是允许有环的

终点	D[w] 从1到各终点的长度和最短路径				
2	5 {1,2}	5 {1,2}			
3	∞	∞	7 {1,2,3}		
4	∞	11 {1,5,4}	11 {1,5,4}	11 {1,5,4}	11 {1,5,4}
5	4 {1,5}				
6	∞	9 {1,5,6}	9 {1,5,6}	9 {1,5,6}	
u	5	2	3	6	4
S	{1,5}	{1,5,2}	{1,5,2,3}	{1,5,2,3,6}	{1,5,2,3,6,4}

每一对顶点间最短路径 —— Floyd算法 (P207)

算法思想：逐个顶点试探，从 v_i 到 v_j 的所有可能存在的路径中，选出一条长度最短的路径

设顶点集 S (初值为空)，用矩阵 D 的每个元素 $D[i][j]$ 保存从 v_i 只经过 S 中的顶点到达 v_j 的最短路径长度：

① 初始时令 $S=\{\}$ ， $D[i][j]$ 的赋初值方式是：

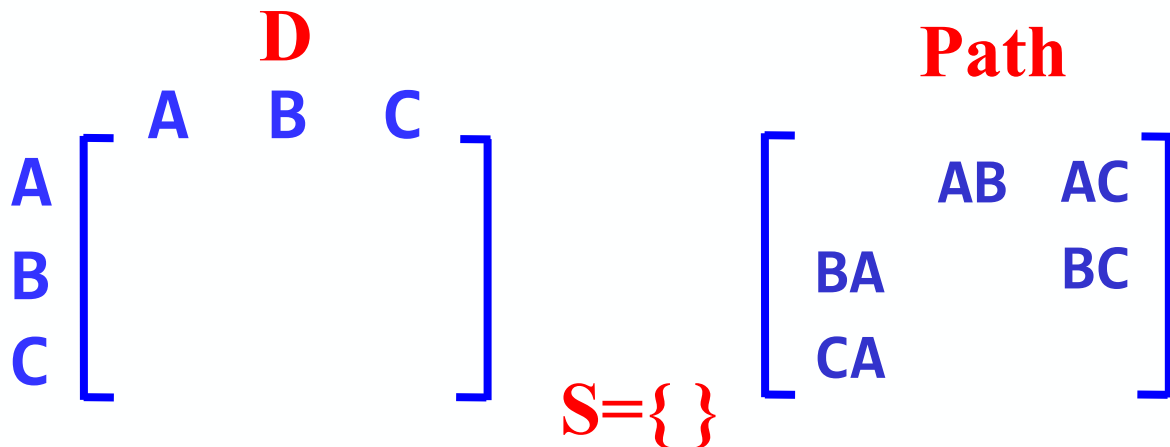
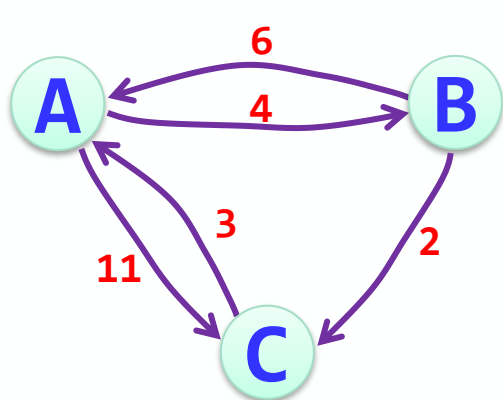
$$D[i][j]=\begin{cases} 0 & i=j\text{时} \\ w_{ij} & i\neq j\text{且}\langle v_i, v_j \rangle \in E, \text{ } w_{ij}\text{为弧上的权值} \\ \infty & i\neq j\text{且}\langle v_i, v_j \rangle \text{不属于} E \end{cases}$$

② 将图中一个顶点 v_k 加入到 S 中，修改 $D[i][j]$ 的值，修改方法是：

$$D[i][j]=\text{Min}\{D[i][j], (D[i][k]+D[k][j])\}$$

③ 重复②，直到 G 的所有顶点都加入到 S 中为止。

例



设顶点集S(初值为空), 用矩阵D的每个元素D[i][j]保存从 v_i 只经过S中的顶点到达 v_j 的最短路径长度:

① 初始时令 $S=\{ \}$, D[i][j]的赋初值方式是:

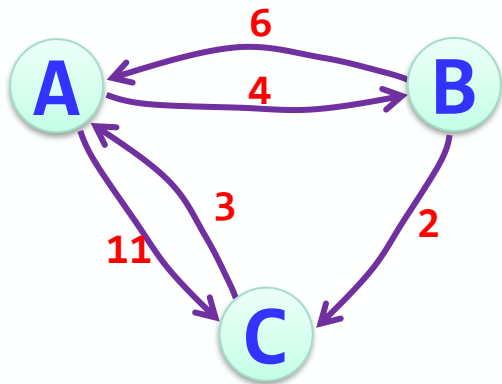
$$D[i][j] = \begin{cases} 0 & i=j \text{ 时} \\ w_{ij} & i \neq j \text{ 且 } \langle v_i, v_j \rangle \in E, \text{ } w_{ij} \text{ 为弧上的权值} \\ \infty & i \neq j \text{ 且 } \langle v_i, v_j \rangle \text{ 不属于 } E \end{cases}$$

② 将图中一个顶点 v_k 加入到S中, 修改D[i][j]的值, 修改方法是:

$$D[i][j] = \min\{D[i][j], (D[i][k] + D[k][j])\}$$

例

步骤	初态	加入A	加入B	加入C
D	$\begin{bmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 3 & \infty & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 4 & 6 \\ 6 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 4 & 6 \\ 5 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix}$
Path	$\begin{bmatrix} & AB & AC \\ BA & & \\ CA & & \end{bmatrix}$	$\begin{bmatrix} & AB & AC \\ BA & & BC \\ CA & CAB & \end{bmatrix}$	$\begin{bmatrix} & AB & ABC \\ BA & & BC \\ CA & CAB & \end{bmatrix}$	$\begin{bmatrix} & AB & ABC \\ BCA & & BC \\ CA & CAB & \end{bmatrix}$
S	{ }	{ A }	{ A, B }	{ A, B, C }



根据上述过程中Path矩阵，得出：

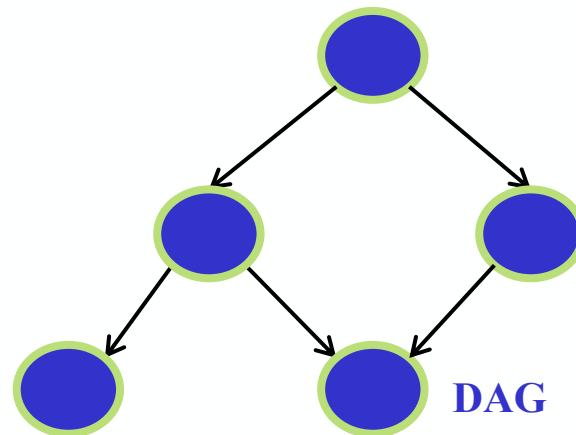
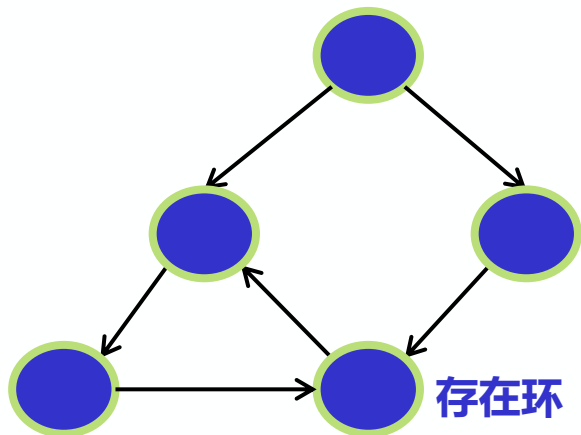
A到B：最短路径是{ A, B }，路径长度是4；

A到C：最短路径是{ A, B, C }，路径长度是6；

.....

6.6.3 有向无环图及其应用

有向无环图：一个有向图中不存在环，则称为有向无环图（DAG）



AOV-网：拓扑排序



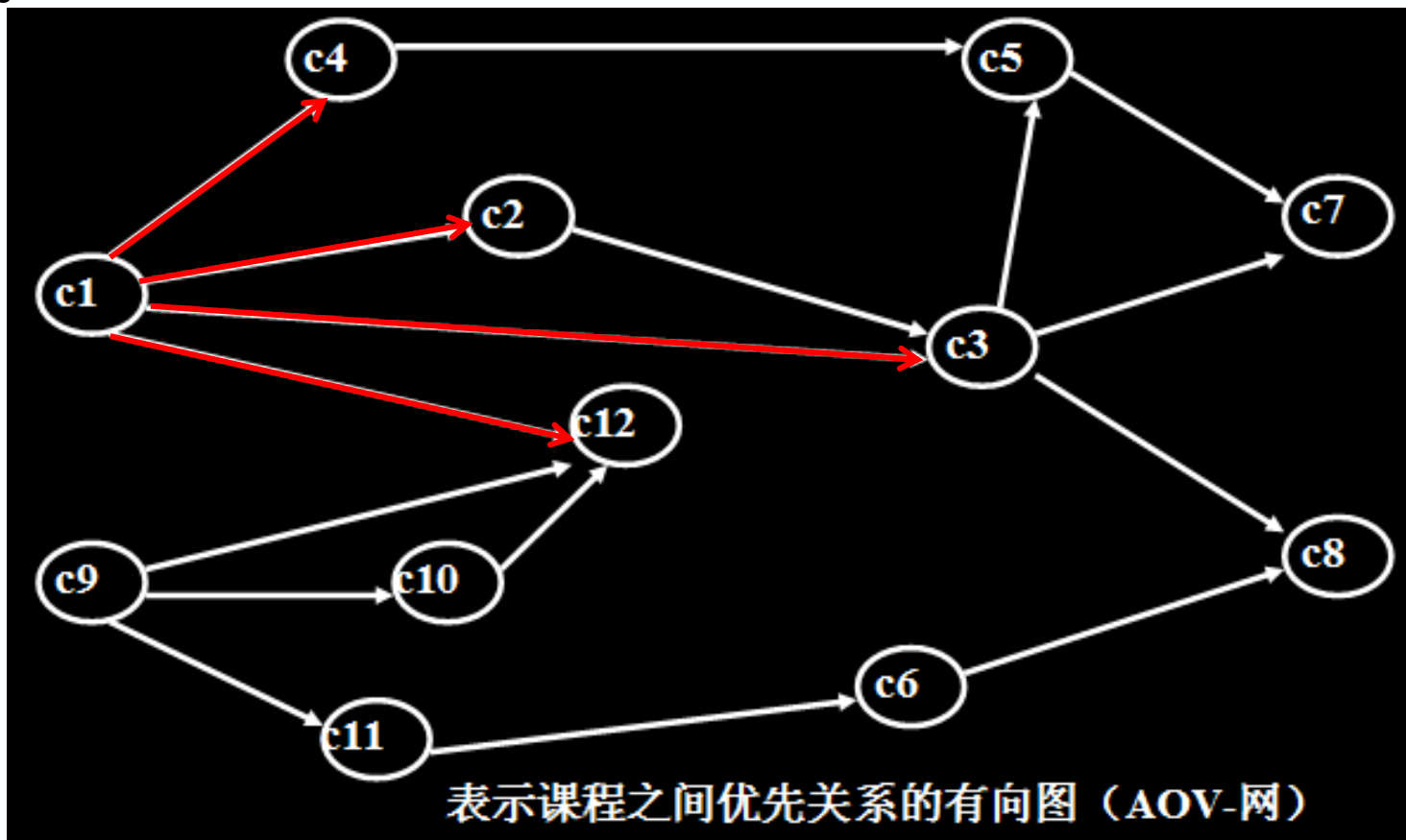
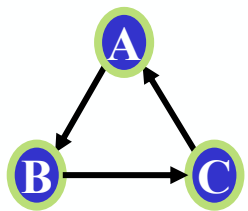
如果用DAG表示一个工程的各子工程及其相互制约的关系。其中**顶点表示活动**，**弧 $\langle v_i, v_j \rangle$ 表示活动之间的优先制约关系**：活动 v_i 必须先于活动 v_j ，这种**有向图称为顶点表示活动的网**，简称**AOV-网**（Activity On Vertex）。

计算机专业学生的必修课程

课程编号	课程名称	需要的先导课程编号
C1	程序设计基础	无
C2	离散数学	C1
C3	数据结构	C1， C2
C4	汇编语言	C1
C5	算法分析与设计	C3， C4
C6	计算机组成原理	C11
C7	编译原理	C5， C3
C8	操作系统	C3， C6
C9	高等数学	无
C10	线性代数	C9
C11	普通物理	C9
C12	数值分析	C9， C10， C1

拓扑排序

AOV-网中**顶点表示活动**，顶点之间的**弧就表示活动先后相互制约关系**。在网中，从顶点 v_i 到顶点 v_j 有一条有向路径，则 v_i 是 v_j 的直接前驱， v_j 是 v_i 的直接后继。



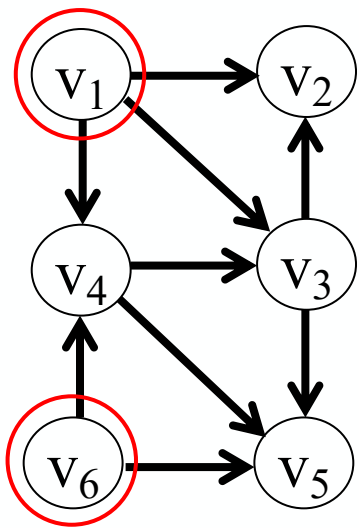
AOV-网中**不允许有回路**，因为如果有回路存在，则表明某项活动以自己为先决条件，显然这是荒谬的。

拓扑排序

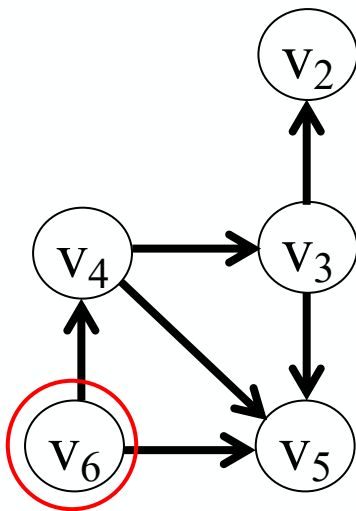
- 对给定的AOV-网应首先判定网中**是否存在环**。
- 检测的办法是对所有的顶点进行拓扑排序，若网中**所有顶点**都在它的拓扑排序的有序序列中，则该AOV-网中必定不存在环。
- **拓扑排序**就是将AOV-网中所有顶点排成一个线性序列。
- 该序列满足：**若在AOV-网中由顶点 v_i 到顶点 v_j 有一条路径，则在拓扑序列中，顶点 v_i 必定在 v_j 之前输出。**

拓扑排序的过程

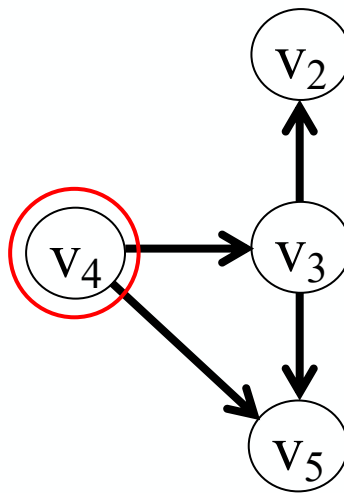
1. 在有向图中选一个**没有直接前驱的顶点**，并输出之；
2. 从图中**删去该顶点**，同时**删去所有它发出的有向边**；
3. 重复以上 1、2 步，直到**全部顶点均已输出**，拓扑排序完成。



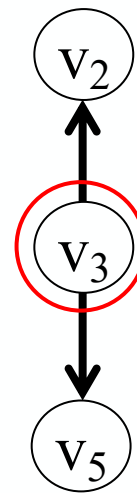
(a) 有向图



(b) 输出 v_1 后



(c) 输出 v_6 后



(d) 输出 v_4 后

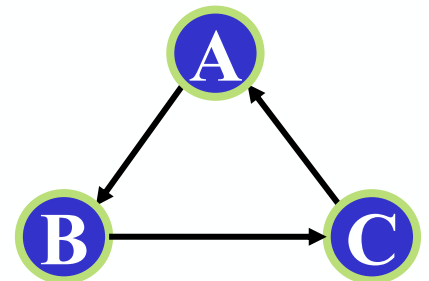


(e) 输出 v_3 后

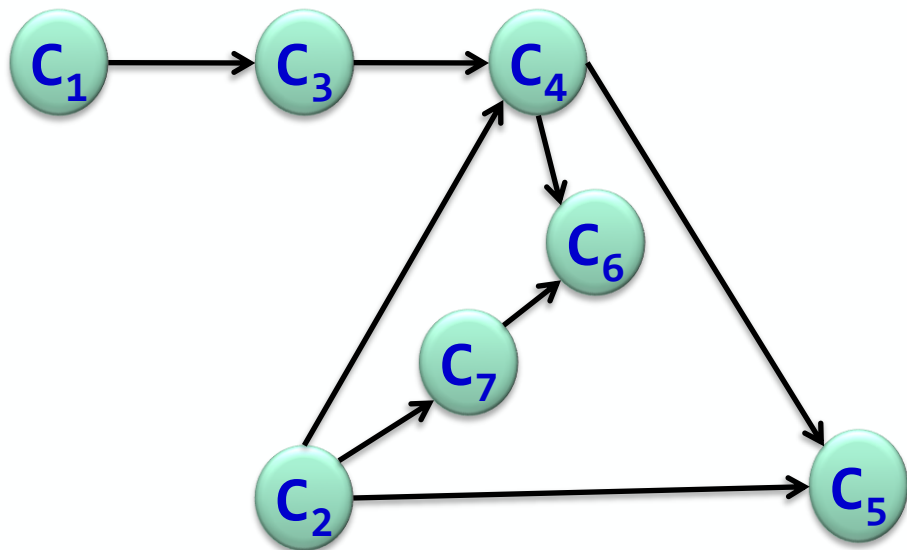
有向图的拓扑排序过程

拓扑有序序列1: $V_1 V_6 V_4 V_3 V_2 V_5$

拓扑有序序列2: $V_6 V_1 V_4 V_3 V_5 V_2$



拓扑排序演示



产生的拓扑序列:

C_1 C_3 C_2 C_7 C_4 C_6 C_5

C_1 C_3 C_2 C_4 C_7 C_6 C_5

可互换

示例

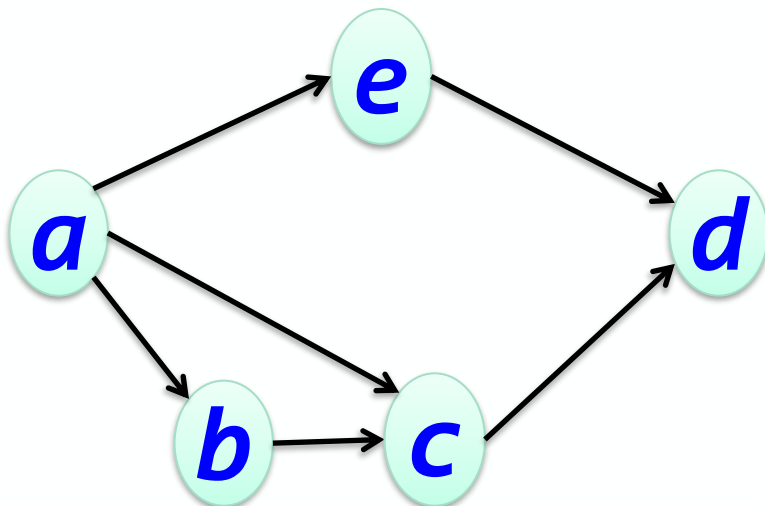
对如图所示的图进行拓扑排序，可以得到不同的拓扑序列个数是（ ）。

A. 4

B. 3

C. 2

D. 1



a e b c d

a b c e d

a b e c d

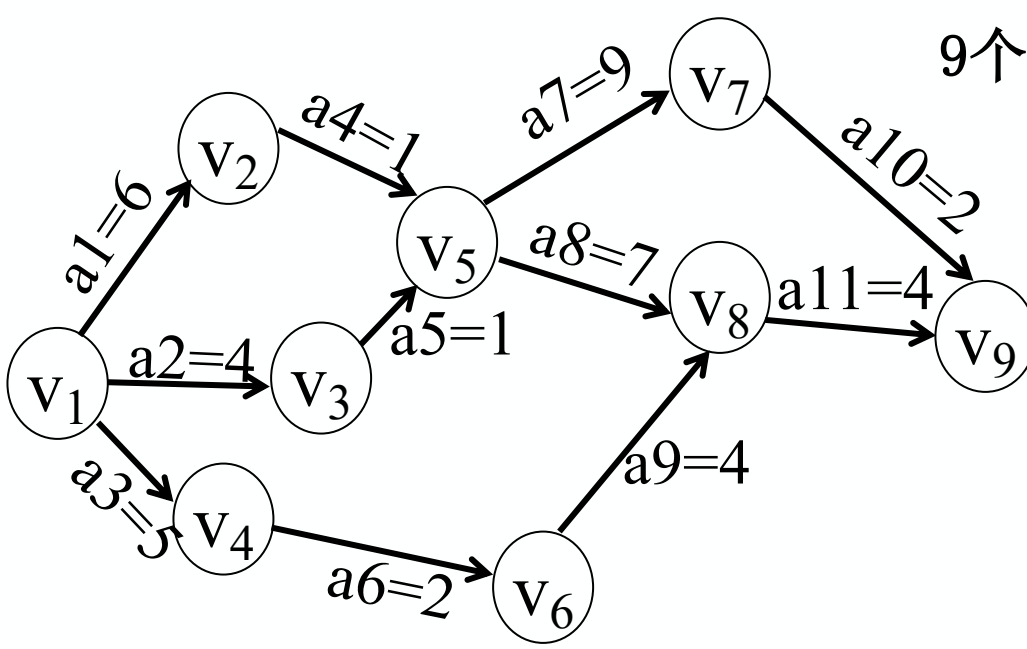
解：不同的拓扑序列有：*aebcd*、*abced*、*abecd*。
答案为**B**。e、c是d的前驱，一定在d之前输出

注：全国考研题

6.6.4 关键路径

AOE-网：关键路径

一个带权的有向无环图，**弧表示活动**，弧上权值表示相应活动所需时间或费用，**顶点表示活动的开始或结束事件**。**弧表示活动的网**简称**AOE-网** (Activity On Edge)，可用于估算工程的完成时间。



一个AOE-网

9个顶点 9个事件；11条弧 11项活动

v_5 : 表示活动 a_4 和 a_5 已完成， a_7 和 a_8 可以开始。

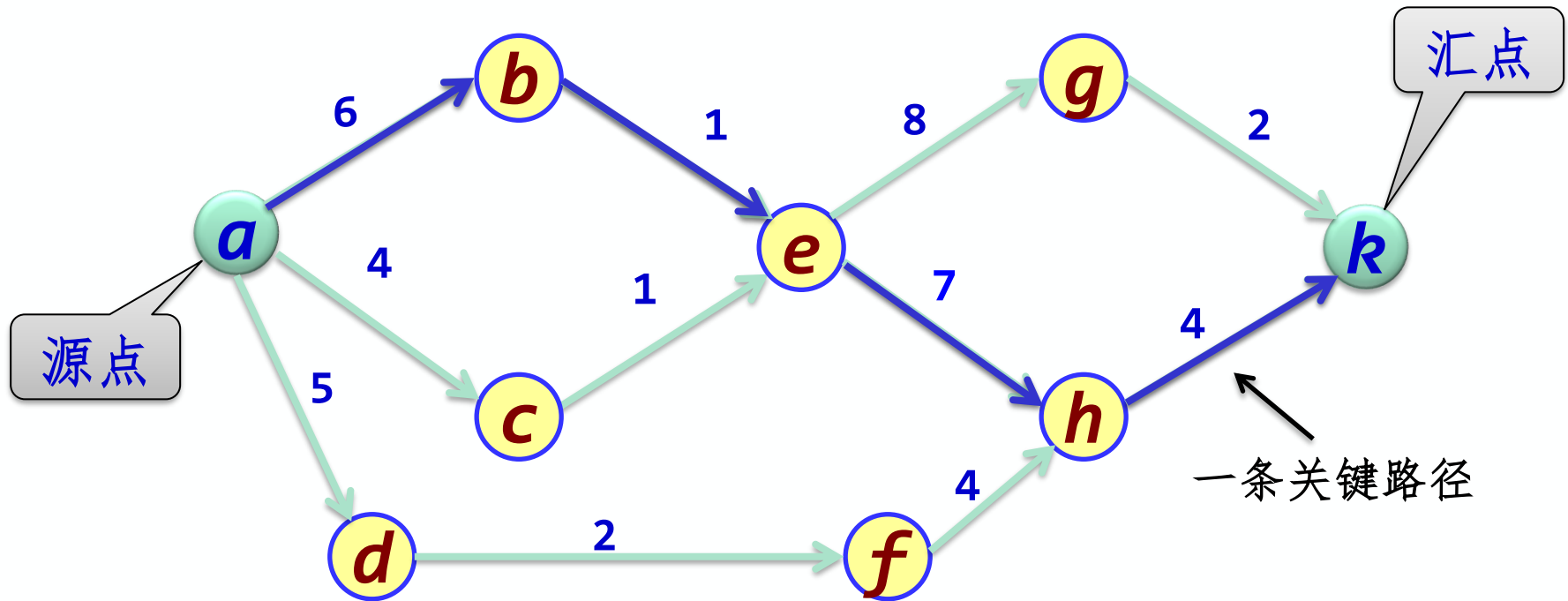
a_2 : 表示活动 a_2 持续时间是4天

事件 v_1 : 表示整个工程开始，入度为0，**源点**

事件 v_9 : 表示整个工程结束，出度为0，**汇点**

关键路径演示

- 从AOE-网中源点到汇点的最长路径，具有最大长度的路径叫**关键路径**。
- 关键路径是由**关键活动**构成的，关键路径可能不唯一。



(a,b,e,h,k)就是一条关键路径。

路径长度=关键活动 $6+1+7+4=18$

如何理解关键活动？



老师布置的作业要求一个星期交，而小明恰好需要一个星期做完，那么这个作业对于小明来说就是关键活动。

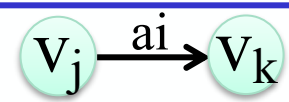


老师布置的作业要求一个星期交，而小红只需要3天做完，那么这个作业对于小红来说就不是关键活动，她有4天的富裕时间。

一个不存在富裕时间的活动必须赶紧完成，否则会影响整个工期。

6.6.4 关键路径

关键路径求解的几个步骤:



- (1) 求出^{early} $ve(k)$ ——事件的最早发生时间, $ve(\text{源点})=0$, $ve(k)=\text{Max}\{ve(j)+w_{j,k}\}$
- (2) 求出^{late} $vl(k)$ ——事件的最迟发生时间, $vl(\text{汇点})=ve(\text{汇点})$, $vl(j)=\text{Min}\{vl(k)-w_{j,k}\}$
- (3) 求出 $e(i)$ ——活动 a_i 的最早开始时间, 若活动 a_i 是弧 $\langle v_j, v_k \rangle$, $e(i)=ve(j)$
- (4) 求出 $l(i)$ ——活动 a_i 的最迟开始时间, 若活动 a_i 是弧 $\langle v_j, v_k \rangle$, $l(i)=vl(k)-w_{j,k}$
- (5) $l(i)-e(i)$ 表示完成活动 a_i 的时间余量, $e(i)=l(i)$ 为关键活动

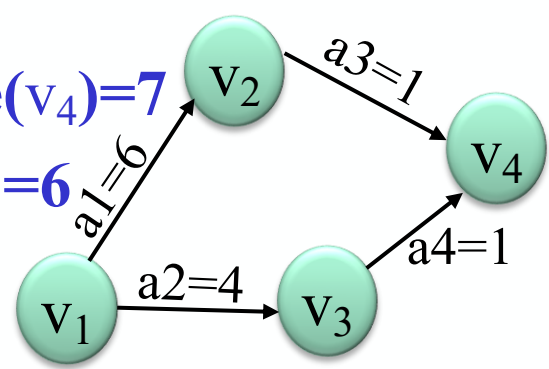
$ve(v_1)=0$ $ve(v_2)=\text{Max}\{ve(v_1)+w_{1,2}\}=6$

$ve(v_3)=\text{Max}\{ve(v_1)+w_{1,3}\}=4$

$ve(v_4)=\text{Max}\{ve(v_2)+w_{2,4}, ve(v_3)+w_{3,4}\}=7$ $vl(v_4)=ve(v_4)=7$

$vl(v_3)=\text{Min}\{vl(v_4)-w_{3,4}\}=6$ $vl(v_2)=\text{Min}\{vl(v_4)-w_{2,4}\}=6$

$vl(v_1)=\text{Min}\{vl(v_3)-w_{1,3}, vl(v_2)-w_{1,2}\}=0$

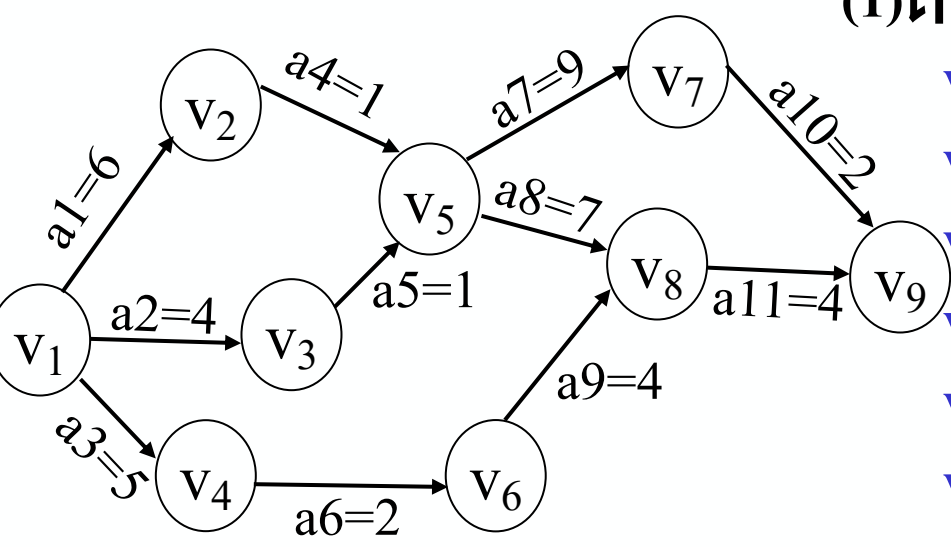


$e(1)=ve(v_1)=0$ $e(2)=ve(v_1)=0$ $e(3)=ve(v_2)=6$ $e(4)=ve(v_3)=4$

$l(4)=vl(v_4)-1=6$ $l(3)=vl(v_4)-1=6$ $l(2)=vl(v_3)-4=2$ $l(1)=vl(v_2)-6=0$

6.6.4 关键路径

【例】对图所示的AOE网，计算关键路径。



(1)计算各顶点事件 v_i 的最早发生时间 $ve(v_i)$

$$\begin{aligned} ve(v_1) &= 0 \\ ve(v_2) &= \text{MAX}\{ve(v_1) + w_{1,2}\} = 6 \\ ve(v_3) &= 4 \\ ve(v_4) &= 5 \\ ve(v_5) &= \text{MAX}\{ve(v_2) + w_{2,5}, ve(v_3) + w_{3,5}\} = 7 \\ ve(v_6) &= 7 \\ ve(v_7) &= 16 \\ ve(v_8) &= \text{MAX}\{ve(v_5) + w_{5,8}, ve(v_6) + w_{6,8}\} = 14 \\ ve(v_9) &= \text{MAX}\{ve(v_7) + w_{7,9}, ve(v_8) + w_{8,9}\} = 18 \end{aligned}$$

(2)计算各顶点事件 v_i 的最迟发生时间 $vl(v_i)$

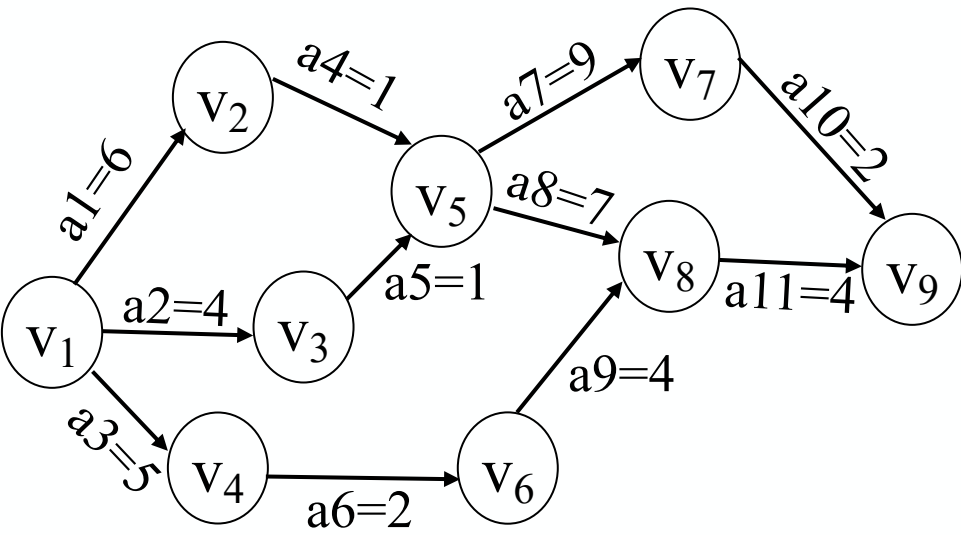
$$\begin{aligned} vl(v_9) &= ve(v_9) = 18 & vl(v_8) &= \text{MIN}\{vl(v_9) - w_{8,9}\} = 14 & vl(v_7) &= 16 & vl(v_6) &= 10 \\ vl(v_5) &= \text{MIN}\{16 - 9, 14 - 7\} = 7 & vl(v_4) &= 8 & & & vl(v_3) &= 6 & vl(v_2) &= 6 \\ vl(v_1) &= \text{MIN}\{vl(v_2) - w_{1,2}, vl(v_3) - w_{1,3}, vl(v_4) - w_{1,4}\} = 0 & & & & & & & & \end{aligned}$$

通常 $vl(\text{源点})=0$

6.6.4 关键路径

【例】对图所示的AOE网，计算关键路径。

(3)计算活动 a_i 的最早开始时间 $e(i)$



顶点i	ve(i)	vl(i)
1	0	0
2	6	6
3	4	6
4	5	8
5	7	7
6	7	10
7	16	16
8	14	14
9	18	18

- $e(1)=0$
- $e(2)=0$
- $e(3)=0$
- $e(4)=ve(v_2)=6$
- $e(5)=ve(v_3)=4$
- $e(6)=ve(v_4)=5$
- $e(7)=ve(v_5)=7$
- $e(8)=ve(v_5)=7$
- $e(9)=ve(v_6)=7$
- $e(10)=ve(v_7)=16$
- $e(11)=ve(v_8)=14$

答：AOE网有两条关键路径：

- ①由活动 (a1,a4,a7,a10)组成的关键路径
- ②由活动 (a1,a4,a8,a11)组成的关键路径

(4)计算活动 a_i 的最迟开始时间 $l(i)=vl(k)-w_{j,k}$

- $l(11)=vl(v_9)-4=14$
- $l(10)=vl(v_9)-2=16$
- $l(9)=vl(v_8)-4=10$
- $l(8)=vl(v_8)-7=7$
- $l(7)=vl(v_7)-9=7$
- $l(6)=vl(v_6)-2=8$
- $l(5)=vl(v_5)-1=6$
- $l(4)=vl(v_5)-1=6$
- $l(3)=vl(v_4)-5=3$
- $l(2)=vl(v_3)-4=2$
- $l(1)=vl(v_2)-6=0$

示例

以下对于AOE网的叙述中，错误的是（ ）。

- A. 在AOE网中可能存在多条关键路径 ✓
- B. 关键活动不按期完成就会影响整个工程的完成时间 ✓
- C. 任何一个关键活动提前完成，整个工程也将提前完成 ✗
- D. 所有关键活动都提前完成，整个工程也将提前完成 ✓

“小算法” 解决 “大问题”

线性结构



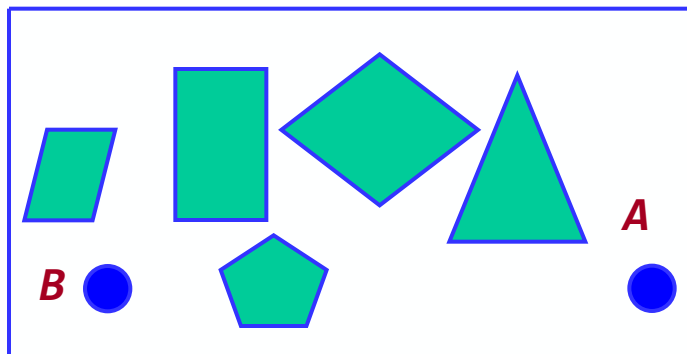
树形结构



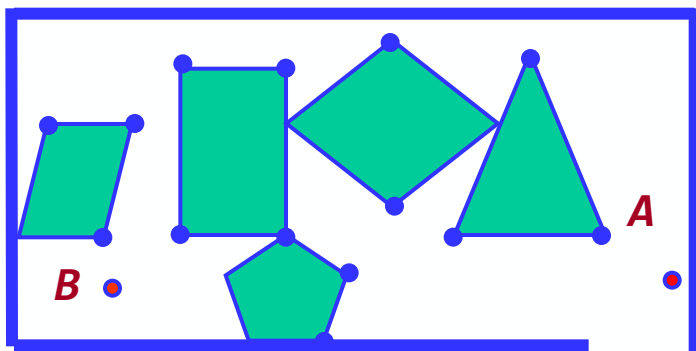
图形结构

- 线性结构可以看成是树形结构的特殊情况。
- 树形结构可以看成是图形结构的特殊情况。
- 图形结构是最普遍的一类数据结构，具有广泛的实际应用。

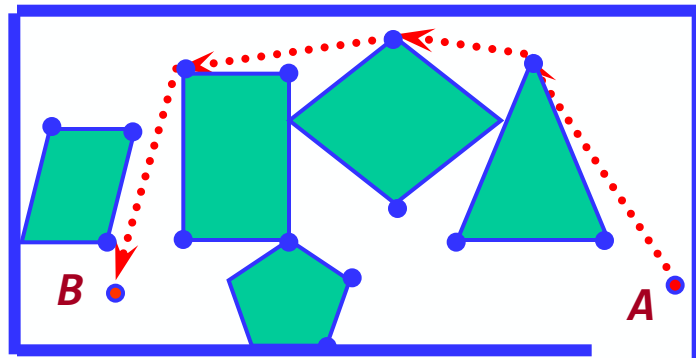
图应用实例1： 机器人路径规划问题



找路径: $A \rightarrow B$



① 膨胀所有物体。

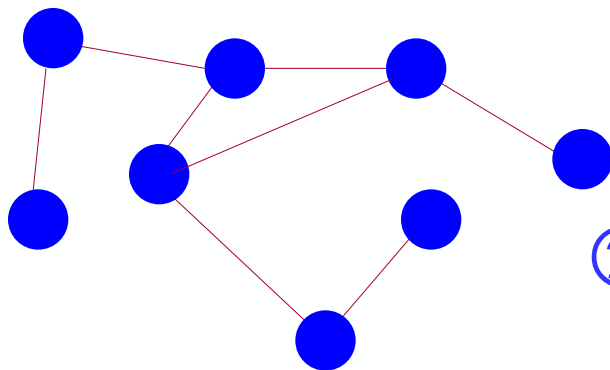


② 将路径搜索转换为图的顶点搜索。可采用图遍历算法。

图应用实例2: GIS求最短路径问题

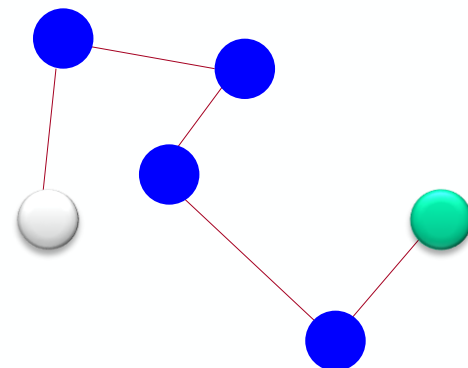


① 地图矢量化



② 图顶点搜索

求最短路径

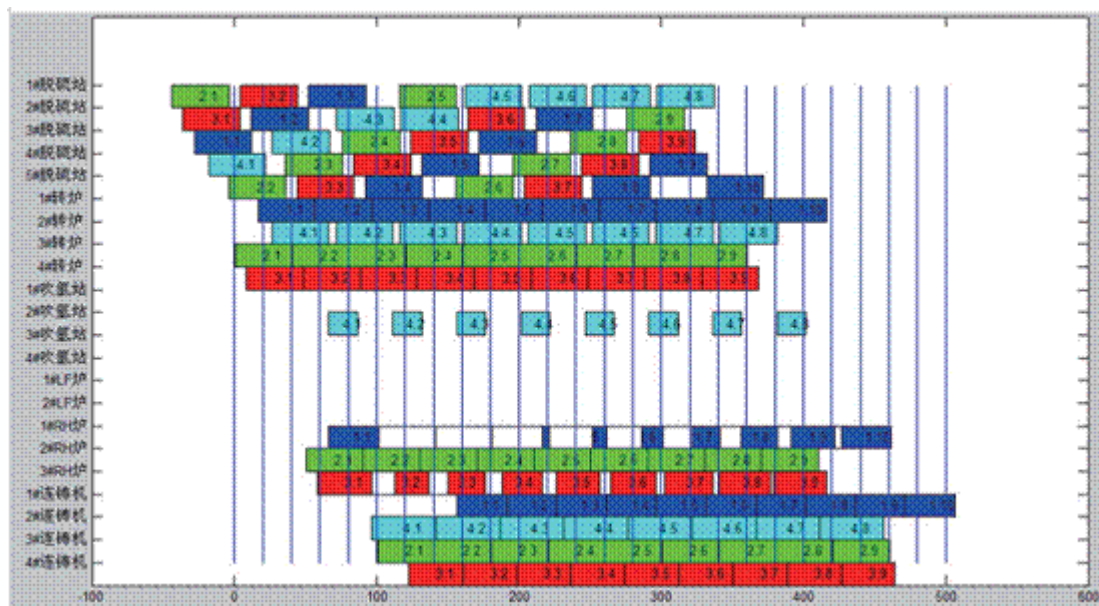


图应用实例3： 城市规划的管网设计



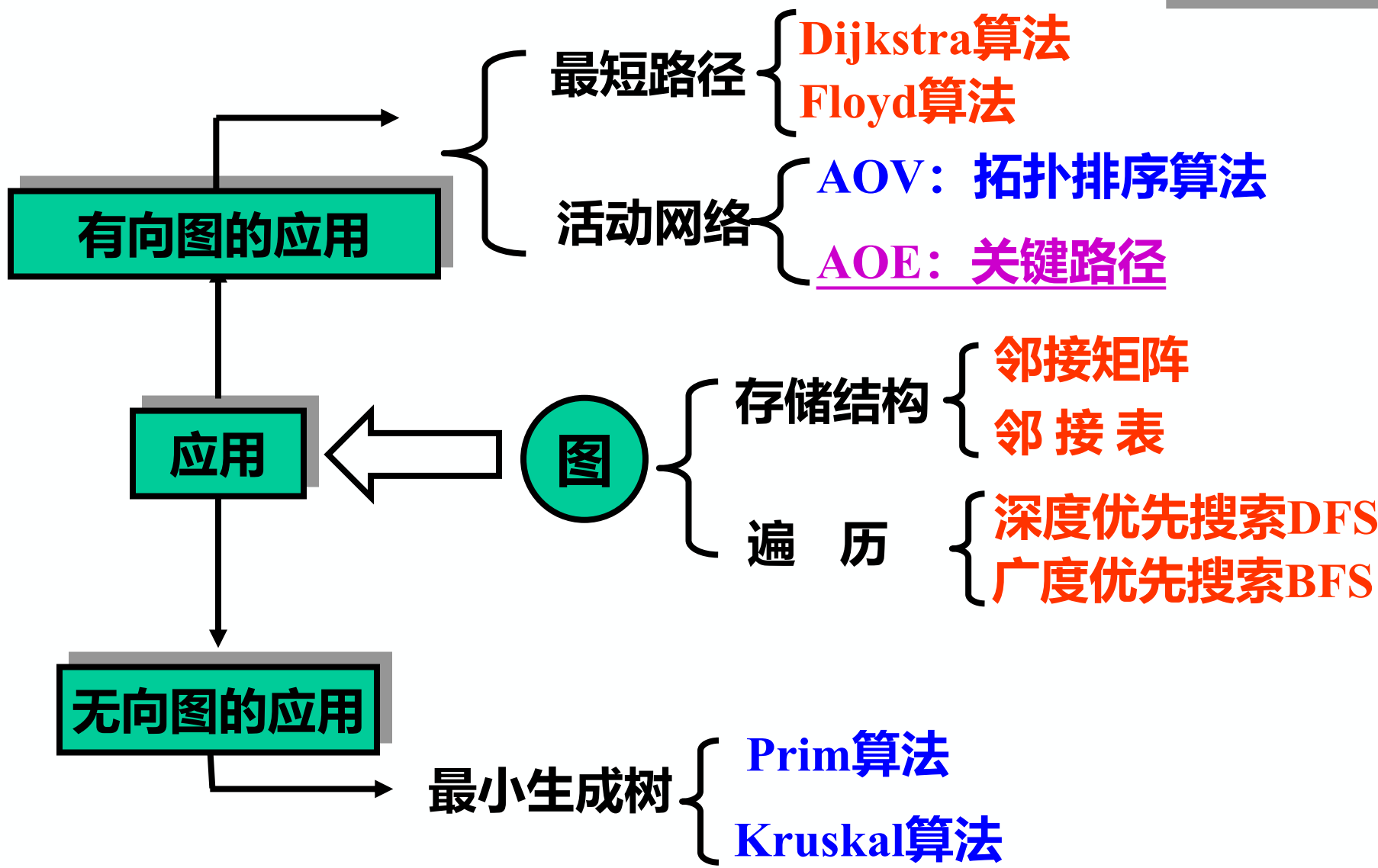
- 最小生成树
- 最短路径

图应用实例4： 生产进度的调度



● 求关键路径

许多实际问题 ⇒ 用图表示数据 ⇒ 用图相关算法求解



练习一下

1. 设无向图的顶点个数为 n ，则该图最多有（**B**）条边。
A. $n-1$ B. $n(n-1)/2$ C. $n(n+1)/2$ D. N
2. 一个 n 个顶点的连通无向图，其边的个数至少为（**A**）。
A. $n-1$ B. n C. $n+1$ D. $n \log n$
3. 要连通具有 n 个顶点的有向图，至少需要（**B**）条边。
A. $n-1$ B. n C. $n+1$ D. $2n$
4. n 个结点的完全有向图含有边的数目（**D**）。
A. $n*(n-1)/2$ B. $n(n+1)$ C. $n/2$ D. $n*(n-1)$

练习一下

5. 一个有 n 个结点的图，最少有（ **B** ）个连通分量，最多有（ **D** ）个连通分量。

A. 0 B. 1 C. $n-1$ D. n

6. 在一个无向图中，所有顶点的度数之和等于所有边数（ **B** ）倍，在一个有向图中，所有顶点的入度之和等于所有顶点出度之和的（ **C** ）倍。

A. $1/2$ B. 2 C. 1 D. 4

练习一下

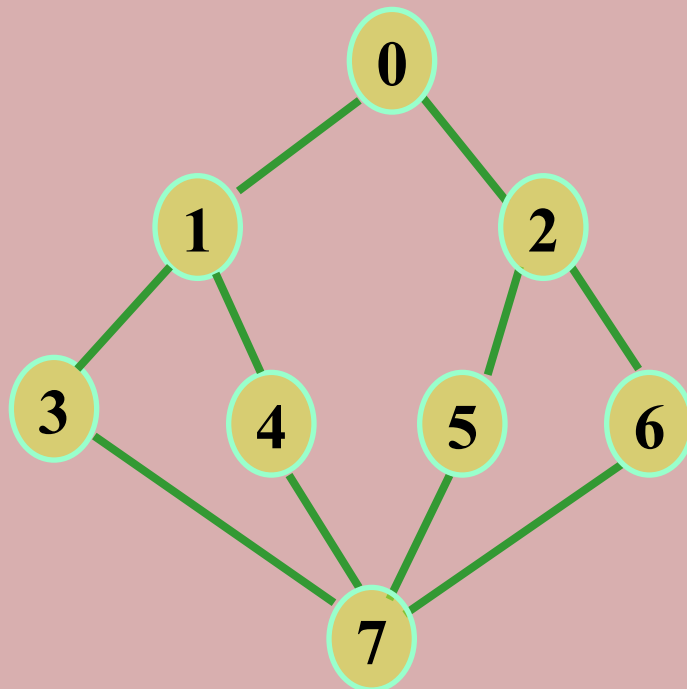
7. n 个顶点的连通图用邻接矩阵表示时, 该矩阵至少有 $2(n-1)$ 个非零元素。
8. 在图 G 的邻接表表示中, 每个顶点邻接表中所含的结点数, 对于无向图来说等于该顶点的度; 对于有向图来说等于该顶点的出度。
9. 在有向图的邻接矩阵表示中, 计算第 i 个顶点入度的方法是求矩阵第 i 列非零元素之和。
10. 对于一个具有 n 个顶点 e 条边的无向图的邻接表的表示, 则表头向量大小为 n , 邻接表的边结点个数为 $2e$ 。

练习一下

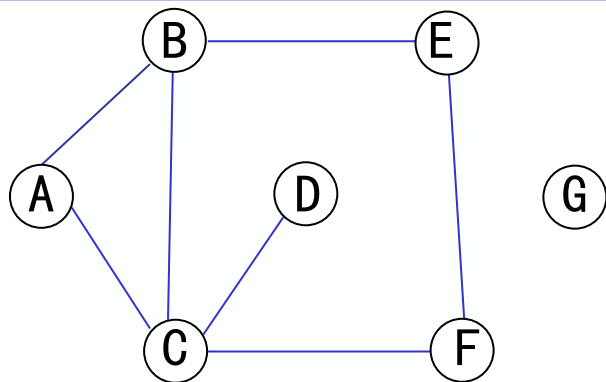
11. 对某个无向图的邻接矩阵来说， A 。
- A. 第 i 行上的非0元素个数和第 i 列的非0元素个数一定相等。
 - B. 矩阵的非0元素个数等于图中的边数。
 - C. 第 i 行上，第 i 列上非0元素总数等于顶点 v_i 的度数
 - D. 矩阵的非全0行的行数等于图中的顶点数。
12. 一个图中包含 k 个连通分量，若按深度优先搜索方法访问所有结点，则必须调用 A 次深度优先遍历算法。
- A. k B. 1 C. $k-1$ D. $k+1$
13. 在无向图 G 的邻接矩阵 A 中，若 $A[i][j]=1$ ，则 $A[j][i]=(1)$ 。

练习一下

14. 给出以下无向图的邻接矩阵和邻接表。



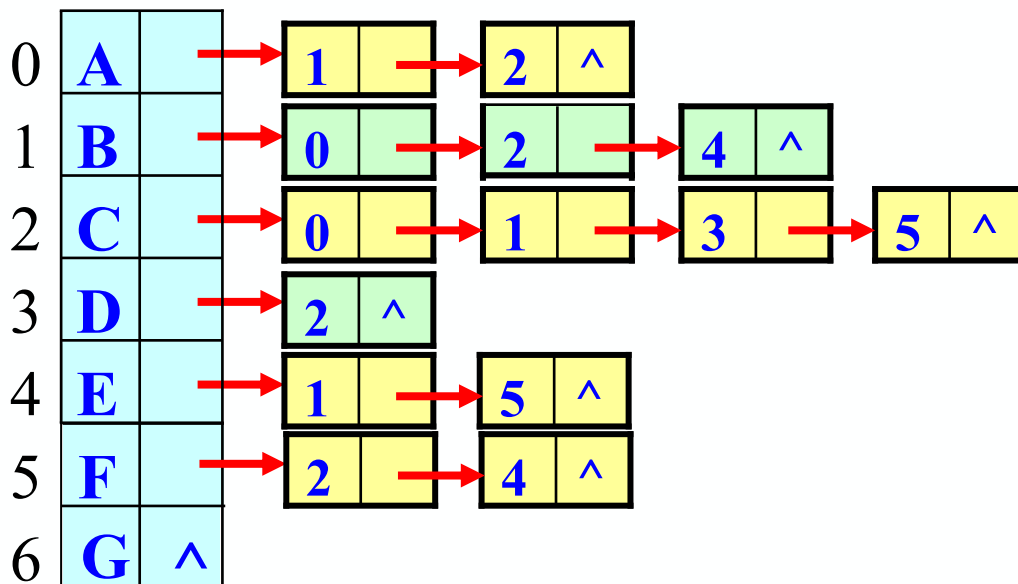
14. 设无向图G如下图：



试给出

- (1) 该图的邻接矩阵；
- (2) 该图的邻接表；
- (3) 根据邻接表给出从A出发的“深度优先”遍历序列；
- (4) 根据邻接表给出从A出发的“广度优先”遍历序列。

	A	B	C	D	E	F	G
A	0	1	1	0	0	0	0
B	1	0	1	0	1	0	0
C	1	1	0	1	0	1	0
D	0	0	1	0	0	0	0
E	0	1	0	0	0	1	0
F	0	0	1	0	1	0	0
G	0	0	0	0	0	0	0



深： ABCDFEG
广： ABCEDFG

练习一下

1. 已知无向图G, $V(G) = \{1, 2, 3, 4, 5, 6, 7\}$, $E(G) = \{(1, 2), (1, 3), (2, 3), (2, 4), (3, 4), (2, 7), (5, 7), (1, 7), (4, 5)\}$

给出其存储结构的邻接链表表示, 然后写出对其分别进行深度, 广度优先遍历的结果。

2. 从邻接矩阵A=

0	1	0
1	0	1
0	1	0

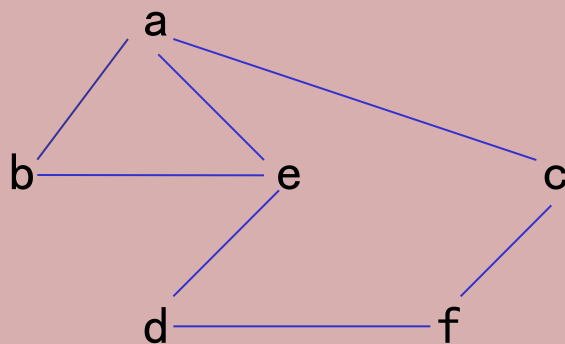
可以看出, 该图共有(**B**)个顶点。

若是有向图, 该图共有(**F**)条弧; 若是无向图, 则共有(**G**)条边。

A. 9 B. 3 C. 6 D. 1 E. 5 F. 4 G. 2 H. 0

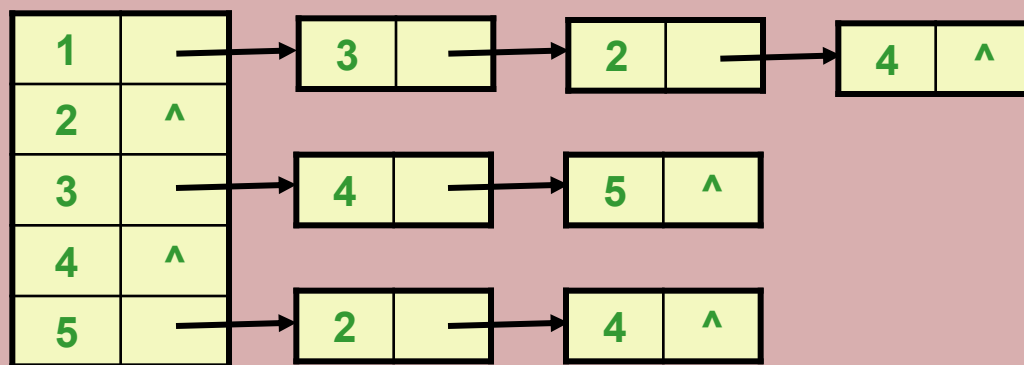
练习一下

3. 已知一个图如下所示，若从顶点a出发按深度优先搜索法进行遍历，则可能得到的一种顶点序列为①；按广度优先搜索法进行遍历，则可能得到的一种顶点序列为②。



- ① A. abecdf B. acfebd C. aebcfd D. aedfcb
② A. abcedf B. abcefd C. aebcfd D. acfdeb

4. 已知一有向图的邻接表存储结构如下所示，按有向图的深度优先搜索法进行遍历，从顶点1出发，所得到的顶点序列为①；按广度优先搜索法进行遍历，从顶点1出发，所得到的顶点序列为②。



- ① A. 12354 B. 12345 C. 13452 D. 14352
② A. 12345 B. 13245 C. 12354 D. 14352

练习一下

1. 对 n 个顶点的连通图来说，它的生成树一定有____条边。
2. 若含 n 个顶点的图形成一个环，则它有____棵生成树。
3. 任何一个无向连通图的最小生成树____。
A. 有1棵或多棵 B. 只有1棵 C. 一定有多棵 D. 可能不存在

练习一下

4. 采用Prim算法从顶点4开始构造出图 G_1 的一棵最小生成树。

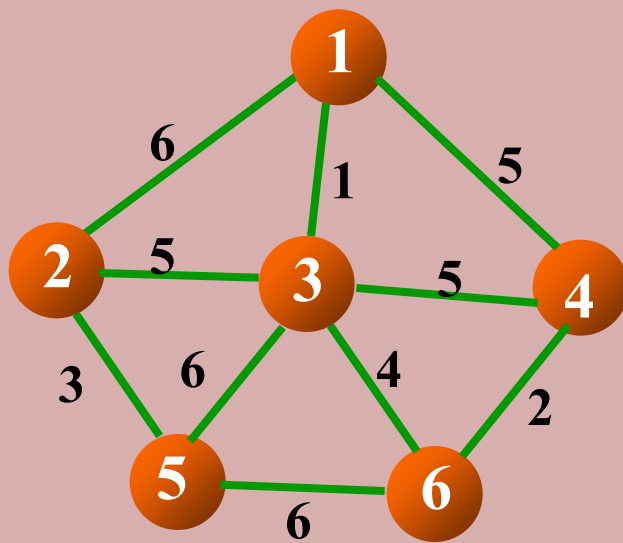
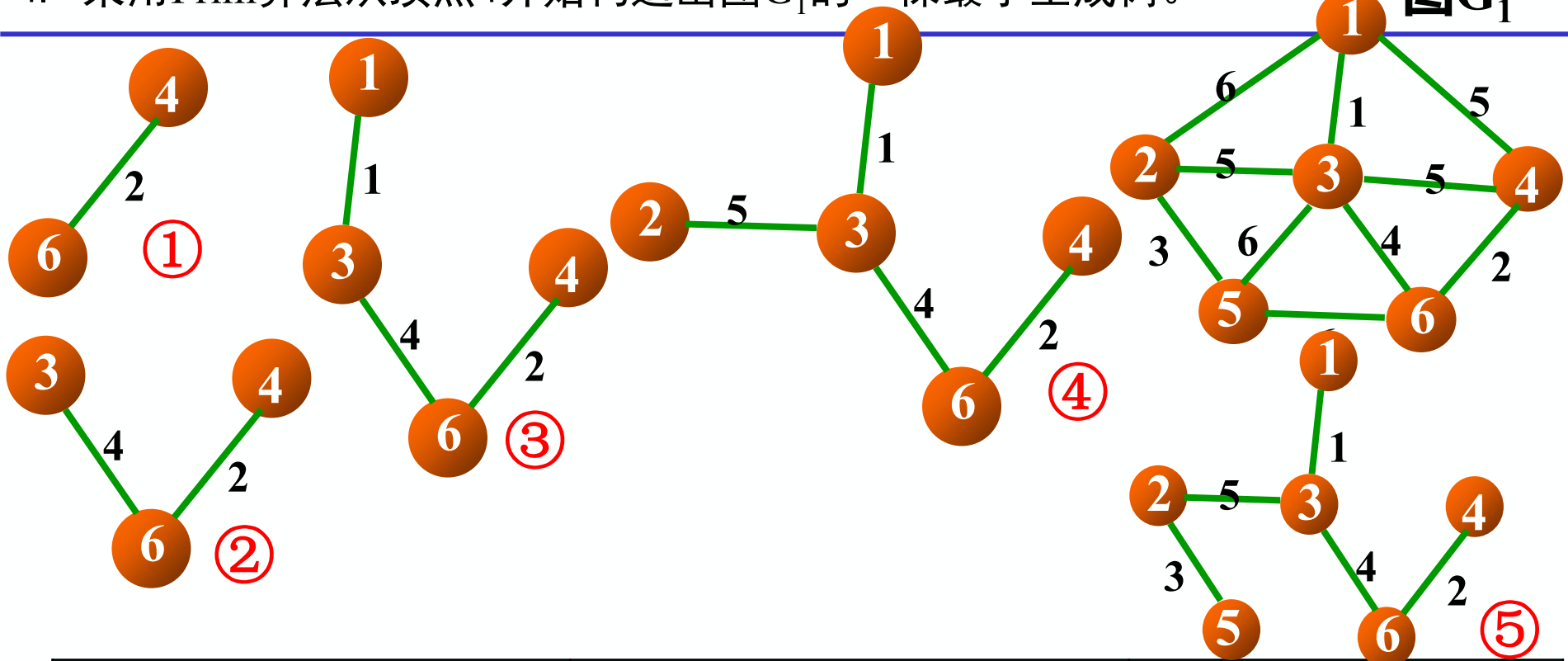


图 G_1

4. 采用Prim算法从顶点4开始构造出图G₁的一棵最小生成树。

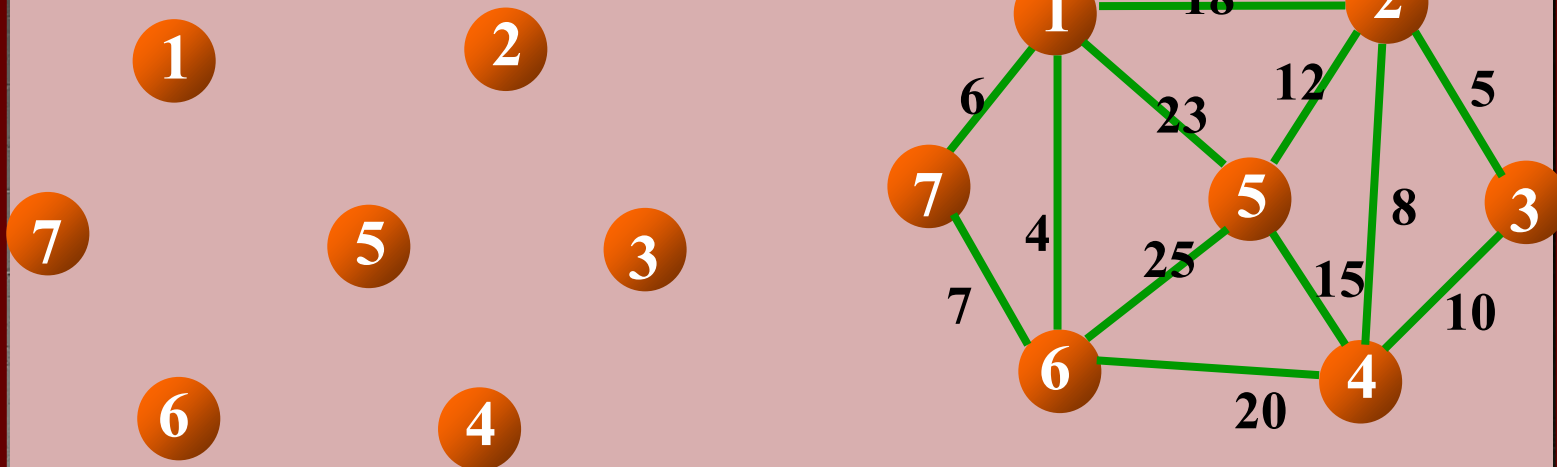
图G₁



U: 当前顶点	V-U : 剩余顶点	TE: 权值最小的边
{4}	{1,2,3,5,6}	{4,6}
{4,6}	{1,2,3,5}	{6,3}
{4,6,3}	{1,2,5}	{3,1}
{4,6,3,1}	{2,5}	{3,2}
{4,6,3,1,2}	{5}	{2,5}

练习一下

5. 采用Kruskual算法构造出图 G_2 的一棵最小生成树。

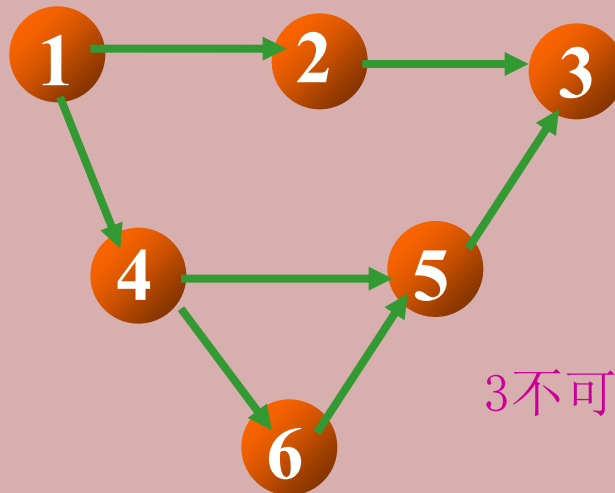


先把全部顶点加入，再按权值的递增次序选择不构成环的边。

练习一下

1. 已知一个图如图所示，则由该图得到的一种拓扑序列为_____。

- A. 146253 B. 123456 C. 142365 D. 124635



拓扑序列

1 2 4 6 5 3

1 4 2 6 5 3

1 4 6 5 2 3

1 4 6 2 5 3

3不可能在2和5的前面输出

2. 判定一个有向图是否存在回路，除了利用深度优先遍历算法以外，还可以用__**D**__。

- | | |
|-------------|---------------------|
| A. 求关键路径的方法 | B. 求最短路径的Dijkstra方法 |
| C. 广度优先遍历算法 | D. 拓扑排序方法 |

练习一下

1. 关键路径是事件结点网络中（ **A** ）。
A. 从源点到汇点的最长路径 B. 从源点到汇点的最短路径
C. 最长的回路 D. 最短的回路

2. 下列说法错误的是（ **A** ）。
A. 在AOE网中，减小任一关键活动上的权值后，整个工期也就相应减小。
B. AOE网工期为关键活动上的权之和。
C. 在关键路径上的活动都是关键活动，而关键活动也必须在关键路径上。

3. 下列说法错误的是（ **B** ）。

- A. 关键活动不按期完成就会影响整个工程的完成时间
- B. 任何一个关键活动提前完成，将使整个工程提前完成。
- C. 所有关键活动都提前完成，则整个工程提前完成。

4. 对下图所示的AOE图，求其关键路径。

